

Interview Questions & Answers

Jeetendra Kumar Patel — AI Engineer

163 questions across 15 parts · 51,055 words · September 2026

The questions recruiters, hiring managers and engineers ask about his record, with the answers his portfolio assistant gives. Every answer lists the facts it rests on. Live and searchable at jeetendra.vercel.app.

Contents

- Part 1** — Fit, Level and the Recruiter Screen (12 questions)
- Part 2** — The Venera Generative-AI Chatbot (12 questions)
- Part 3** — RAG and Retrieval (12 questions)
- Part 4** — Prompt Engineering (11 questions)
- Part 5** — Fine-Tuning and Model Adaptation (10 questions)
- Part 6** — LangChain and LangGraph (10 questions)
- Part 7** — Evaluation, Guardrails and Honesty About Failure (11 questions)
- Part 8** — Quality Engineering and Testing (10 questions)
- Part 9** — Production, Cloud and the SDLC (11 questions)
- Part 10** — Projects (10 questions)
- Part 11** — How This Website Works (10 questions)
- Part 12** — Behavioural Questions (11 questions)
- Part 13** — Hard and Sceptical Questions (11 questions)
- Part 14** — Skills, Tools and Stack (11 questions)
- Part 15** — Education, Trajectory and Logistics (11 questions)

Part 1 — Fit, Level and the Recruiter Screen

The first thing almost every visitor asks, in one form or another, is whether Jeetendra is worth an interview slot. These are those questions. The answers are deliberately honest about level — he is early-career, and a corpus that oversells him would be caught in the first twenty minutes of a real conversation, which costs everyone more than a straight answer would have.

Q1.1 — Is Jeetendra a fit for an AI Engineer role?

recruiter · asked by Recruiter, Hiring Manager · fit, hiring, ai-engineer, level

For a junior-to-mid AI Engineer role, yes, and the reason is narrower than "he knows LangChain".

He is the primary engineer on Venera Technologies' first generative-AI feature — a retrieval-augmented chatbot built on Qwen open-weight models that generates QC templates for Pulsar, their legacy on-premise product, from a plain-language description instead of manual configuration. He built it from scratch alongside an external AI consultant, and he owns both the implementation and the testing of it. That is production generative AI inside a real product with real customers, not a weekend demo.

What makes that unusual for someone at his stage is the order he came to it in. He spent six months in quality engineering on the same platform first — 150 automated Python test scripts, 120 validated REST endpoints across AWS microservices, zero critical defects across seven builds — and converted to full-time on that record. So when he builds an LLM feature, grounding, structured-output validation and an evaluation set are where he starts, not what he adds after the first bad output reaches a customer. On a legacy product, an invalid generated template is a support ticket.

Where he is genuinely early-career: he has not trained a foundation model, has not run a large distributed training job, and has about a year of professional experience. If the role needs someone to own an ML platform or set research direction, he is not that person yet.

If the role is building LLM-backed product features — retrieval, prompting, evaluation, integration, making it reliable enough to ship — that is what he is doing now, in production, and he can talk through the decisions rather than the vocabulary.

Follow-up

"What level would you slot him at?" — Junior to mid. He has roughly a year of professional experience, but with production ownership of an AI feature rather than ticket work, which is ahead of the median for that tenure.

Grounded in: Current role at Venera Technologies, Pulsar generative-AI chatbot, internship metrics, career summary

Q1.2 — What has he actually shipped with LLMs?

hiring-manager · asked by Hiring Manager, Engineer · llm, shipped, production, rag, qwen

One thing in production, and several built end to end outside it.

In production, at Venera Technologies: the Pulsar template-generation chatbot. A user describes the QC template they want in plain language; the system retrieves against the existing template corpus and schema, generates a candidate template grounded in those, validates the structured output against the schema, and refuses rather than guessing when the request is outside what it can produce. It runs on Qwen open-weight models specifically so the model runs inside the on-premise deployment rather than calling a third-party API — which is what that product and its customers require. He is the primary engineer on it and has owned it since August.

Built end to end outside it:

- A document Q&A chatbot over a 200-page knowledge base — LangChain orchestration, OpenAI API, Pinecone as the vector store, Streamlit front end. It cut retrieval time by roughly 70%.

- NLP pipelines on Hugging Face Transformers for classification and summarisation, with a prompt-evaluation loop that scores prompt variants against a hand-labelled question set so answer quality is measured rather than eyeballed.
- This website's assistant: a LangGraph agent over a 163-question corpus about his record, with embeddings-plus-BM25 retrieval measured at roughly 90% recall@3 on reworded questions, nine interchangeable model providers with automatic failover, and a deterministic check on every answer's numbers and citations. It started as a BM25 index over his resume, which is still there as the fallback and still has to pass its own 36-case evaluation.

The thread through all of them is that he treats output quality as something to measure. Every one of those has an eval set attached, which is unusual at this level and is the part worth probing in an interview — ask him what his eval sets *caught*, because the answers are specific.

Follow-up

"Which of those is the strongest signal?" — The Pulsar chatbot, because it shipped into a product with customers and constraints he did not choose. The rest he controlled end to end.

Grounded in: Pulsar generative-AI chatbot, Document Q&A Chatbot project, NLP Pipelines project, this portfolio assistant

Q1.3 — He has about a year of experience. Why should we interview him over someone with five?

sceptical · asked by Hiring Manager, Recruiter · level, experience, objection, hiring

If you need five years of experience, hire the person with five years. That is a real answer and the corpus is not going to argue you out of it.

The case for the interview is that the relevant experience in this field is not five years old. Production RAG, structured output from language models, open-weight model deployment and LLM evaluation are mostly a two-to-three-year-old discipline in industry. A candidate with five years of software experience and six months of LLM exposure is not obviously ahead of one with one year of software experience and eight months of owning an LLM feature in production.

The second argument is the one he would actually make. Most people arriving at AI engineering come from a data science or backend background and learn reliability engineering on the job, usually after shipping something that fails quietly. He arrived from the opposite direction: six months in quality engineering on a live AWS platform, where his job was specifically to find the failure before the customer did — 79 defects managed, a severity-1 503 intercepted before release, zero critical defects across seven builds. Applied to LLM systems, where the characteristic failure is a confident wrong answer rather than a stack trace, that instinct is the scarce part. The model is the easy half.

What you would be trading away is breadth and judgement that only comes from having seen more systems. He has not architected something from nothing at scale, has not owned a large service through a bad incident, and has a narrower set of reference points to reason from.

The honest framing: interview him if you want someone who will build the evaluation harness before the demo, and who has enough production AI experience to be useful in month one rather than month six.

Follow-up

"What would he need from the team to succeed?" — Code review from someone more senior, and exposure to systems at a larger scale than he has seen. He has not worked at high traffic volumes.

Grounded in: Career summary, internship metrics, Pulsar chatbot ownership, quality engineering background

Q1.4 — What kind of role is he actually looking for?

recruiter · asked by Recruiter · role, preferences, availability, hiring

AI Engineer or ML Engineer roles, with a preference for ones where the work is building LLM-backed product features rather than research.

Concretely, the work he is best matched to: retrieval systems and RAG pipelines, LLM application engineering — prompting, grounding, structured outputs, integration — and the evaluation and guardrail layer that makes those reliable enough to put in front of users. That is what he does at Venera now and what every project outside work has been about.

He is open to roles that are partly platform or infrastructure, because his background is production delivery on AWS and he is comfortable with CI/CD, release readiness and the operational side. He is not a research engineer and is not the right hire for a role that expects novel modelling work or publications.

He is currently based in Noida, Uttar Pradesh, and is open to AI Engineer / ML Engineer roles. For anything beyond that — notice period, compensation expectations, location or remote preferences, work authorisation — the resume does not cover it and this assistant will not guess. Those are worth asking him directly at jeetendrapatel1711@gmail.com.

One thing worth knowing for fit: he has a documented preference for measuring rather than asserting. Every project he has built has an evaluation set attached, and the write-up of this website is mostly about trade-offs and mistakes rather than features. In a team that values that, he will be a good cultural match. In a team that wants someone to move fast and sort out correctness later, there will be friction.

Follow-up

"Is he available now?" — He is open to AI Engineer / ML Engineer roles and is currently employed at Venera Technologies. Specific timing is a question for him directly.

Grounded in: Availability note, location, focus areas, current role

Q1.5 — Summarise his background in thirty seconds.

recruiter · asked by Recruiter, Hiring Manager · summary, background, elevator-pitch

Jeetendra Kumar Patel is an AI Engineer based in Noida, currently Associate Software Engineer at Venera Technologies.

He joined Venera in February 2026 as a graduate trainee on the QA side of their media quality control platform, working across Quasar (the AWS cloud product) and Pulsar (the on-premise one). Over six months he wrote 150 automated Python test scripts with Selenium and the Page Object Model, cutting manual regression time by 65%; validated 120 REST API endpoints across AWS microservices at a 98.3% pass rate; and shipped seven consecutive builds with zero critical defects reaching production, including intercepting a severity-1 failure and coordinating a same-day hotfix. That converted into a full-time offer in August.

Since August his primary work has been Venera's first generative-AI feature: a retrieval-augmented chatbot built on Qwen open-weight models that generates QC templates for Pulsar from plain language. He built it from scratch with an external AI consultant and owns both the implementation and the testing.

Alongside that he has built a document Q&A chatbot on LangChain, OpenAI and Pinecone that cut retrieval time roughly 70% over a 200-page corpus, NLP pipelines on Hugging Face Transformers with a measured prompt-evaluation loop, and the assistant answering this question — a LangGraph agent over about a hundred pages of questions and answers about his record, with measured retrieval, versioned prompts and nine interchangeable model providers.

He holds a B.E. in Computer Science from Chandigarh University, graduated June 2026, and is working through the AI Engineer Bootcamp 2026.

The one-line version: a quality engineer who moved into applied AI and kept the habits — he ships LLM features with evaluation sets attached.

Follow-up

"What is the single strongest thing on there?" — Owning a production generative-AI feature end to end, implementation and testing, less than a year into his career.

Grounded in: Full career history, education, current role, projects

Q1.6 — Is the AI experience real, or is it a bootcamp and some side projects?

sceptical · asked by Hiring Manager, Engineer · credibility, objection, production, bootcamp

It is a fair thing to check, because the pattern you are describing is extremely common right now.

The part that is not a side project: the Pulsar template-generation chatbot at Venera. It is a feature in a commercial product, he is the primary engineer on it, it was built from scratch, and he owns the testing as well as the implementation. It has constraints he did not get to pick — it runs on Qwen open-weight models specifically because the model has to run inside an on-premise deployment rather than calling an external API, which is a requirement of that product's customers. Side projects do not come with that kind of constraint, and working within it is most of the engineering.

The part that is training: the AI Engineer Bootcamp 2026 through 365 Careers, which is in progress and covers LLMs, NLP, LangChain, LangGraph, Hugging Face, Pinecone, the OpenAI API, RAG and Streamlit. He would describe that as where he learned the vocabulary, not where he learned the job.

The part that is side projects: the document Q&A chatbot, the Hugging Face NLP pipelines, and this website's assistant. These are real and finished rather than tutorials — the portfolio assistant has retrieval and prompt evaluations with published results, nine provider integrations with automatic failover, and a published log of twelve bugs that shipped broken before being caught. But they are his own work on his own terms, and they should carry less weight than the production feature.

The way to test it in an interview is to ask about failures rather than features. Ask what his eval set caught that he had not predicted. The answers are specific, which is hard to fake.

Follow-up

"What would you ask to verify it?" — Ask him to walk through why the Pulsar chatbot uses open-weight models, and what the validation layer does when the model produces a template that does not fit the schema. Both answers are specific to that system.

Grounded in: Pulsar chatbot, AI Engineer Bootcamp certification, portfolio assistant, project list, build log

Q1.7 — What are his strongest measurable results?

recruiter · asked by Recruiter, Hiring Manager · metrics, results, impact, numbers

The numbers on his resume, with the context that makes them mean something:

- **150 automated Python test scripts** built with Selenium and the Page Object Model, which cut manual regression time by 65% and reached 78% coverage of P1/P2 cases. Three critical regressions were caught pre-release as a direct result.
- **120 REST API endpoints validated** across AWS microservices — 75 on Quasar, the cloud QC product, and 45 on Pulsar, the on-premise one — at a 98.3% pass rate, with two production-blocking defects resolved before deployment.

- **Zero critical defects reaching production across seven consecutive builds.** Within that, he intercepted a severity-1 503 failure and coordinated a same-day hotfix, avoiding roughly four hours of user-facing downtime.
- **79 defects managed in Zoho**, 61 resolved, 10 of them critical, with test assets maintained in Bitbucket under structured PR review.
- **Roughly 70% reduction in retrieval time** on the document Q&A chatbot, over a 200-page knowledge base, using LangChain with Pinecone.
- **36 out of 36 on the resume-index retrieval eval** on this website, across 29 questions that must retrieve the right resume section and 7 that must retrieve nothing at all.
- **About 90% recall@3 on reworded questions** over the site's interview corpus, against about half for keyword search alone — measured on questions written the way visitors actually type.

Two caveats he would offer himself. The generative-AI work at Venera is the most significant thing he has done and it is the one item on this list without a headline number attached — it shipped recently and the measurement that exists is the internal evaluation set, not a public metric. And the "zero critical defects" figure is a team outcome he contributed to, not a solo achievement; he was the person doing the test coverage, not the only person responsible.

Follow-up

"Which number is he proudest of?" — The severity-1 interception, because it is the one where the counterfactual is concrete: without it, roughly four hours of customer-facing downtime happened instead.

Grounded in: Headline metrics, internship achievements, Document Q&A project, portfolio assistant eval

Q1.8 — Does he have production experience, or only project experience?

hiring-manager · asked by Hiring Manager, Engineer · production, experience, aws, release

Production, and it is the more established half of his record.

He has worked on Venera's media quality control platform since February 2026 — first as a graduate trainee and QA engineer, then full-time from August. That platform has two products: Quasar, which is cloud-native on AWS, and Pulsar, which is deployed on-premise at customer sites. He has worked across both.

What "production" means concretely in his case:

- He has owned release readiness, not just written code. He worked with dev and product teams to define acceptance criteria and review test strategies, and the defect escape rate came down sprint over sprint as a result.
- He has handled a live incident. A severity-1 503 failure was intercepted and a same-day hotfix coordinated, which is a different experience from building something that has never been under load.
- He has worked inside the constraints of a legacy on-premise product, which is where the generative-AI chatbot lives. That deployment cannot call out to a third-party model API, which is why the feature runs on Qwen open-weight models. Designing around a constraint like that is production engineering.
- He has worked in a real SDLC — Bitbucket with structured PR reviews, defect tracking in Zoho, CI/CD, Jenkins.

Where the production experience is thin: he has not operated a high-traffic service, has not been on call for a large system, and the scale he has worked at is a commercial media QC platform rather than consumer-scale infrastructure. If a role depends on experience at high QPS or with large distributed

systems, he does not have that yet.

Follow-up

"Has he deployed an LLM system to production himself?" — Yes, the Pulsar template chatbot, running on open-weight models inside an on-premise deployment. That is the one.

Grounded in: Current role, internship at Venera, Quasar and Pulsar products, severity-1 incident, Pulsar chatbot

Q1.9 — Why did he move from QA into AI engineering?

hiring-manager · asked by Hiring Manager, Recruiter · career, motivation, transition, qa

The short version is that the move happened at work rather than away from it — Venera needed someone to build their first generative-AI feature and he was the person who took it on, which is a better transition story than leaving to do a course.

The longer version is about what QA taught him that turned out to be the scarce skill. Six months of quality engineering on a live platform is six months of learning that a system is not done when it produces output — it is done when you can show it produces the *right* output. He watched a severity-1 failure get caught an hour before release because someone bothered to check.

That instinct maps unusually well onto LLM systems, because their characteristic failure is a confident wrong answer rather than a crash. There is no stack trace. Nothing goes red. The only thing that catches it is having decided in advance what correct looks like and measuring against it — which is exactly what a test strategy is. The generative-AI work he does now is, from his side, the same discipline applied to a system that fails more quietly.

What he was moving *toward* rather than away from: he wanted to build the thing rather than verify it. The Pulsar chatbot is the first time he has owned a feature's implementation and not just its quality, and that is the direction he wants to keep going.

He is also clear-eyed that the QA background is a genuine asset rather than something to explain away, which is why it stays prominent on the resume rather than being buried.

Follow-up

"Does he want to leave testing behind entirely?" — No. He still owns the testing of the chatbot he built, and he describes that as the right arrangement rather than a leftover obligation.

Grounded in: Career transition narrative, internship background, Pulsar chatbot ownership, About section

Q1.10 — What is he learning right now?

recruiter · asked by Recruiter, Hiring Manager · learning, growth, certifications, langgraph

Three things, with different levels of depth.

Formally: the AI Engineer Bootcamp 2026 through 365 Careers on Udemy, currently in progress. It covers LLMs, NLP, LangChain, LangGraph, Hugging Face, Pinecone, the OpenAI API, RAG and Streamlit.

In practice at work: open-weight model deployment. The Pulsar chatbot runs on Qwen models inside an on-premise product rather than calling a hosted API, and everything that follows from that constraint — what the model can do at that size, how to ground it well enough that the smaller model is sufficient, how to validate its structured output — is what he is actively learning by doing.

On his own: LangGraph, for multi-agent and stateful AI workflows. This is the one he would describe as genuinely in progress rather than learned. He has used LangChain in a shipped project and is extending toward LangGraph from there.

He also maintains a deliberate habit of building evaluation into whatever he learns. The Hugging Face NLP work came with a prompt-evaluation loop that scores prompt variants against a hand-labelled

question set; this website's assistant came with retrieval and prompt evaluations. That pattern — learn a technique, then build the thing that tells you whether it is working — is consistent enough across his projects to be a real trait rather than a coincidence.

Previously completed: Android Developer Badges from Google, Database Fundamentals and Design Thinking from LinkedIn.

Follow-up

"Is he strong on LangGraph or just familiar?" — Familiar and actively learning. He has shipped with LangChain; LangGraph he is still building with. Worth asking him directly rather than assuming depth.

Grounded in: Certifications, Pulsar chatbot on Qwen, NLP Pipelines project, portfolio assistant eval, skills list

Q1.11 — What is he not good at?

sceptical · asked by Hiring Manager, Engineer · weaknesses, gaps, honesty, limits

The gaps that matter for an AI engineering role, stated plainly:

Scale. He has not worked on a high-traffic system. The platform he works on is a commercial media QC product, not consumer infrastructure, and he has not had to reason about thousands of requests per second, sharding, or the operational problems that only appear at volume.

Model training. He has not trained a model from scratch, has not run a distributed training job, and has not done a significant fine-tune. He has used fine-tuning conceptually and works with open-weight models at inference, but if a role needs someone to own a training pipeline, he is not there.

Breadth of systems seen. About a year of professional experience means a small number of reference points. Senior engineers are largely valuable because they have watched many things fail in many ways; he has watched a few things fail well.

Depth outside Python. Python is his primary language by a wide margin. He has Java, Kotlin, C and C++ on the resume and has used them — the Android internship was Kotlin, and there is a Java servlet project — but he would not claim current depth in any of them.

Front-end and design. He builds interfaces that work; he is not a front-end specialist.

The one he would flag himself, because it is the most relevant: the generative-AI work is recent. He has owned a production AI feature since August, which is real but short. He has not yet had the experience of maintaining an LLM system through model drift, a provider change, or a year of accumulated edge cases — and he knows that is where the hard parts of this job actually live.

Follow-up

"Is he honest about this in an interview, or only in a scripted answer?" — The build log on this site lists twelve bugs that shipped broken before he caught them, including one where documenting a bug reintroduced it. The disposition seems consistent.

Grounded in: Experience level, skills list, build log, current role tenure, career summary

Q1.12 — Would he suit a startup or a larger company better?

recruiter · asked by Recruiter, Hiring Manager · fit, culture, startup, enterprise

Either can work, and the record suggests where each would stretch him.

What suits a startup. He works independently: he was handed a feature with no internal precedent and owns both its implementation and its testing, and everything outside his job — including this site, with a few thousand lines of retrieval, provider integration, evaluation and front end — he built because he wanted to. He is comfortable across the stack, from retrieval code to deployment configuration to the stylesheet. And he is cost-conscious by habit: this site runs entirely on free tiers because it was designed to, not by accident.

What would stretch him at a startup. His strongest habit is building the measurement before the feature. On a team that needs to ship something rough on day two and learn from real users, that instinct can cost a week. He also has less experience than a startup's first engineers usually need with infrastructure he would have to own outright.

What suits a larger company. He came up through a real SDLC — acceptance criteria, release readiness, structured code review, defect management — on a commercial product with customers. He is used to working inside constraints he did not choose, which is most of what engineering at a larger company involves, and his evaluation-first discipline is exactly what a team shipping AI to many users needs.

What would stretch him at a larger company. Scale he has not seen, and operating inside a big organisation's process at more senior levels.

The honest summary: he would probably do best somewhere that ships LLM features to real users, values correctness, and has experienced engineers around him — which describes some startups and some larger teams. The company's size matters less than whether that is true.

Follow-up

"What does he say he wants?" — Exposure to larger-scale systems and code review from senior engineers. Ask him how he weighs those against scope and ownership.

Grounded in: Pulsar feature ownership, self-directed projects, free-tier design, SDLC experience, eval-first habit, stated wants

Part 2 — The Venera Generative-AI Chatbot

This is the work most visitors end up asking about, so it gets the most space. Everything here is described at the level of problem, constraint and engineering reasoning. Venera's template schema, customer names and internal architecture specifics are deliberately absent — it is their product, and a public portfolio is not the place for them. If you need that depth, ask him directly and he can talk about it under the appropriate terms.

Q2.1 — Walk me through the generative-AI chatbot he built at Venera.

hiring-manager · asked by Hiring Manager, Engineer · pulsar, chatbot, rag, qwen, flagship, on-premise

The problem: Pulsar is Venera's on-premise media quality control product. To use it, an operator configures a QC template — the set of checks a piece of media gets run through. Building one by hand means knowing the schema and picking through options, which is slow and is the kind of task where people copy an existing template and edit it until it roughly works.

The feature: describe the template you want in plain language, and the system generates it.

The architecture, at the level he would describe it publicly: it is a retrieval-augmented generation pipeline. The user's description is used to retrieve relevant material from the existing corpus of templates and the schema that defines what a valid template can contain. That retrieved material is assembled into context for a Qwen model, which generates a candidate template. The candidate is then validated programmatically against the schema before anything is shown or used, and a template that does not validate does not get through.

Retrieval is embedding-based over a vector store rather than keyword matching, because operators describe what they want in their own words rather than in schema terminology — the gap between "check this will play on broadcast" and the specific fields that encode it is a semantic gap, not a lexical one.

The whole thing runs inside the customer's deployment. No outbound API calls. That single constraint drove most of the decisions and is covered in the next question.

He is the primary engineer on it, built it from scratch alongside an external AI consultant, and owns both the implementation and the testing — including the evaluation set it is scored against.

Follow-up

"How long has it been running?" — He has owned it since August 2026, when he converted to full-time. It is his primary work.

Grounded in: Pulsar generative-AI chatbot, current role at Venera Technologies, Qwen models, on-premise constraint

Q2.2 — Why Qwen and open-weight models rather than an API like OpenAI or Claude?

engineer · asked by Engineer, Hiring Manager · qwen, open-weights, on-premise, architecture, constraints

It was not a preference, it was a requirement, and that distinction matters because it is what makes the project interesting.

Pulsar is deployed on-premise at customer sites. For the customers who run it that way, the reason they run it that way is that their media and their workflows do not leave their infrastructure. A feature that took an operator's description of their QC requirements and posted it to a third-party API would violate the premise of the product they bought. In some deployments there is no outbound internet route at all.

So the model had to run inside the deployment, which rules out every hosted API and puts you in open-weight territory. Qwen was the family chosen.

What follows from that constraint is most of the engineering:

- **You cannot reach for a bigger model when quality is short.** With an API the escape hatch is to upgrade the tier. Here the model is fixed by what the customer's hardware can run, so quality has to come from retrieval and grounding instead. Getting the right context in front of a smaller model does more than a larger model with worse context would.
- **Retrieval has to be local too.** No hosted embedding API, so the embedding model ships with the product and the vector store runs alongside it.
- **Validation matters more, not less.** A smaller model is more likely to produce a well-formed-looking template that is subtly wrong, so the schema validation layer is load-bearing rather than a safety net.
- **Versioning is your problem.** There is no provider silently updating the model, which is good for reproducibility and means upgrades are a deliberate release decision.

The honest upside: it forced a design where the system does not depend on the model being excellent, which is a better design regardless.

Follow-up

"Would he have chosen open weights without the constraint?" — For this feature, probably yes, for the versioning and reproducibility reasons. But he would not claim the constraint was his idea; it came from the product.

Grounded in: Pulsar on-premise deployment, Qwen open-weight models, current role

Q2.3 — How does he stop it generating an invalid template?

engineer · asked by Engineer, Hiring Manager · validation, structured-output, guardrails, schema, reliability

By not trusting the model, structurally.

The generated template is validated against Pulsar's template schema before it can be used. If it does not validate, it does not get through. That is the guarantee, and it is enforced in code rather than requested in a prompt — which is the whole point. A prompt that says "only produce valid templates" is a preference the model will usually honour. A validator is a property the system has.

The layers, in order:

1. **Retrieval grounds the generation.** The model is given real templates and the relevant parts of the schema, so it is completing a pattern that exists rather than inventing structure from its own priors. Most malformed output comes from the model having to guess at shape. 2. **Constrained, structured generation.** The output is asked for in a defined structure rather than as prose that gets parsed afterward. 3. **Schema validation.** The hard gate. Field names, types, required fields, allowed values. 4. **Rejection over repair-at-any-cost.** If validation fails, the system does not quietly patch the output into something that passes. A bounded retry is reasonable; silently coercing a wrong template into a valid-looking one is worse than failing, because the operator then runs QC with checks they did not ask for.

The failure mode he is most concerned with is not malformed output — that is caught. It is *well-formed and wrong*: a template that validates perfectly and encodes a different intent than the operator described. Schema validation cannot see that, and it is the reason the evaluation set exists and includes expected outputs rather than only checking that something parseable came back.

This is also where the QA background shows up most directly. The instinct to ask "what does this do when it is wrong" before asking "how good is it when it is right" is the same instinct that found a severity-1 failure before release.

Follow-up

"What happens if it cannot produce a valid template?" — It refuses rather than guessing. The evaluation set includes requests it is expected to decline, which is the half of the eval that people skip.

Q2.4 — What does the evaluation set for the chatbot look like?

engineer · asked by Engineer, Hiring Manager · evaluation, testing, eval-set, quality

It has two halves, and he would say the second half is the one that matters.

The must-generate half: descriptions paired with the template that should come out of them. These are the cases where a correct answer exists and the system either produces it or does not. Scoring them tells you whether a change to retrieval, the prompt or the model helped or hurt.

The must-refuse half: requests the system is expected to decline. Things outside what a QC template can express, requests that are too vague to produce a specific template, and things that are simply not what the feature is for. Without this half, an eval rewards a system that always produces *something*, which is exactly the behaviour you do not want — a confident template for an ambiguous request is worse than a refusal, because the operator has no signal that it guessed.

He holds the same view on his own portfolio assistant, where the eval is 29 must-match cases plus 7 must-decline cases, and the must-decline half was added only after it answered an unrelated personal question by padding it with his career summary. That failure is why he builds the refusal half in from the start now.

Why an eval set at all, rather than trying things and looking at the output: because with a generative system you cannot tell whether a change helped by reading a few examples. The variance between runs is large enough that two or three samples will confirm whatever you already believed. A scored set turns "this feels better" into a number that moved or did not.

It also protects against the specific danger of this kind of work: retrieval and prompt changes have non-local effects. Improving the context for one class of request frequently degrades another, and nothing tells you unless you are measuring both.

Follow-up

"Who wrote the eval set?" — He did. He owns the testing side of the feature as well as the implementation.

Grounded in: Chatbot evaluation set, portfolio assistant eval (36 cases), build log bug 04

Q2.5 — What was the hardest part of building it?

engineer · asked by Engineer, Hiring Manager · challenges, design, retrieval, constraints

The vocabulary gap between how operators describe what they want and how the schema expresses it.

An operator asks for something in the language of their job — what they are checking for, what the media is for, what they are worried about. The template schema is a structured configuration of specific checks with specific parameters. Those two vocabularies barely overlap in surface terms, which is precisely why keyword retrieval was not going to work and the retrieval layer is embedding-based.

That is the same class of problem he hit on his own portfolio assistant from the other direction: recruiters ask "can he do X" while a resume says "built X", and closing that gap took stemming, phrase normalisation and a weighted synonym layer rather than a better model. He would say recognising it as a retrieval problem rather than a generation problem is the transferable part — the instinct when output is wrong is to improve the prompt, and often the prompt is fine and the model was handed the wrong context.

The second hard part, related: working within a fixed model. With a hosted API, a quality shortfall has an expensive but easy answer — use a better model. Running inside an on-premise deployment means the model is whatever the customer's hardware supports, so every improvement has to come from retrieval quality, grounding and validation. That is more constrained and, he would argue, better

engineering practice, because it forces the parts that actually generalise.

The third, which is less technical: deciding what the system should refuse. It is easy to specify what a feature should do and much harder to draw the line where it should stop and say it cannot help. That line is a product decision as much as an engineering one, and getting it into the evaluation set meant making it explicit rather than leaving it to the model's judgement.

Follow-up

"What would he do differently?" — Worth asking him directly; the answers to that are the most specific thing he can offer about the project and are not reduced to a summary well.

Grounded in: Pulsar chatbot retrieval design, portfolio assistant vocabulary-gap work, on-premise model constraint

Q2.6 — He worked with an external AI consultant. What did he actually do himself?

sceptical · asked by Hiring Manager, Recruiter · ownership, consultant, credit, collaboration

A fair question, and the resume says "with an external professional AI consultant" rather than hiding it, which is the right way round.

The division as he describes it: he is the primary engineer on the feature and owns the implementation and the testing. The consultant was engaged as expertise Venera did not have in-house — this is the company's first generative-AI feature, so there was no internal precedent for how to approach it.

What that means in practice is that he was the person building it day to day and the person responsible for it working, with access to someone more experienced to check direction against. Which is, frankly, a good arrangement for someone's first production AI feature, and a better signal than "built it alone" would be — a company putting a junior engineer on a first-of-its-kind feature with expert support and having them own it is a company that thought about how to make it succeed.

The parts that are unambiguously his: the implementation, the evaluation set, and the testing strategy. The testing half in particular is where his prior six months of quality engineering on the same platform was the reason he was the right person for it — he already knew the product, the schema domain and the release process.

What he would not claim: that he independently chose the overall approach with no input, or that he has the depth of someone who has built many of these. This is his first production generative-AI system and he has been on it since August.

The way to calibrate it in an interview is to ask him about a decision he made that the consultant disagreed with, or a problem he solved after the consultant's involvement ended. Those questions separate ownership from attendance.

Follow-up

"Would he be able to build the next one without a consultant?" — Worth asking him. He has been through one full cycle of it now, including the parts that went wrong.

Grounded in: Pulsar chatbot built with external AI consultant, implementation and testing ownership, prior QA work on same platform

Q2.7 — How does the retrieval part of the chatbot work?

engineer · asked by Engineer · rag, retrieval, embeddings, vector-store

It is embedding-based retrieval over a vector store, run locally alongside the model.

The corpus being retrieved over is the existing body of templates plus the schema material that defines what a valid template can contain. When an operator describes what they want, that description is embedded and used to find the most relevant existing templates and schema sections, which become the context the model generates from.

Embeddings rather than keyword search, for the reason covered earlier: operators describe intent in their own vocabulary and the schema uses its own. Lexical overlap between those is poor, so BM25-style matching would miss the right template while confidently returning one that shared a few words. This is the mirror image of the choice he made on his portfolio assistant, where the corpus is thirty short chunks and lexical retrieval is the *right* call — the technique follows from the corpus, not from fashion.

Everything runs inside the deployment, so the embedding model ships with the product rather than being a hosted API call. That is a constraint with a real cost — you are limited to embedding models small enough to run there — and it is another reason the validation layer carries weight.

What he would emphasise about the design is that retrieval quality, not model quality, is the lever he actually has. With the model fixed by the deployment, the difference between a system that works and one that does not is almost entirely whether the right template ends up in the context window. That makes retrieval the thing worth measuring and the thing worth spending time on, which is why the evaluation set scores end-to-end output rather than treating retrieval as plumbing.

Follow-up

"Which vector store and embedding model?" — Not covered here; that is Venera's implementation detail. Ask him directly.

Grounded in: Embedding-based retrieval over a vector store, on-premise constraint, portfolio assistant BM25 decision

Q2.8 — What does this project prove that his side projects do not?

hiring-manager · asked by Hiring Manager · production, credibility, constraints, ownership

Three things, and they are the three that side projects structurally cannot demonstrate.

Working inside constraints he did not choose. Every side project is built on the stack its author picked, targeting the requirements its author set. The Pulsar chatbot had to run offline inside an on-premise product, on models small enough for customer hardware, producing output conforming to an existing schema he did not design. Most real engineering is working inside constraints like those, and a portfolio of self-directed projects gives no evidence of it.

Consequences. If his document Q&A chatbot returns a bad answer, he notices and fixes it. If the Pulsar chatbot produces a template that validates but encodes the wrong checks, an operator runs quality control with the wrong checks, and that reaches a customer. Designing for that changes what you build — it is the reason the validation layer is a hard gate rather than a best-effort repair.

Someone else's timeline and someone else's product. It shipped as part of a commercial product on a release schedule, with the review, integration and readiness work that involves — not when he decided it was finished.

What it does *not* prove, to be fair to the question: scale, longevity, or that he can do this repeatedly. It is one feature, owned since August, in a product that is not consumer-scale. And he had expert support from an external consultant. It is strong evidence for his level and weak evidence for anything beyond it.

The side projects still carry information, just different information — they show what he does when nobody sets the bar, which is where the eval sets and the published build log come from.

Follow-up

"Which would you rather ask him about in an interview?" — This one, because the constraints make the reasoning visible. Side projects mostly reveal taste.

Grounded in: Pulsar chatbot constraints and ownership, Document Q&A project, portfolio assistant, build log

Q2.9 — How does he know the chatbot is actually good?

sceptical · asked by Hiring Manager, Engineer · evaluation, measurement, quality, evidence

Through the evaluation set, and he would be the first to say that is the only honest answer available.

There is no way to look at a generative system and know whether it is good. Sampling a few outputs tells you almost nothing, because the variance between runs is wide enough that a handful of examples will confirm whatever you expected to see. The only thing that produces a real answer is deciding in advance what correct looks like across a spread of cases, and then scoring against it whenever something changes.

So what he has is: a set of descriptions paired with the templates that should result, plus a set of requests that should be refused, scored as a suite. When retrieval changes, the prompt changes, or the model version changes, that number moves or it does not.

What he would not claim: that the number is a guarantee. An eval set measures the cases you thought of. The failures that matter most are usually the ones nobody put in the set, which is why the set grows — every real failure that gets reported should end up in it before the fix lands, so the fix is provably a fix and the case cannot silently come back.

He has direct experience of that discipline mattering. On his portfolio assistant the eval caught a regression he had no reason to expect: writing up a bug in the site's own build log added text to the indexed corpus that reintroduced the exact matching behaviour the bug was about, dropping the score from 36/36 to 34/36. Nothing about that failure was predictable by reading the code, and without a scored suite it would have shipped.

Follow-up

"What is the actual score on the Pulsar eval?" — Not published here. The portfolio assistant's is 36/36 and is reproducible by running `eval_retrieval.py` in this repo.

Grounded in: Chatbot evaluation set, portfolio assistant eval 36/36, build log bug 05

Q2.10 — Is this a real production feature or an internal prototype?

sceptical · asked by Hiring Manager, Recruiter · production, credibility, scope

It is a feature in Pulsar, which is a commercial on-premise product that Venera's customers run. That is what the resume claims and it is the claim worth testing directly with him rather than through an assistant.

What supports it: the constraints are the constraints of a shipped product rather than a prototype. A prototype does not need to run entirely offline inside a customer deployment. A prototype does not need its generated output validated against a production schema before use. A prototype does not need an evaluation set covering the requests it must refuse. Those requirements come from the thing being used by people who are not you.

It is also worth being precise about what is being claimed. It is a *feature in a product*, not an entire product, and it is his first production generative-AI system. It has been his primary work since August 2026, which at time of writing is a matter of months rather than years. He has not maintained it through a model upgrade, a year of accumulated edge cases, or significant drift in how operators use it — and he would tell you that is where the genuinely hard parts of this job live.

The site takes a deliberate position on this kind of question: the system prompt for this assistant instructs it to be honest about his being early-career rather than overselling, and to decline rather than guess when something is not covered. If an answer here sounds hedged, that is the design rather than evasion.

Follow-up

"How would you verify it?" — Ask him what the feature does when an operator's description is ambiguous. Prototype answers are about the happy path; production answers are about the edges.

Grounded in: Pulsar product, chatbot constraints, current role since August 2026, assistant system prompt

Q2.11 — What does he own on it day to day?

hiring-manager · asked by Hiring Manager, Recruiter · ownership, responsibilities, role, day-to-day

Both halves of it — the implementation and the testing — which is an unusual combination and is the thing most worth understanding about his role.

On the implementation side: the retrieval layer over the template corpus and schema, prompt design and grounding, and the structured-output validation that sits between the model and the product. That is the pipeline from an operator's sentence to a template that is safe to use.

On the testing side: the evaluation set the system is scored against, covering both the templates it must generate correctly and the requests it must refuse. He wrote it and he maintains it.

Most teams split those roles, and the usual reason to split them is that the person who built something is a poor judge of whether it works. His argument for holding both — and it is the same argument his whole career makes — is that on a generative system the person who understands where it is likely to be wrong is the person who built it, and the risk of self-assessment is managed by the eval set being written down and scored rather than performed by intuition. A suite that runs and produces a number does not care who wrote it.

Alongside that he continues to contribute across the full SDLC on the wider platform — acceptance criteria, test strategy review, AWS cloud-native release readiness on the Quasar side.

The context that makes the combination make sense: he spent six months doing quality engineering on this exact platform before building on it. He knew the product, the domain and the release process before he wrote a line of the feature, which is why he was a reasonable person to hand it to.

Follow-up

"Is he a QA engineer or a developer now?" — Developer, on this feature, who owns its quality. His title is Associate Software Engineer.

Grounded in: Implementation and testing ownership, evaluation set, SDLC contributions, prior QA role on same platform

Q2.12 — How would he approach building something similar for us?

hiring-manager · asked by Hiring Manager, Engineer · approach, methodology, process, transferable

The sequence he has actually used, which is visible in both the Pulsar work and his own projects:

- 1. Find out what wrong looks like, first.** Before anything is built, what does a bad output do to the user? For Pulsar, an invalid template is caught by validation; a valid template encoding the wrong intent is not, and it reaches an operator. That distinction determined the architecture.
- 2. Write the evaluation set early.** Cases it must get right, and cases it must refuse. Written before the system is tuned, so it is a target rather than a description of what was built. He is firm on the refusal half because he has been caught by omitting it.
- 3. Establish the cheapest baseline that works.** On his portfolio assistant that meant BM25 over thirty chunks rather than embeddings and a vector database — the corpus was small enough that the extra infrastructure bought nothing measurable. On Pulsar the vocabulary gap made embeddings necessary. The point is that the choice followed from the corpus, and he has a documented example of choosing the *less* fashionable option because it measured better.
- 4. Make grounding structural, not requested.** Retrieve the real material, constrain the output shape, validate in code. A prompt asking for good behaviour is a preference; a validator is a property.

5. Design the failure path before improving the happy path. This site returns retrieved resume text directly when every model provider is unavailable, so it degrades rather than erroring.

6. Measure after every change, including the ones that obviously help. The regression that taught him this was one where documenting a bug reintroduced it — entirely invisible without a scored suite.

What he would want from you before starting: what the failure costs, what data already exists, and what constraints are non-negotiable. The Pulsar architecture was largely determined by "must run offline", and knowing that on day one is worth more than a month of exploration.

Follow-up

"What if we do not have an eval set or labelled data?" — Then building a small one is the first task, not a blocker. Thirty cases you wrote yourself beats no measurement.

Grounded in: Pulsar chatbot approach, portfolio assistant BM25 decision, build log bugs 04 and 05, zero-key fallback design

Part 3 — RAG and Retrieval

Technical questions about retrieval, answered through what he has actually built rather than as definitions. He has shipped three distinct retrieval systems — an embedding-based one inside a production on-premise product, a Pinecone-backed one over a 200-page corpus, and a dependency-free lexical one running on this page — and the interesting part is that they use three different techniques for defensible reasons.

Q3.1 — Explain RAG as he would, based on what he has built.

engineer · asked by Engineer, Hiring Manager · rag, retrieval, grounding, architecture

A language model knows what was in its training data and nothing about your documents. RAG fixes that by retrieving the relevant material at question time and putting it in the model's context, so the answer is generated from your content rather than from its memory.

The pipeline is four steps: chunk your corpus into retrievable pieces, index them, retrieve the pieces relevant to a question, and generate an answer constrained to what was retrieved.

The way he would frame it from experience is that the interesting work is almost entirely in step three, and most teams spend their time on step four. When a RAG system gives a wrong answer, the instinct is to rewrite the prompt. In his experience the prompt is usually fine and the model was handed the wrong passages — no amount of instruction fixes context that does not contain the answer.

The second thing he would emphasise is that RAG buys you two properties beyond knowledge: **citability** and **the ability to decline**. Because the answer is grounded in specific retrieved passages, you can show which ones, and a user can check. And because retrieval can come back empty, the system has a signal meaning "the corpus does not cover this" — which is the only way to build something that says "I don't know" reliably rather than generating something plausible.

He learned that second point the hard way on this site. An early version substituted default chunks when retrieval scored nothing, so there was always something to answer from. That destroyed the only signal meaning "not covered", and a question about his hobbies came back with his career summary attached, reading as though it had answered. Retrieval returns empty now and the app declines before any model call.

Follow-up

"Does RAG eliminate hallucination?" — No. It reduces it and introduces two new failure modes: the model ignoring correct retrieved context, and answering confidently from context that was retrieved wrongly. Retrieval quality becomes the dominant variable.

Grounded in: Pulsar chatbot RAG pipeline, portfolio assistant retrieval, build log bug 04

Q3.2 — Why does this site use BM25 instead of embeddings?

engineer · asked by Engineer, Hiring Manager · bm25, embeddings, retrieval, trade-offs, cost

Because the corpus is thirty short chunks, and at that size embeddings would add an API round trip, a cold start and a bill in exchange for a recall improvement he could not measure.

The reasoning he documented:

- **Latency.** BM25 runs in-process in about 0.04 milliseconds, measured over 2,900 runs. An embedding call is a network hop.
- **Failure modes.** A local lexical index cannot be rate-limited, cannot be down, and does not need a key. This site answers questions with zero API keys configured because retrieval never depends on the network.
- **Cost.** No vector database, no embedding API, no bill.

- **The actual problem was not similarity.** The hard part was that recruiters ask "can he do X" while a resume says "built X". That is a vocabulary gap, and he closed it with stemming, phrase normalisation and a weighted synonym layer that maps hiring language onto resume vocabulary — cheaper and far more targeted than reaching for a bigger model.

What makes this a defensible answer rather than a rationalisation is that he chose differently on the Pulsar chatbot at work, where the corpus is larger and the vocabulary gap between how an operator describes their intent and how the schema encodes it is genuinely semantic. That one uses embedding-based retrieval over a vector store. Same engineer, opposite choice, because the corpora are different.

He was also explicit about when the decision would expire, and it has. The site's roadmap said to move beyond BM25 once the corpus grew past a few hundred chunks. The interview corpus behind this assistant is that growth — 163 questions and answers — and he measured before switching: on questions reworded the way visitors actually type them, BM25 found the right answer in the top three less than half the time and embeddings about nine times in ten. So the live assistant now leads with embeddings, and the original BM25 resume index remains as the fallback that works with no network at all. Same principle both times: choose from the corpus, and measure.

Follow-up

"Is BM25 not just keyword matching?" — It is lexical, but it weights terms by inverse document frequency and normalises for document length, so a rare discriminating term outweighs a common one. That is why the weighted synonym layer mattered: unweighted expansion let broad synonyms outvote the one term that discriminated.

Grounded in: Portfolio assistant BM25 design, 0.04 ms measurement, 30 chunks, synonym layer, Pulsar embedding retrieval

Q3.3 — How does he decide on chunk size?

engineer · asked by Engineer · chunking, retrieval, evaluation, tuning

By measuring it, which sounds obvious and is uncommon — chunk size is usually picked once from a blog post and never revisited.

His position is that chunk size is a hyperparameter of the retrieval system and should be tuned against a retrieval eval set like any other. The trade-off is legible: chunks that are too small lose the context that makes a passage meaningful, so a retrieved fragment refers to a subject resolved in the previous chunk. Chunks that are too large dilute the signal — the passage matches on something incidental, and the part that actually answers the question is buried among paragraphs that are not about it, where the lost-in-the-middle effect means the model may not use it.

What he does in practice:

- **Split on structure, not on character count.** Headings, paragraphs, list items. A split that respects the document's own boundaries produces chunks that are about one thing, which is the actual goal — fixed-width splitting is a proxy for it and a poor one.
- **Keep the unit of meaning intact.** On this site each chunk is one coherent thing: a job, a project, a skill group, a design decision. On the interview corpus each chunk is one question and its answer, because that is the unit a visitor's question maps to.
- **Overlap only where splitting was forced.** Overlap is a patch for having cut something that should not have been cut; if structure-aware splitting is doing its job, you need less of it.
- **Measure in tokens, not characters,** because the context budget and the embedding model's limit are both token-denominated.

The part he would insist on: none of this is decidable in the abstract. It depends on the corpus and the questions. The only way to know is a labelled set of real queries with the passages that must appear in the top-k, scored on every change — which is what `eval_retrieval.py` is for on this site, and what the retrieval eval over the interview corpus is for.

Follow-up

"What if a single answer is longer than a sensible chunk?" — Split it and attach a short contextual header to each piece so the fragment carries its own subject. Losing the subject is the failure that matters.

Grounded in: Portfolio assistant chunk design, interview corpus chunking, retrieval eval sets

Q3.4 — What does hybrid retrieval mean, and where has he used it?

engineer · asked by Engineer · hybrid-retrieval, bm25, embeddings, fusion, rrf

Hybrid retrieval runs a lexical search and a semantic search over the same corpus and combines their rankings, because the two fail in opposite directions.

Lexical search — BM25 — is exact. It will find a specific product code, an error string, a person's surname or an API symbol reliably, and it will completely miss a passage that means the same thing in different words. Semantic search over embeddings is the reverse: it bridges vocabulary gracefully and will confidently miss an exact identifier because the embedding model never learned a distinctive vector for that token.

Running both and fusing them covers both failures. The standard fusion is Reciprocal Rank Fusion, which combines by rank position rather than score. That detail matters: BM25 scores and cosine similarities are on incomparable scales with no meaningful calibration between them, so any weighted-sum approach requires tuning a normalisation that drifts the moment you change the embedding model. RRF sidesteps it entirely by only using the ordering.

Where he uses it: over the interview corpus behind this assistant, and the way he tuned it is the interesting part, because the result was not what the textbook predicts. He measured every combination on 52 reworded questions plus 14 exact-term ones — tool names, numbers, identifiers. Embeddings alone scored about nine in ten. Fusing BM25 in at equal weight *lowered* that slightly, because on this corpus the embedding model handled exact terms as well as BM25 did, so an equal lexical vote only diluted the ranking. BM25 at a quarter weight cost nothing measurable, and he kept it there as insurance for identifiers an embedding model has never seen — a case his eval does not cover, which he says openly rather than claiming it was measured.

The engineering around it follows the site's principle that nothing may *depend* on the network. Documents are embedded once, offline, and shipped as a compact artifact; only the visitor's question is embedded at answer time, against a hard two-second deadline. If that call fails, times out or hits the free-tier quota, retrieval falls back to BM25 plus a corpus-only latent-semantic index computed in-process, and the answer's trace says so.

Follow-up

"So is hybrid retrieval overrated?" — Not in general; it depends on the corpus and the embedding model, which is the point of measuring. On this corpus, equal-weight fusion made things slightly worse, and assuming otherwise would have shipped a worse system.

Grounded in: Interview retrieval eval (52 reworded + 14 exact-term questions), live weights dense 1.0 / BM25 0.25, embedding fallback to BM25 + LSA, 2-second embedding deadline

Q3.5 — How does he measure whether retrieval is working?

engineer · asked by Engineer, Hiring Manager · evaluation, retrieval-metrics, recall, measurement

A labelled set of real questions, each paired with the passages that must appear in the top-k, scored as a suite that fails the build.

On this site that is `eval_retrieval.py`: 29 questions that must retrieve the right resume section and 7 that must retrieve nothing at all, currently 36 out of 36. It runs in a second, exits non-zero on failure, and is meant to be run after any change to the profile data or the retrieval code.

The metrics he uses are the standard ones — `recall@k` as the primary (did the right passage make the cut at all), with rank position mattering because of the lost-in-the-middle effect. A passage retrieved at position 9 of 10 is technically a hit and practically often a miss.

Two things he would emphasise that are less standard:

The must-decline half is not optional. Seven of his 36 cases assert that retrieval returns *nothing*. Without them, an eval rewards a retriever that always returns something, which is the exact behaviour that makes a grounded system answer questions it should refuse. He added that half only after the system answered an unrelated personal question by attaching his career summary, and he now treats it as the first thing to build rather than the last.

Known limitations are recorded rather than hidden. The eval prints two cases it does not pass cleanly and explains why each is the prompt layer's job rather than the index's — for instance, a question about databases legitimately retrieves the certifications chunk, and refusing an off-task request is not something retrieval can decide. Recording those is more useful than tuning the index until the number is clean.

The discipline this enforces showed its value on a change that had nothing to do with retrieval: editing his experience section broke a case that had passed for months, and the eval caught it immediately.

Follow-up

"What is the failure the eval was written for?" — Asked what he had shipped with LLMs, the assistant returned a Kotlin chat app and a Java CRUD app. He reproduced it at 26/29, then fixed scoring until it hit 29/29.

Grounded in: `eval_retrieval.py`, 36/36 score, build log bugs 02, 04 and 12, known limitations output

Q3.6 — What is the hardest retrieval bug he has fixed?

engineer · asked by Engineer · debugging, retrieval, bugs, synonyms

The one that taught him the most is documented as bug 02 on this site.

Someone asked the assistant "what has he actually shipped with LLMs?" and it answered with a Kotlin chat app and a Java CRUD app — the two least relevant projects in the corpus.

The cause was his own query expansion. To bridge the gap between how recruiters ask and how resumes are written, he expands query terms with synonyms. But the expansion was unweighted, so a broad synonym like "shipped" expanding to "project" matched every project chunk equally — and because his own chunk template stamps the word "project" onto every project, all of them scored the same noise while "LLM", the single term that discriminated, was outvoted.

The fix had three parts: typed terms keep full weight while expansions carry 0.3, so a synonym can never outvote a word the visitor actually typed; light suffix stemming so "LLMs" matches "LLM"; and phrase normalisation so multi-word terms collapse to one canonical token before tokenising. That last one came from a related failure where "large language model" matched his *programming languages* list — token overlap with no meaning overlap.

What makes it the hardest is not the fix, which is straightforward. It is that he only found it because he wrote the eval set first, reproduced the failure at 26/29, and had a number to move. Without that he would have tried a better prompt.

The second-hardest is stranger and is bug 05: writing up bug 04 in the site's build log added that text to the indexed corpus, which reintroduced the exact matching behaviour the bug was about and dropped the eval from 36/36 to 34/36. Documenting a bug caused it to recur. It is why the build log's wording is deliberately generic about the terms involved.

Follow-up

"What is the general lesson?" — In a retrieval system the index is the product, and anything that changes the corpus changes retrieval. Content edits are code changes.

Grounded in: Build log bugs 02, 03 and 05, weighted expansion at 0.3, phrase normalisation, eval progression 26/29 to 36/36

Q3.7 — How would he improve a RAG system that returns the right documents but wrong answers?

engineer · asked by Engineer, Hiring Manager · rag, debugging, grounding, generation

First he would check that the premise is true, because it usually is not. "The right documents were retrieved" normally means someone looked at the top result and recognised it. The questions that matter are whether the supporting passage survived into the assembled prompt, and where it landed.

Assuming it genuinely did, the causes in order of likelihood:

Position. Retrieval accuracy inside a context window is U-shaped — material at the start and end is used far more reliably than material in the middle. A supporting passage sitting at position nine of ten is frequently ignored. Retrieving fewer, better-ranked passages beats retrieving more, which is also cheaper.

Competing context. Other retrieved passages contradict or dilute the right one. A reranker over the shortlist helps here more than anything else, because it scores query and passage jointly rather than comparing independent embeddings.

The prompt does not say context wins. When retrieved context conflicts with what the model learned in training, behaviour is unpredictable unless you are explicit. His system prompt on this site states that the retrieved context is the complete record and forbids answering outside it.

Truncation. The passage was in the retrieved set and got cut during assembly because the budget was not managed explicitly. Reserve space for output and for instructions, then give retrieval what remains — and truncate retrieval, never the instructions.

Model capability. Test by handing the identical context to a stronger model. If it answers correctly, this is a routing decision, not a retrieval problem.

The discipline he would apply throughout: add the failing case to the eval set *before* fixing it, so the fix is provably a fix and it cannot silently come back. And resist the instinct to patch it by appending a sentence to the system prompt addressing this one instance — that is how prompts become hundreds of tokens of accumulated scar tissue nobody will touch.

Follow-up

"What would he check first?" — Whether the assembled prompt actually contained the passage, verbatim. That single check eliminates most hypotheses.

Grounded in: Portfolio assistant system prompt, retrieval eval discipline, Pulsar chatbot grounding

Q3.8 — Does he have vector database experience?

recruiter · asked by Recruiter, Engineer · pinecone, vector-database, embeddings, tools

Yes, on two systems, with different stores.

Pinecone, on his document Q&A chatbot. That project indexes a 200-page knowledge base — chunking and embedding pipeline into Pinecone as the vector store, LangChain orchestrating retrieval over it, the OpenAI API for generation, served through Streamlit. It cut retrieval time by roughly 70%.

A locally-run vector store, on the Pulsar chatbot at Venera. That one retrieves over the existing template corpus and schema using embeddings, and everything runs inside the customer's on-premise deployment — so both the embedding model and the store run locally rather than as hosted services. The specific store is Venera's implementation detail and not something this assistant will name.

Worth adding for accuracy: he has also deliberately *not* used one where it was not warranted. This site's resume assistant runs BM25 in-process over thirty chunks, because at that size a vector database adds a network hop, a cold start and a bill for recall he could not measure a gain from. Being able to say why you did not use a tool is usually better evidence of understanding it than having used it.

What he has not done: operated a vector database at large scale, tuned approximate-nearest-neighbour index parameters under a real recall/latency budget, or run a sharded deployment. If a role needs someone who has pushed HNSW parameters around at a hundred million vectors, that is beyond what he has done.

Follow-up

"Which would he pick for a new project?" — He would ask how large the corpus is first. His own record shows him choosing differently at thirty chunks and at production scale, and that is the question that decides it.

Grounded in: Document Q&A Chatbot with Pinecone, Pulsar chatbot local vector store, portfolio assistant BM25 decision, skills list

Q3.9 — How does he handle a question the corpus does not cover?

engineer · asked by Engineer, Hiring Manager · refusal, grounding, out-of-scope, guardrails

Declines, deterministically, before the model is ever called — and this is one of the design decisions he is most opinionated about.

The mechanism on this site: retrieval returns an empty list when nothing scores above threshold. That empty result is the signal meaning "the corpus genuinely has nothing on this". The application checks it and returns a refusal directly, with his email address, without spending a model call. No generation means no chance of the model padding an answer out of unrelated context.

The reason it works this way is that it did not, originally. An earlier version substituted a few default chunks whenever retrieval scored nothing, so there was always *something* to answer from. That felt safer and was strictly worse: it destroyed the only signal that meant "not covered", so neither the prompt nor the fallback could decline, and a question about his hobbies came back with his career summary attached — reading as though it had answered the question.

Three things followed from that fix:

- Empty is now the honest return value from retrieval, and callers decide how to decline.
- Seven must-decline cases went into the eval set, so the behaviour cannot silently regress.
- The system prompt handles the harder middle case: when there is no new match but there *is* conversation history, a follow-up like "how long did that take?" is legitimate, so the model is allowed to try — but is told plainly that the corpus returned nothing this turn.

The same principle is in the Pulsar chatbot at work, where the evaluation set includes requests it must refuse rather than guess at. A confident template for an ambiguous request is worse than a refusal, because the operator gets no signal that it guessed.

Follow-up

"Can you make it answer something out of scope?" — Try it. Ask this assistant something his resume does not cover and it should decline and point you at his email.

Grounded in: Build log bug 04, empty-retrieval design, 7 must-decline eval cases, system prompt, Pulsar eval refusal cases

Q3.10 — What would he do differently on his next RAG system?

engineer · asked by Engineer, Hiring Manager · retrospective, improvement, roadmap, reranking

The things on his own roadmap for this site, plus what the Pulsar work taught him.

Write the must-decline cases first. He has now been caught twice by evaluating only what a system should get right. On the next one, the refusal set goes in before the success set, because it is the half that determines whether the thing is trustworthy and the half that is always deprioritised.

Add a reranker earlier. Retrieve a wider shortlist and rerank it with a model that scores query and passage jointly. It is the single highest-value addition to most RAG systems and he has not had one in production yet — on this site the corpus was too small to need it, which is a reason and not an excuse.

Instrument from day one. Log the query, the retrieved chunk ids and scores, the assembled prompt, and the response, for a sample of traffic. Every retrieval bug he has debugged was tractable because the answers name their own sources; the ones that would have been hard are the ones where that trace does not exist.

Treat content edits as code changes. Bug 05 on this site — where writing up a bug reintroduced it — and bug 12, where editing his experience section broke a retrieval case that had passed for months, are the same lesson. In a retrieval system the corpus *is* the index. Anything that edits content should run the eval.

Stream the response. Currently this site waits for the full answer before rendering. That is a user-experience gap rather than a correctness one, and it is on the roadmap.

Hybrid from the start where the corpus warrants it, rather than starting lexical and migrating. He was right that thirty chunks did not need it; he would not make the same call at three hundred.

Follow-up

"Which of those has he already done?" — Hybrid retrieval, on the interview corpus backing this assistant. The rest are genuinely still on the list.

Grounded in: Site roadmap (hybrid, expanded eval, streaming), build log bugs 04, 05 and 12, interview corpus hybrid retrieval

Q3.11 — How large a corpus has he worked with?

recruiter · asked by Recruiter, Engineer · scale, corpus-size, limits, honesty

Modest, and it is one of the clearer limits on his experience.

The sizes on record:

- **A 200-page knowledge base** on the document Q&A chatbot, indexed into Pinecone with LangChain orchestration. Roughly 70% faster retrieval was the measured result.
- **The Pulsar template corpus** at Venera — the existing templates plus the schema material. Production, but a product's template library rather than a web-scale corpus.
- **Around 100 pages** of interview question-and-answer content backing this assistant, which is the largest corpus he has built retrieval over and the reason it uses hybrid retrieval.

- **Thirty chunks** for the resume index, which is small enough that lexical retrieval was measurably the right choice.

What he has not done: retrieval over millions of documents, distributed or sharded indexes, or tuning approximate-nearest-neighbour parameters under a hard recall-versus-latency budget at scale. The problems that only appear at volume — index build time, memory pressure, recall degradation you cannot see without measuring against exact search — he has read about and not lived.

He would argue the techniques transfer more than the scale suggests, because the decisions that determine whether a RAG system works are chunking, retrieval quality, grounding and evaluation, and those are the same at every size. But the operational side genuinely does not transfer, and a role that needs someone who has run retrieval infrastructure at scale should treat that as a real gap rather than a formality.

Follow-up

"Is the corpus size why this site uses BM25?" — Yes, directly. Thirty chunks did not justify a vector database. The 100-page interview corpus did justify hybrid retrieval, and got it.

Grounded in: Document Q&A 200-page corpus, Pulsar template corpus, interview corpus, 30-chunk resume index

Q3.12 — How would he build a RAG system over our internal documents?

engineer · asked by Hiring Manager, Engineer · rag, design, approach, retrieval

The order he would work in, drawn from the three retrieval systems he has built:

- 1. Collect real questions before touching the documents.** Twenty or thirty questions people actually ask, each paired with the passage that answers it — plus questions the documents do *not* answer. That set is the evaluation, and it is built first because every later decision is judged against it. His own suite has 29 must-match and 7 must-decline cases.
- 2. Understand the constraints.** Can data leave your network? On the Pulsar feature it could not, which meant local open-weight models, local embeddings and a local vector store — a completely different architecture from calling a hosted API. What does a wrong answer cost? Who is allowed to see which documents? Tenant or permission filtering belongs in the query to the index, never as a post-filter and never in the prompt.
- 3. Chunk by structure.** Split on the documents' own boundaries — headings, sections, list items — so each chunk is about one thing, and attach a short header carrying the document and section so a fragment keeps its subject.
- 4. Start with the cheapest retrieval that could work, then measure.** For a small corpus that may be lexical search, which is what he chose for thirty resume chunks. For anything larger or phrased in varied language, hybrid: lexical and semantic passes fused by rank, which covers exact terms and paraphrase at the same time. Add a reranker over the shortlist if the eval says precision is the problem.
- 5. Ground and constrain generation.** The prompt says retrieved context is the whole record; answers cite their sources; empty retrieval declines rather than improvising; structured output is validated in code.
- 6. Design the failure path.** What happens when the model provider is down or slow — his systems degrade to returning the retrieved text rather than erroring.
- 7. Log real traffic and grow the eval from it.** The questions people actually ask will differ from the ones you collected.

Follow-up

"How long would a first version take?" — Depends on the corpus and constraints. The evaluation set is the part worth not rushing; the pipeline itself is fast to build.

Grounded in: Three retrieval systems (Pinecone RAG, portfolio BM25, Pulsar local embeddings), eval design 29+7, hybrid retrieval over interview corpus, refusal and fallback design

Part 4 — Prompt Engineering

He has written prompts for three systems that are still running: the Pulsar template generator at Venera, the assistant on this page, and a prompt-evaluation loop over Hugging Face NLP pipelines. The through-line in his answers is that he treats a prompt as a versioned artifact with an eval attached, rather than as a string that gets edited until the demo looks good.

Q4.1 — What is his actual approach to prompt engineering?

engineer · asked by Engineer, Hiring Manager · prompting, methodology, versioning, evaluation

Treat the prompt as code: version it, test it, and never change it without measuring.

The concrete practice he follows:

Prompts live in one place, not at the call site. On this site the system prompt is a single module-level constant with the retrieval layer feeding it. You cannot A/B test or roll back a template that is spliced into a request handler, and once it is inline nobody knows which of the three copies is the live one.

Every prompt has an eval set behind it. On the Hugging Face NLP work he built a prompt-evaluation loop specifically for this — each prompt variant is scored against a hand-labelled question set, so "this one is better" is a number rather than an impression. That project exists because he did not trust his own judgement reading a handful of outputs, and he was right not to: sampling three responses from a generative system will confirm whatever you already believed.

Constraints beat instructions. A prompt asking for valid JSON is a preference; a schema validator is a property. On the Pulsar chatbot the generated template is validated against the schema in code, so "produce a valid template" is not something the prompt has to get right. He puts guarantees in code and uses the prompt for the things code cannot express.

Say what to do when there is no answer. The hardest part of a grounded prompt is the refusal path. His system prompt here states that retrieved context is the complete record, forbids inventing employers, dates, titles or numbers, and gives explicit instructions for the case where the context says nothing matched.

Resist accretion. The failure mode of prompt maintenance is that every bug gets a new sentence appended addressing that one case, and eighteen months later the prompt is 800 tokens of scar tissue nobody dares touch. If a fix belongs in retrieval or validation, it goes there.

Follow-up

"How does he know a prompt change helped?" — The eval score moved. Without a scored set the honest answer is that he does not know, and he would say so.

Grounded in: Portfolio assistant SYSTEM_PROMPT, NLP Pipelines prompt-evaluation loop, Pulsar schema validation

Q4.2 — Walk me through the system prompt on this site.

engineer · asked by Engineer, Hiring Manager · system-prompt, grounding, guardrails, honesty

It is short by design and every clause is there because of something that went wrong.

The structure: it establishes who the assistant is and who it is talking to — recruiters, hiring managers and engineers evaluating him for AI/ML roles — then imposes rules.

Answer only from the retrieved context, which is the complete record. This is the grounding clause. Without it the model fills gaps from its training data, and the gaps it fills most confidently are exactly the plausible-sounding ones — an employer he never worked for, a framework he has never used.

Never invent employers, dates, titles, tools or numbers. Named explicitly because those are the categories a language model fabricates most fluently, and the ones a recruiter is most likely to act on.

When the context says nothing matched, say so and point to his email. The explicit refusal path. There is a subtlety here: mid-conversation follow-ups like "how long did that take?" are legitimate even with no new retrieval match, so the model is allowed to use earlier turns — but it is told plainly that the corpus returned nothing this turn, and told not to substitute his career summary for an answer to an unrelated question. That last instruction is there because it did exactly that.

Be honest about level. It states that he is early-career with production QA experience and self-directed applied AI work, and instructs the model not to oversell him as a senior researcher. He would rather a recruiter get an accurate picture than a flattering one that collapses in the first interview.

Lead with the concrete answer, quote real metrics, keep it under about 150 words.

What is deliberately *not* in it: anything the retrieval layer or the validation layer should handle. When the assistant answered an out-of-scope question badly, the fix was to make retrieval return empty and decline before the model call — not to add a sentence to the prompt.

Follow-up

"Is the system prompt a secret?" — No, and treating one as a secret is a mistake. Context is extractable. Nothing in it would matter if published.

Grounded in: SYSTEM_PROMPT in core/llm.py, build log bug 04, OUT_OF_SCOPE handling, guardrails list

Q4.3 — How does he prevent hallucination in a system he builds?

engineer · asked by Engineer, Hiring Manager · hallucination, grounding, guardrails, validation

He would start by reframing it: hallucination is not a bug to be fixed, it is a property of maximum-likelihood text generation. The model produces the most plausible continuation, and plausible overlaps with true but is not the same thing. A design that assumes it can be eliminated is a design that will fail.

So the question is containment, and his layers are:

Ground the answer in retrieved context, and say so in the prompt. The model is answering from material rather than from memory.

Give it a rewarded path to decline. This is the one people skip. If "I don't know" is not an available and explicitly instructed output, the model produces something, because a refusal is a low-probability continuation in most contexts. On this site, retrieval returning empty triggers a deterministic refusal before any model call.

Validate structurally rather than asking nicely. On the Pulsar chatbot the generated template is checked against the schema in code. Malformed output cannot get through regardless of what the model did.

Cite sources so claims are checkable. Every answer this assistant gives carries the resume section titles it drew from, which is also how he found a bug: the answers naming their own engine is how he discovered every model call was silently failing while the health check reported green.

Measure the rate rather than assuming it. Both his eval sets include cases that must be refused, so the refusal behaviour is scored rather than hoped for.

The failure he is most alert to, because validation cannot catch it, is output that is well-formed and wrong — a template that validates perfectly but encodes different intent than the operator described. Only an eval set with expected outputs finds that.

Follow-up

"Does temperature 0 fix it?" — It reduces sampling-driven invention and does not touch the underlying cause. It also does not make a hosted model deterministic: batching and floating-point reduction order still move the logits.

Grounded in: Portfolio assistant guardrails, deterministic refusal path, Pulsar schema validation, source citations, build log bug 08

Q4.4 — What is the prompt-evaluation loop he built?

engineer · asked by Engineer, Hiring Manager · prompt-eval, measurement, nlp, hugging-face

It is part of his Hugging Face NLP pipelines project, and it exists because he did not trust eyeballing output.

The mechanics: a hand-labelled question set with known-correct answers, and a set of prompt variants. Each variant is run across the whole set and scored, so comparing two prompts produces a number rather than an impression. Unsupported answers — where the output asserts something the source material does not contain — are tracked as a metric rather than estimated.

Why he built it: the standard way to iterate on a prompt is to change it, run two or three examples, and decide it is better. That process is close to worthless, because output variance between runs on the same prompt is wide enough that a small sample confirms whatever you expected. He wanted a measurement that was not his own impression.

What it gives you beyond a score:

- **Regression detection.** Prompt changes have non-local effects. Improving behaviour on one class of question routinely degrades another, and nothing tells you unless both are in the set.
- **A record of why the current version is the current version.** Every prompt in a mature system is the way it is because of a specific failure, and the eval is where that reasoning is stored.
- **Grounds to reject a change.** Being able to say "that phrasing scored worse" ends an argument that otherwise runs on taste.

The same discipline is behind the two other systems he has shipped: the 36-case retrieval eval on this site, and the must-generate plus must-refuse evaluation set for the Pulsar chatbot. He would describe "learn a technique, then build the thing that tells you whether it is working" as the most consistent habit across his projects.

Follow-up

"How large was the labelled set?" — Hand-labelled and modest, which he would be upfront about. Thirty cases you wrote yourself beats no measurement; it is not a benchmark.

Grounded in: NLP Pipelines with Hugging Face project, prompt-evaluation loop, unsupported answers tracked as a metric

Q4.5 — How does he get reliable structured output from a model?

engineer · asked by Engineer · structured-output, json, validation, schema, reliability

In layers, and with the assumption that the model will eventually produce something that does not fit.

His ordering:

Constrain the generation where the stack allows it. Asking for output in a defined structure, and constraining decoding to a schema where that is available, makes invalid output structurally impossible rather than merely unlikely. This is the strongest layer and the one to use if you have it.

Provide the schema and one complete example in the prompt, keep the structure shallow, and prefer enums over free strings. Deeply nested optional structures fail far more often, and an enum constrains what the model can choose at all.

Validate, always. This is the layer that must exist even if every other one is in place. On the Pulsar chatbot the generated template is validated against the schema before anything is used, and a template that does not validate does not get through. That is a hard gate.

Bounded repair, then fail loudly. Feeding the invalid output and the validation error back with "fix this to match the schema" recovers most failures in one attempt. Cap it at two and then fail visibly — an unbounded repair loop is a cost incident waiting to happen.

Reject rather than coerce. He is firm on this one. If validation fails, do not quietly patch the output into something that passes. On a QC template, silently coercing a malformed generation into a valid-looking one means the operator runs quality control with checks they did not ask for, and nothing anywhere says so. Failing is better than being wrong quietly.

The failure that validation cannot catch, and which he designs the eval set around: output that parses perfectly and means the wrong thing. Schema validation confirms shape, not intent.

Follow-up

"What if the model must sometimes say it cannot?" — Make refusal a valid value in the schema. A nullable field or an explicit variant is far more reliable than hoping for an empty object.

Grounded in: Pulsar structured-output validation, schema gating, refusal cases in eval set

Q4.6 — Does he use chain-of-thought prompting?

engineer · asked by Engineer · chain-of-thought, reasoning, cost, latency

Selectively, and he is wary of it as a reflex.

Where it earns its cost: multi-step reasoning, where the model needs to work through intermediate results. The mechanism is that a transformer does a fixed amount of computation per token, so forcing an immediate answer caps the compute available, while generating intermediate tokens creates more forward passes and writes intermediate results into the context where later steps can use them. More tokens is literally more computation.

Where it costs more than it returns, which is most of what he builds:

- **Extraction and classification.** Producing a structured template from a description, or retrieving and summarising a resume section, does not benefit from a reasoning preamble. It multiplies output tokens and latency for no accuracy gain.
- **Latency-sensitive paths.** Reasoning tokens generate serially. On an interactive assistant that is seconds of visible delay for an answer that was not going to change.
- **Anywhere it gets shown to a user as justification.** This is his strongest objection. The stated chain is not a faithful account of the computation that produced the answer — models produce correct answers via flawed reasoning and plausible reasoning leading to wrong answers. Displaying it makes a wrong answer *more* convincing, which is the opposite of what a grounded system should do.

The compromise he would use when it is warranted: reason into a field that is not displayed, then present a concise answer. You get the accuracy and the user gets an answer rather than a monologue.

On a constrained model in an on-premise deployment there is an additional consideration — output tokens cost latency on hardware you do not control, so a reasoning chain is a real budget decision rather than a free accuracy improvement.

Follow-up

"What about self-consistency?" — Sampling several chains and majority-voting is one of the more reliable accuracy gains available, at k times the cost. Justifiable on high-value low-volume decisions, not on an interactive assistant.

Q4.7 — How does he handle prompt injection?

engineer · asked by Engineer, Security · security, prompt-injection, guardrails, owasp

By assuming it succeeds and limiting what that buys an attacker, because there is no reliable way to prevent it — instructions and data share one channel, which is architectural rather than a bug to patch.

The distinction he works from: *direct* injection is a user typing "ignore previous instructions", which is mostly a nuisance. *Indirect* injection is instructions hidden in content the system retrieves and reads while doing something legitimate, and that is the dangerous one for any RAG system — including both of his.

What he actually relies on:

Scope of action. Neither system he has built can do anything consequential. This site's assistant reads a resume index and returns text; it has no tools, no write access, no ability to send anything. The Pulsar chatbot produces a candidate template that is validated against a schema before use. A successful injection against either gets you wrong text, not a side effect.

Treat retrieved content as data, never instructions. The corpus is his own content in both cases, which lowers the risk considerably — but the principle holds regardless.

Validate output structurally. An injected instruction that makes the model produce a different template still has to pass schema validation.

Escape model output before rendering. On this site the model's response is HTML-escaped before the markdown renderer runs, so a response cannot inject markup. This is the "insecure output handling" risk rather than injection proper, and it is the one most people miss — model output is untrusted input to whatever consumes it.

Rate limit. Per-IP limiting keeps a stray script from draining a free tier, which is the denial-of-wallet variant.

What he would not claim: that he has hardened a system against a motivated adversary. Neither system has meaningful privileges, which is the real reason they are safe.

Follow-up

"What changes if you give it tools?" — Everything. Least privilege, allow-listed tools, typed and validated arguments, and human confirmation for anything irreversible. He has not built an agent with real permissions.

Grounded in: Portfolio assistant guardrails (HTML escaping, rate limiting), Pulsar schema validation, no-tool architecture

Q4.8 — How does he decide between changing the prompt and changing the retrieval?

engineer · asked by Engineer, Hiring Manager · debugging, retrieval, prompting, diagnosis

By looking at what was actually retrieved before touching anything — and he would say the instinct to edit the prompt first is the single most common mistake in RAG work, including his own.

The test is direct. Look at the passages that ended up in the assembled prompt. If the material needed to answer the question is not there, no prompt change can help, and any improvement you think you see from rewording is noise. If it is there and the answer is still wrong, then it is a prompt, position or model problem.

The diagnostic order he uses:

1. **Was it retrieved at all?** If not, it is chunking, scoring, or the vocabulary gap between how the question is phrased and how the content is written.
2. **Did it survive into the prompt?** Truncation and budget management live here.
3. **Where did it land?** Position matters — a passage at the bottom of ten

is frequently ignored. 4. **Does the prompt say context wins?** If retrieved context conflicts with the model's training knowledge, behaviour is undefined unless you are explicit. 5. **Is it a capability problem?** Hand the same context to a stronger model. If that fixes it, the answer is routing, not prompting.

His clearest example is bug 02 on this site. Asked what he had shipped with LLMs, the assistant returned a Kotlin chat app and a Java CRUD app. Every instinct says rewrite the prompt. The actual cause was unweighted query expansion letting a broad synonym outvote the one discriminating term, and the fix was entirely in the scoring — weighted expansion, stemming, phrase normalisation. A prompt change would have produced a differently-worded wrong answer.

The habit that makes this tractable is that his answers cite the sections they came from, so looking at what was retrieved is a glance rather than an investigation.

Follow-up

"When is it genuinely the prompt?" — When the right passage is present, near the top, and the answer contradicts it or ignores its framing. That is a grounding instruction problem.

Grounded in: Build log bug 02, source citations on answers, retrieval eval methodology

Q4.9 — What makes a prompt maintainable a year later?

engineer · asked by Engineer, Hiring Manager · maintainability, versioning, prompt-management

Four things, and he has watched the absence of each cause problems.

It lives in one place and is versioned. Not inline at the call site, not duplicated across three handlers. On this site it is a single constant in the module that assembles the request. You cannot roll back or compare something you cannot locate.

Every clause has a recorded reason. The most dangerous prompt is one where nobody knows why a sentence is there, because then nobody will remove it and it accumulates. His system prompt's clauses each map to a specific failure — the "do not substitute the career summary" instruction exists because it did exactly that, and that is documented in the build log. A clause with a recorded cause can be deleted when the cause is gone.

It has an eval attached. Without one, nobody will touch it, because nobody can tell whether a change made it worse. A prompt with a scored set behind it is editable; a prompt without one is frozen.

Fixes go to the right layer. The strongest force degrading a prompt over time is that every bug gets a sentence appended addressing that one instance. Eighteen months of that produces 800 tokens of accumulated special cases, on every single request, that nobody dares to touch. His discipline is to ask whether a fix belongs in retrieval or validation first — the out-of-scope failure on this site was fixed by making retrieval return empty and declining before the model call, not by adding a "do not answer questions outside the resume" line.

The cost argument reinforces all of this: the system prompt is on every request, so length is a recurring bill as well as a maintenance problem.

Follow-up

"How does he decide when to rewrite rather than patch?" — When the clauses start contradicting each other, or when the eval shows a change helping one case and hurting another. That is the signal the prompt is doing too many jobs.

Grounded in: SYSTEM_PROMPT structure, build log bug 04, guardrails documentation, retrieval-layer fixes

Q4.10 — Has he done anything with few-shot prompting?

engineer · asked by Engineer · few-shot, in-context-learning, examples, prompting

Yes, and the interesting version is on the Pulsar chatbot, where the retrieved templates function as the examples.

That is worth unpacking because it is the thing that makes RAG and few-shot prompting the same mechanism in practice. When an operator describes a template they want, the system retrieves real existing templates relevant to that description and puts them in the context. The model is completing a pattern demonstrated by genuine examples from the corpus, not inventing structure from its priors. That is dynamic few-shot — examples selected per request by similarity, rather than a fixed set — and it is materially better than a static set because the examples are always relevant to the specific request.

It also matters more on a constrained model. Running a smaller open-weight model in an on-premise deployment means the escape hatch of upgrading the model tier is unavailable, so the quality has to come from the context. Good retrieved examples do more for a small model than a large model does with poor ones.

The details he would flag from doing it:

- **Format consistency matters more than people expect** — keeping the structure and delimiters identical across examples is the highest-leverage detail, because much of what few-shot examples communicate is the shape of the task rather than its content.
- **Examples cost tokens on every single request.** Twenty examples in a prompt is twenty examples billed, or twenty examples of prefill latency, every time. That cost is the main economic argument for fine-tuning once volume justifies it.
- **Order affects output**, and recency bias means the last example carries extra weight.

On the prompt-evaluation work with Hugging Face, prompt variants included different example selections scored against the labelled set, so "which examples" was a measured decision rather than a guess.

Follow-up

"Is dynamic few-shot just RAG?" — Mechanically, nearly. The difference is intent: RAG retrieves facts to answer from, dynamic few-shot retrieves examples to imitate. The Pulsar chatbot does both at once.

Grounded in: Pulsar retrieval of existing templates as context, constrained on-premise model, NLP prompt-evaluation loop

Q4.11 — Does he version his prompts, and how?

engineer · asked by Engineer · prompt-versioning, registry, evaluation, maintainability

Yes, and the reasons are practical rather than bureaucratic: you cannot compare, roll back or explain a prompt that exists only as a string edited in place.

On the resume assistant the system prompt is a single constant in one module, which is enough for a system with one prompt. Over the interview corpus behind this assistant there is a proper registry, because there are several prompt variants being compared. Each version records:

- **An identifier and a one-line description**, so a result can name exactly which prompt produced it.
- **The template itself**, with its variables — the retrieved context, the question, and any selected examples.
- **What it was written to fix.** Every prompt in a mature system is the way it is because of a specific failure; writing the failure down beside the text is what later lets someone delete a clause safely instead of leaving it forever.
- **Its score on the evaluation set.**

The progression through versions follows the order that tends to matter: a plain instruction first, then an explicit role and the rule that retrieved context is the whole record, then an explicit refusal path, then retrieved examples of good answers, then a requirement to cite sources. Each step is kept only if the score moves.

The discipline that makes it work is the same one behind his retrieval eval: a prompt change is a behaviour change, so it gets scored before it ships, and a regression on one class of question is as visible as an improvement on another.

What he would caution against is treating a registry as the goal. Versioning only pays off because of the evaluation attached to it; a registry of unscored prompts is a filing system.

Follow-up

"How does he pick which version is live?" — The highest-scoring one on the evaluation set that does not regress refusal behaviour. Refusal regressions disqualify a version even when its average improves.

Grounded in: SYSTEM_PROMPT as single constant, interview prompt registry, NLP prompt-evaluation loop, retrieval eval discipline

Part 5 — Fine-Tuning and Model Adaptation

This is the part of the stack where he is most careful about what he claims. He works with open-weight models in production and has built the dataset and evaluation scaffolding for fine-tuning, but he has not run a large training job and does not pretend otherwise. The answers below are as much about knowing when not to fine-tune as about the technique.

Q5.1 — Has he fine-tuned a model?

sceptical · asked by Hiring Manager, Engineer · fine-tuning, honesty, limits, lora

Not at production scale, and this is a place where the honest answer is narrower than the resume keyword suggests.

What he has done: fine-tuning experiments with Hugging Face Transformers as part of his NLP pipelines work, and the dataset-and-evaluation scaffolding around fine-tuning — building an instruction dataset from a corpus, splitting it, checking for contamination against the eval set, and scoring a tuned model against a base one. He has worked with LoRA as the approach.

What he has not done: trained a foundation model, run a distributed or multi-GPU training job, fine-tuned a model that then went into production serving real traffic, or owned a training pipeline as an ongoing system. If a role's core requirement is running training infrastructure, that is a genuine gap.

What he *has* done that is adjacent and more relevant to most AI engineering roles: shipped a production system on open-weight models. The Pulsar chatbot runs Qwen inside an on-premise deployment, which means he has dealt with model selection under hardware constraints, the quality implications of a fixed smaller model, and getting adequate behaviour out of it through retrieval and grounding rather than through more parameters. That is model adaptation without weight updates, and for a lot of production work it is the more common skill.

His view, which he would state unprompted: fine-tuning is the option teams reach for too early. On most problems he has seen, better retrieval and a rewritten prompt deliver more in two days than a training pipeline delivers in two weeks — and he would rather be the engineer who says that than the one who builds the pipeline first.

Follow-up

"Would he be able to run a fine-tune if asked?" — He has built the dataset and evaluation side and worked with LoRA in experiments. He would be honest that his first production fine-tune would be his first, and would want review.

Grounded in: NLP Pipelines fine-tuning experiments, Hugging Face Transformers, Pulsar on Qwen open-weight models, skills list

Q5.2 — When would he fine-tune instead of using RAG?

engineer · asked by Engineer, Hiring Manager · fine-tuning, rag, decision-making, architecture

They solve different problems, and he would push back on the question being framed as a choice between them.

The decomposition he uses:

- **Prompting** changes instructions — what the task is and how to respond.
- **RAG** changes knowledge — what facts are available at answer time.
- **Fine-tuning** changes behaviour — format, tone, task-specific skill, conventions that are tedious to specify.

So the diagnostic is what is actually wrong:

"It doesn't know our data." → RAG, essentially always. Fine-tuning is a poor way to inject facts: it is expensive to update, it teaches the fact without teaching the model to cite it, and facts seen a few times in training are learned unreliably. Anything that changes weekly must not live in weights. The Pulsar chatbot is exactly this case — the template corpus changes, so it is retrieved rather than trained in.

"It knows the answer but formats it wrong or sounds wrong." → Prompting first, fine-tuning if the prompt has grown to hundreds of tokens replicated on every request.

"It can't do the task at all, and examples don't help." → Fine-tuning, if you have data.

"It's right but too slow or expensive." → This is where fine-tuning has its clearest return: train a small model on outputs from a large one and get comparable quality on a narrow task at a fraction of the cost.

The order he would actually work in is prompt, then retrieval, then fine-tuning, measuring at each step — and he would say most teams invert it. Two weeks building a training pipeline frequently produces less improvement than a better retriever and a rewritten prompt would have in two days, and you cannot tell which without an eval set, which is the real prerequisite for the decision.

They also combine. The strongest production systems use all three: a tuned model that follows house conventions, over retrieved context, with a versioned prompt.

Follow-up

"Someone says they want to fine-tune on their documentation. What do you say?" — That is the textbook case for RAG. Documentation changes, and you want citations. Fine-tuning would make it confidently wrong about last quarter's docs.

Grounded in: Pulsar RAG architecture, prompt-evaluation work, retrieval-first approach

Q5.3 — Explain LoRA and why it matters.

engineer · asked by Engineer · lora, peft, fine-tuning, efficiency

Full fine-tuning updates every weight in the model, which means holding the weights, their gradients and the optimiser state in memory simultaneously — for a 7B model in fp16 that is roughly 14 GB of weights and several times that again in training state. It puts fine-tuning out of reach without serious hardware.

LoRA — low-rank adaptation — freezes the base model entirely and inserts small trainable matrices alongside the layers it adapts. The insight is that the *update* a fine-tune applies is empirically low-rank: it lies in a much smaller subspace than the full parameter space. So rather than learning a full-size weight delta, you learn two thin matrices whose product approximates it. With a rank of 8 or 16 you train a fraction of a percent of the parameters.

What that buys, and why it changed practice:

- **Memory.** You only need gradients and optimiser state for the adapter, so fine-tuning a 7B model becomes feasible on a single consumer GPU. QLoRA goes further by quantising the frozen base to 4-bit.
- **Small artifacts.** An adapter is megabytes rather than gigabytes, so you can version them cheaply, store many, and swap them per customer or per task against one loaded base model. For a product with per-tenant behaviour that is a serving architecture, not just a training trick.
- **The base is untouched.** You cannot catastrophically forget what you never modified, and rolling back is deleting a file.
- **Rank is a regulariser.** Constraining the update to a low-dimensional subspace limits capacity, which is often why LoRA overfits less than full fine-tuning on small datasets.

The trade-off is that it does not match full fine-tuning on tasks requiring large behavioural change, and rank and which modules to adapt become hyperparameters you have to tune.

For his context it is the relevant technique precisely because the constraint is always hardware — the Pulsar deployment runs on customer machines, and anything requiring a training cluster is not an option.

Follow-up

"What rank would he start with?" — 8 or 16 on attention projections, then measure. Higher rank is more capacity and more overfitting risk; it is a hyperparameter like any other and should be tuned against an eval set.

Grounded in: LoRA in skills list, NLP Pipelines fine-tuning experiments, on-premise hardware constraints

Q5.4 — How would he build a fine-tuning dataset?

engineer · asked by Engineer, Hiring Manager · dataset, fine-tuning, data-quality, contamination

Carefully, because the dataset determines the outcome far more than the hyperparameters do, and because the failure modes are silent.

His process:

Define the target behaviour precisely first. Fine-tuning teaches behaviour, so you need to be able to state what the model should do differently. "Better" is not a target; "produces output in this format, with these conventions, and refuses these categories" is.

Build instruction–response pairs that demonstrate it, including the refusals. A dataset made only of successful cases teaches the model that every input has an answer — which is exactly the behaviour that makes a grounded system dangerous. This is the same instinct behind the must-decline half of his eval sets.

Deduplicate, including near-duplicates. Repeated examples are silent over-weighting. In a corpus built from real content, near-duplicates are common and the model will memorise them.

Check contamination against the eval set. This is the step people skip and it invalidates everything downstream. If evaluation examples appear in training, the score measures memorisation and the model will look excellent and behave worse in production. He would run this as an explicit check that fails loudly, not as a manual review.

Split by group, not randomly, wherever examples cluster — multiple questions from the same document, or several variants of one task. Random splitting puts near-identical items on both sides and inflates the score.

Hold quality above volume. The evidence on instruction tuning is consistently that a small well-curated set outperforms a large noisy one. A thousand good examples is a realistic target; a hundred thousand scraped ones is usually worse.

Keep provenance. Record where each example came from, so when the model develops a strange behaviour you can find the examples that taught it.

Follow-up

"What if you have no labelled data?" — Use a strong model to generate candidates, have a human correct a sample to measure the annotator's own accuracy, then train on the corrected set. Skipping the accuracy measurement means not knowing whether you are training on 95% correct labels or 70%.

Grounded in: NLP Pipelines fine-tuning experiments, must-decline eval design, prompt-evaluation labelled sets

Q5.5 — How would he evaluate a fine-tuned model?

engineer · asked by Engineer, Hiring Manager · evaluation, fine-tuning, regression, measurement

Against the base model, on a held-out set the tuned model never saw, measuring both the thing he was trying to improve and the things he might have broken.

The structure:

A task set for the target behaviour. The cases the fine-tune was supposed to fix, scored identically for base and tuned so the comparison is apples to apples. Improvement here is the reason the fine-tune exists.

A regression set for everything else. This is the half that matters and the half people omit. Fine-tuning shifts a model's behaviour globally, not only on the target task, and catastrophic forgetting is real — a model tuned to produce one output format can lose the ability to do things it did fine before. So the evaluation has to include general capability cases outside the training distribution.

Refusal behaviour. Whether the tuned model still declines what it should. Instruction tuning on a dataset of successful completions teaches the model that every input has an answer, and refusal quality degrades quietly.

Format compliance, measured programmatically. If the point was structured output, the metric is what fraction parses and validates — a number, not an impression.

Two methodological points he would insist on. The eval set must be checked for contamination against the training data, otherwise the score measures memorisation. And a perplexity improvement on held-out text is a training diagnostic, not evidence the model is more useful — instruction tuning routinely makes raw perplexity worse while making the model far better at the job.

If the fine-tune was for cost rather than capability, the evaluation is explicitly a comparison: does the small tuned model match the large base model closely enough on this task to justify the switch? That is a threshold decision, and without a scored set you are guessing at a downgrade.

Follow-up

"How do you know it did not just memorise?" — Contamination check, plus evaluating on examples collected after the training data was assembled. Time-separated evaluation catches what deduplication misses.

Grounded in: NLP Pipelines prompt-evaluation methodology, must-decline eval design, retrieval eval discipline

Q5.6 — What does he know about running open-weight models?

engineer · asked by Engineer, Hiring Manager · open-weights, qwen, self-hosting, deployment

This is the strongest part of his model-layer experience, because it is what the Pulsar chatbot required.

That system runs Qwen open-weight models inside Venera's on-premise deployment, with no outbound API calls, because the customers who run Pulsar on-premise do so precisely so their media and workflows stay on their own infrastructure. So he has had to work with:

Model selection under hardware constraints. The model is bounded by what the customer's machines can run, not by what performs best. That flips the usual optimisation — you cannot buy quality with a bigger model, so it has to come from retrieval, grounding and validation.

Local embedding and retrieval. No hosted embedding API either, so the embedding model ships alongside and the vector store runs locally.

Version control as an advantage. A self-hosted model does not silently change under you. There is no provider repointing an endpoint alias at a new checkpoint, which means behaviour is reproducible and upgrades are a deliberate release decision. For a product that ships to customer sites, that

predictability is worth a lot.

Designing around a fixed capability ceiling. Which he would argue produced a better system than an API would have, because it forced grounding and structured validation to be load-bearing rather than optional.

What he has not done: served open-weight models at scale with a production inference stack, tuned throughput with continuous batching, managed GPU capacity and utilisation economics, or quantised a model and measured the quality cost. Those are the operational skills of someone running an inference platform, and Pulsar's deployment model is different — it ships into customer environments rather than being centrally served.

Follow-up

"Would he know how to serve one at scale?" — He knows the vocabulary — vLLM, continuous batching, PagedAttention, KV cache pressure — and has not operated it. Worth treating as knowledge rather than experience.

Grounded in: Pulsar chatbot on Qwen, on-premise offline deployment, local embedding and vector store, hardware constraints

Q5.7 — When is fine-tuning the wrong answer?

engineer · asked by Engineer, Hiring Manager · fine-tuning, anti-patterns, decision-making

More often than it is proposed, in his experience. The cases where he would argue against it:

To add knowledge that changes. Weights are a bad database. Updating a fact means retraining, the model learns the fact without learning to cite it, and facts seen rarely in training are learned unreliably. Anything that changes on a weekly cadence belongs in a retrieval corpus. The most common version of this request — "fine-tune it on our documentation" — is the textbook case for RAG.

Before there is an eval set. Without a scored set you cannot tell whether the fine-tune helped, which means you cannot tell whether it was worth doing and cannot detect that it broke something else. Building the eval is the prerequisite, not a follow-up.

Before trying prompting and retrieval. Two weeks on a training pipeline frequently produces less improvement than two days on a better retriever. The ordering matters because the cheap options also tell you more about the problem.

When the training data is small or noisy. A few hundred inconsistent examples will teach inconsistency. The evidence on instruction tuning favours small and clean over large and noisy, and "we have a lot of logs" is not a dataset.

When you cannot maintain it. A fine-tuned model is an ongoing obligation: base model upgrades mean redoing it, behaviour drifts as the task evolves, and someone has to own the pipeline. A prompt is edited in a pull request.

When determinism or auditability matters more than quality. A fine-tune makes behaviour harder to explain, not easier.

The case where he *would* argue for it, to be balanced: cost and latency on a narrow, stable, high-volume task, where a small tuned model matches a large one closely enough. That is where the economics are unambiguous.

Follow-up

"Has he ever talked someone out of fine-tuning?" — Worth asking him directly. His documented pattern is choosing the cheaper option and measuring it — this site uses BM25 rather than embeddings for exactly that reason.

Grounded in: Retrieval-first approach, portfolio assistant BM25 decision, eval-set discipline, Pulsar RAG architecture

Q5.8 — How would he reduce the cost of an LLM feature?

engineer · asked by Engineer, Hiring Manager · cost, optimisation, caching, routing, efficiency

In the order of effort-to-saving, which is roughly the opposite of the order people try.

Avoid the call entirely where you can. On this site, questions the resume does not cover are declined deterministically before any model call — no tokens spent, and it is also the more correct behaviour. Caching repeated queries is the other free win.

Retrieve less, better. Three well-ranked passages beat ten mediocre ones on cost *and* accuracy, because of the lost-in-the-middle effect. A reranker that lets you cut top-k pays for itself twice.

Shorten what is on every request. The system prompt is billed on every call. A 400-token system prompt at meaningful volume is pure recurring overhead. Structuring the stable part of a prompt first also lets provider-side prompt caching do its job.

Constrain output. Output tokens typically cost several times input. Instruct for brevity, set a maximum, return structured data rather than prose where a program consumes it, and do not add chain-of-thought where it is not earning its cost.

Route by difficulty. Small model first, escalate only on failure or low confidence. With large price differences between tiers this is usually the biggest single lever.

Self-host when volume justifies it. Which is where the Pulsar architecture sits — though there it was a data-residency requirement rather than a cost decision. The honest version of this trade is that a GPU costs the same idle, so it only wins at sustained utilisation.

Make it visible. Log tokens and cost per request. Teams that cannot attribute spend cannot reduce it, and a runaway retry loop will find that out for you. This site rate-limits per IP for exactly that reason.

Follow-up

"Where do costs usually hide?" — Retries, agent loops re-reading the same context each iteration, re-embedding a corpus on every deploy, and evaluation runs nobody counted.

Grounded in: Portfolio assistant deterministic refusal, rate limiting, provider chain design, Pulsar self-hosted architecture

Q5.9 — What is his experience with Hugging Face?

recruiter · asked by Recruiter, Engineer · hugging-face, transformers, nlp, tools

Hugging Face Transformers is on his resume for the NLP pipelines project, and it is where his model-level work sits.

That project built transformer-based text pipelines for classification and summarisation — tokenisation, inference, and fine-tuning experiments. Attached to it is the prompt-evaluation loop that scores prompt variants against a hand-labelled question set, with unsupported answers tracked as a metric rather than estimated. He describes the evaluation part as the point of the project more than the pipelines themselves.

It is also the ecosystem behind the open-weight side of his production work, since Qwen models are distributed through it.

What that means concretely for his skill level: he is comfortable loading models and tokenisers, running inference, working with the tokenisation layer, and setting up fine-tuning experiments. He has not trained a model from scratch on it, has not published a model, and has not used it for large-scale distributed training.

One thing he would be precise about, because it is a common overstatement: using Transformers to run and experiment with models is a different skill from training them, and most of what he has done is the first. He would rather say that than let "Hugging Face Transformers" on a skills list imply research-level

depth it does not.

Follow-up

"Which is stronger — his Hugging Face work or his LangChain work?" — The LangChain work has shipped in a project with measured results; the Hugging Face work is where the evaluation methodology came from. Different kinds of evidence.

Grounded in: NLP Pipelines with Hugging Face project, prompt-evaluation loop, skills list, Qwen models

Q5.10 — Could he take a model from an experiment into production?

hiring-manager · asked by Hiring Manager · production, deployment, mlops, readiness

He has done it once, which is the honest framing — the Pulsar chatbot went from nothing to a feature in a commercial product, and he was the primary engineer on it.

What that involved beyond making a model produce good output: working inside a deployment model he did not control, designing for an environment with no outbound network access, building a validation layer that makes bad output structurally unable to reach the product, writing the evaluation set that says whether it works, and integrating with an existing release process he already knew from six months of QA work on the same platform.

The parts he brings from the quality-engineering side are the ones that usually decide whether an experiment survives contact with production: defining acceptance criteria, thinking about the failure path before the happy path, and having a scored suite that can say whether a change is safe. He has done release readiness work on AWS cloud-native services as a formal part of his job.

Where he would want support: he has done this once, with an external consultant available, on a system that is not high-traffic. He has not carried a model through a version upgrade, a year of drift, or an incident caused by the model itself. He has not built a retraining pipeline or a monitoring stack for model quality in production.

So: yes for a feature of comparable scope, with code review. Not yet for owning the ML platform that other people's models go through.

Follow-up

"What would he set up first on day one?" — Logging of inputs, outputs and versions for a sample of traffic, and an eval set. Without the first, debugging becomes archaeology; without the second, nobody can tell whether a change is safe.

Grounded in: Pulsar chatbot end-to-end ownership, release readiness experience, evaluation set, experience level

Part 6 — LangChain and LangGraph

He has shipped with LangChain and is actively building with LangGraph. The distinction matters and these answers keep it: the document Q&A chatbot is a finished LangChain project with a measured result, while LangGraph is where he is currently extending. The assistant answering this question runs a LangGraph agent over the interview corpus, so it is also a live example.

Q6.1 — What has he built with LangChain?

recruiter · asked by Recruiter, Engineer · langchain, projects, rag, pinecone

The document Q&A chatbot, which is the clearest finished example.

That project answers questions over a 200-page knowledge base. LangChain orchestrates the pipeline: the chunking and embedding stage that builds the index, retrieval against Pinecone as the vector store, and the generation call to the OpenAI API with retrieved passages as context. It is served through Streamlit. The measured result was roughly a 70% reduction in retrieval time over that corpus.

The part he emphasises is not the framework usage but the grounding pass: answers are constrained to the retrieved passages, and unsupported responses — where the output asserts something the retrieved material does not contain — are tracked as a metric rather than estimated. That is the same instinct that produced the eval sets on everything else he has built.

He also uses LangChain-style composition in the assistant backing this page. The pipeline is expressed as a chain — retrieve, format context, build the prompt, call the model, parse the answer — which is the LangChain Expression Language shape. There is a deliberate design choice there worth knowing about: this site has to run when nothing is installed except Flask, so the chain runs on real LangChain when it is available and on a small stdlib-only implementation of the same interface when it is not. Same pipeline definition either way.

What he would not claim: deep experience with LangChain's broader surface area — the agent toolkits, the document loader ecosystem, LangSmith in a team setting. He has used the parts that build a retrieval pipeline.

Follow-up

"Is LangChain necessary for a RAG system?" — No, and he would say so. This site's resume assistant does retrieval and generation in a few hundred lines with no framework at all. The framework earns its place when you need composition, streaming and swappable components.

Grounded in: Document Q&A Chatbot project (LangChain, OpenAI, Pinecone, Streamlit), 70% retrieval improvement, interview assistant chain

Q6.2 — What does he think of LangChain as a framework?

engineer · asked by Engineer, Hiring Manager · langchain, opinions, abstractions, trade-offs

Useful where it composes, a liability where it hides things, and the judgement is about which mode you are in.

What it genuinely gives you: a common interface across models and vector stores so swapping a component is a line rather than a rewrite; composition that makes a pipeline readable as a pipeline rather than as nested function calls; streaming, batching and async handled once rather than per project; and a large ecosystem of integrations so you are not writing yet another PDF loader.

What he is wary of, from having used it:

Abstraction over a thin thing. A RAG pipeline is retrieve, assemble, call, parse. That is not complicated, and wrapping it in framework types can make it harder to see what is actually sent to the model. When a RAG system gives a wrong answer the first thing you need is the assembled prompt,

verbatim, and a framework that makes that hard to inspect is costing you more than it saves.

Debuggability. Deep call stacks through generic runnables are harder to reason about than a function you wrote. He mitigates this by logging the assembled prompt and the retrieved chunk ids, which is the thing that actually makes RAG debuggable.

Version churn. The API has moved substantially, and code written against an older version is often not a small upgrade.

Dependency weight. Relevant to him specifically — this site deploys on a free tier, and pulling in a large dependency tree for something a few hundred lines can do is a real cost. It is why the resume assistant uses no framework and why the interview pipeline can run without one.

His summary: use it when you want composition and swappability, skip it when the pipeline is fixed and small, and either way make sure you can see the final prompt.

Follow-up

"Does he prefer writing it himself?" — For something small and fixed, yes, and this site is the evidence. For anything that needs to swap models and stores, the framework wins.

Grounded in: Document Q&A Chatbot with LangChain, portfolio assistant no-framework design, free-tier deployment constraints

Q6.3 — What is LangGraph and how is it different from LangChain?

engineer · asked by Engineer · langgraph, agents, state, control-flow

LangChain composes a pipeline; LangGraph composes a state machine. The difference is loops and branching.

A LangChain chain is a directed acyclic pipeline — input goes in, passes through stages, output comes out. That covers most RAG, and it is the right shape when the steps are known in advance.

LangGraph models the workflow as a graph with explicit nodes, explicit state, and **conditional edges** — so the path through it is decided at runtime by the state, and it can cycle. That is what you need when a step's outcome determines what happens next: retrieve, judge whether the retrieved material is good enough, and if not rewrite the query and retrieve again before generating.

The pieces that matter:

- **State is explicit and typed.** Every node receives the state and returns an update to it, and the graph defines how updates merge. Compared to passing a growing dictionary between functions, that makes the data flow legible and the whole run inspectable.
- **Conditional edges** route based on state, which is how you express "retry" or "escalate" or "this is good enough".
- **Cycles with a recursion limit**, so a loop that never converges terminates rather than running up a bill.
- **Checkpointing**, so a run's state can be persisted, resumed, or interrupted for human approval.

The practical reason he cares: it makes the agent's trace a first-class artifact. Because every node writes to shared state, you get a record of which path a request took and why, which is exactly what you need to debug a system whose behaviour is not deterministic. On the assistant backing this page the graph's node trace is available with the answer.

Follow-up

"Could you build this without LangGraph?" — Yes, and it is a few hundred lines. The framework buys you the state-merge semantics, the checkpointing and the recursion guard rather than anything you could not write.

Grounded in: Interview assistant LangGraph agent, LangGraph in skills list and certifications, exploring LangGraph for stateful workflows

Q6.4 — Has he actually used LangGraph, or is it aspirational?

sceptical · asked by Hiring Manager, Engineer · langgraph, honesty, experience, limits

Both, and the resume says so — it lists LangGraph under "exploring" for the self-directed work rather than claiming production depth.

The honest state of it:

What is real and running: the assistant answering this question routes through a LangGraph agent over the interview corpus. It is a genuine stateful graph — route the question, retrieve, grade whether there is enough evidence to answer, re-read a follow-up with the previous question if needed, generate, verify the answer's numbers and citations against its sources, and either finish, regenerate once, or quote the source instead. It runs on real LangGraph where installed, with a standard-library stand-in of the same graph when not. The node trace is returned with every answer, so you can see which path your question took.

What is training: LangGraph is part of the AI Engineer Bootcamp 2026 syllabus he is working through.

What is not true: he has not shipped a LangGraph agent in a commercial product. The production generative-AI work at Venera — the Pulsar template chatbot — is a RAG pipeline rather than an agent graph, and he has not run a multi-agent system under real load.

He would also be straight about the scale of what "agent" means here. The graph on this site has a handful of nodes and one retry loop. It is not an agent with tool access, external side effects or autonomous multi-step planning, and those are where the genuinely hard problems live — permissions, cost control on runaway loops, and failure modes that compound across steps.

So: real working code you can interrogate, on a personal system, not production experience. Anyone hiring for deep agent experience should treat this as competence rather than expertise.

Follow-up

"What would you ask to test it?" — Ask what his graph does when retrieval comes back weak. The answer is specific: it re-reads the question with the previous one only if it is a follow-up, at most once, and otherwise declines without spending a model call.

Grounded in: Interview assistant LangGraph agent, AI Engineer Bootcamp syllabus, "exploring LangGraph" on resume, Pulsar chatbot architecture

Q6.5 — Describe the agent graph running on this site.

engineer · asked by Engineer · langgraph, architecture, agent, self-referential

It is a stateful graph over the interview corpus, and the design is deliberately conservative — every loop is bounded, and every path ends in an answer, a refusal, or a quoted source.

The state carries the question, the conversation so far, the query being searched, the retrieved entries, rewrite and regeneration counters, the draft answer with its citations, and a trace of every node that ran with its timing.

The nodes:

Route. Rejects an empty or oversized question before anything else runs.

Retrieve. Embeddings plus BM25 over the corpus, fused by rank. If the embedding call misses its two-second deadline or fails, BM25 plus a corpus-only latent-semantic index take over in-process.

Grade. Decides whether there is enough evidence to answer at all. This is the node that makes it a graph rather than a chain — its outcome picks the next edge: generate, rewrite, or refuse without

spending a model call.

Rewrite. Deliberately narrow: only for follow-ups. "How long did that take?" retrieves nothing on its own, but does when read together with the previous question. It runs at most once, and it never paraphrases a standalone question into something the corpus happens to match — that would turn a correct decline into a stretched answer.

Generate. A LangChain chain: the retrieved entries, two retrieved examples of good answers, and a versioned prompt, sent through the Groq-then-Gemini provider chain. The prompt tells the model to decline anything outside his professional record, which is where questions like "is he married" are caught — similarity scores alone cannot separate those from real questions.

Verify. Every number in the answer must appear in the retrieved entries, and every citation must point at an entry that was actually retrieved. On a failure it regenerates once with a stricter instruction; if that also fails, it quotes the source entry instead of the generated answer.

The verify node exists because the failure mode of this whole class of system is a fluent answer that is not supported by its sources, and the only defence is checking rather than trusting. It is the same reasoning behind validating generated templates against the schema on the Pulsar chatbot. It checks what is checkable without another model call — invented figures above all, which are the fabrication a recruiter is most likely to act on — and it does not claim to check meaning.

The trace of which nodes ran is returned with every answer: open "How this was answered" under a reply to see it.

Follow-up

"What happens when the model provider is down?" — Generation falls back to quoting the best-matching corpus entry directly, labelled as such. Retrieval and the rest of the graph still run.

Grounded in: Interview agent graph (route, retrieve, grade, rewrite, generate, verify, refuse, fallback), one rewrite and one regeneration cap, numeric and citation verification, embedding deadline and fallback

Q6.6 — When is an agent the wrong architecture?

engineer · asked by Engineer, Hiring Manager · agents, architecture, over-engineering, judgement

Most of the time, and he would say the industry is currently over-applying it.

An agent is worth the complexity when the sequence of steps genuinely cannot be determined in advance — when what to do next depends on what the previous step found. If you can draw the pipeline on a whiteboard and it does not have a decision point in it, it is a chain, and expressing it as an agent adds state management, non-determinism and a class of failure (loops that do not terminate) for nothing.

Where he would argue against it:

Fixed pipelines. Retrieve, ground, generate, validate. That is the Pulsar chatbot and it is not an agent, deliberately. The steps are known, so a graph would add moving parts to a system whose main requirement is predictability.

Anything where latency matters. Each agent iteration is at least one model round trip. A three-iteration loop is three times the latency and three times the cost of a single call, on an interactive path where the user is waiting.

Anything with real side effects and no supervision. An agent that can send, delete, or spend turns a reasoning error into a real-world action. That needs least privilege, allow-listed tools, typed arguments and human confirmation for anything irreversible — and he has not built one with those properties, so he would not propose one lightly.

When you cannot bound the cost. An unbounded loop is a denial-of-wallet incident. Every cycle needs an iteration cap, and his own graph allows exactly one query rewrite and one regeneration for that reason.

Where an agent does earn it: retrieval that may need reformulation, tasks that decompose into sub-tasks of unknown number, and workflows with genuine branching. The graph on this site qualifies narrowly — it has one real decision point, whether the retrieved material is good enough.

His general position: start with the chain, and add the graph when you find the decision point that justifies it. Not the other way round.

Follow-up

"Why is the site's assistant a graph then?" — Because two decisions depend on what an earlier step produced — whether to generate at all, and whether to accept an answer after checking it — and one of them loops. Also because a portfolio is a reasonable place to demonstrate the technique; he would flag that as a legitimate reason and not pretend it was purely necessity.

Grounded in: Pulsar chatbot chain architecture, interview assistant graph with bounded retry, no-tool design

Q6.7 — How does he handle state in a multi-turn conversation?

engineer · asked by Engineer · state, conversation, history, context-management

Explicitly, with bounds, and with an awareness that conversation history is the easiest place to leak data between users.

On this site the history is client-supplied and then sanitised server-side before use: trimmed to the last eight turns, each message capped in length, roles validated to be user or assistant, and leading assistant turns dropped so the conversation always opens with a user message — which every provider requires. Nothing is trusted because it arrived in the request body.

The bound exists for three reasons: cost, since history is re-sent on every request and grows without limit otherwise; the lost-in-the-middle effect, where a long history pushes the actually relevant material into the part of the context the model uses least reliably; and the hard limit of the context window.

The design decision he would highlight is what happens on a follow-up with no new retrieval match. A question like "how long did that take?" is legitimate mid-conversation and retrieves nothing, because it has no content words. Declining it would be wrong. So the system distinguishes the two cases: with no history and no retrieval match it refuses deterministically without a model call; with history it lets the model try, but states plainly in the prompt that the corpus returned nothing this turn and instructs it not to substitute unrelated material. That nuance exists because an earlier version got it wrong in the other direction.

In the LangGraph agent, state is the graph's own typed state object rather than an ad-hoc dict, which makes what is carried between nodes explicit and inspectable.

The thing he would warn about in any shared system: conversation state keyed per session and never evicted is both a memory leak and a data-leak risk. Bounds are not optional.

Follow-up

"Is conversation history stored server-side?" — On this site, no. It is sent by the client each turn and the server holds nothing between requests, which is the simplest way to avoid cross-session leakage.

Grounded in: `_normalise_history` in `core/llm.py`, `MAX_HISTORY_TURNS` of 8, follow-up handling, build log bug 04

Q6.8 — What is the value of the node trace the agent returns?

engineer · asked by Engineer, Hiring Manager · observability, debugging, trace, transparency

It turns a non-deterministic system into one you can actually debug, which is the single biggest practical problem with this class of software.

When a RAG or agent system gives a bad answer, the question is which stage failed — and without a trace, every hypothesis is equally plausible and people default to blaming the prompt. With one you can see whether routing sent it to a refusal, whether retrieval came back empty, whether grading rejected the first attempt and the rewrite helped, and whether verification passed. That collapses the search space immediately.

He has direct experience of this mattering. The assistant on this site reports which provider actually produced each answer, and that is how he found bug 08: with a valid key configured, the health check reported the engine live while every answer was silently coming from the local extractive fallback, because the HTTP layer was sending Python's default user-agent and the CDN in front of the provider was rejecting it outright. The graceful fallback was hiding a total failure. He only noticed because the answers named their own source.

The generalisation he took from that: **make the system report what it actually did, not what it was configured to do**. A health check that confirms a key is present is not a health check. The same reasoning is why the status badge on this page is corrected from the first real answer rather than rendered from config, and why the trace is returned with the response here.

For a portfolio there is a second benefit: it makes the engineering visible. A visitor can see that the answer came from retrieval over specific corpus sections rather than from a model's memory, which is a claim the site would otherwise just be asserting.

Follow-up

"Would you return the trace to end users in a real product?" — Usually not; it is an internal artifact. Here it is deliberate, because demonstrating how it works is the point.

Grounded in: Build log bugs 08 and 11, engine status reporting, interview agent trace, source citations

Q6.9 — How would he choose between LangChain, LlamaIndex and writing it himself?

engineer · asked by Engineer, Hiring Manager · frameworks, tooling, decision-making, trade-offs

By how much of the pipeline is going to change, which is the question that actually decides it.

Write it yourself when the pipeline is fixed and small. A RAG system is retrieve, assemble, call, parse — that is a few hundred lines you fully understand, with no dependency tree, no version churn and a prompt you can see. This site's resume assistant is exactly that: BM25 retrieval, provider adapters and prompt assembly in three modules, no framework, and it deploys on a free tier because it weighs almost nothing. He would defend that choice for that system without hesitation.

LangChain when you need composition and swappability — several models, several stores, streaming, async, and a pipeline whose shape will change. That is what the document Q&A chatbot needed.

LlamaIndex when the hard part is ingestion and indexing rather than orchestration: many document formats, hierarchical or multi-index retrieval, and document-centric abstractions. It is the more opinionated tool for the retrieval half specifically.

The factors he would actually weigh: how much of this is genuinely mine versus boilerplate; can I see the final prompt without fighting the abstraction; what does this add to the deployment footprint; and how stable is the API, because a framework that moves fast becomes a migration project.

The position he holds most firmly is that the framework question is usually less important than people make it, because it is not where the quality comes from. Whether a RAG system works is determined by

chunking, retrieval quality, grounding and evaluation — and every framework lets you do those well or badly. Choosing the framework first is optimising the part that matters least.

Follow-up

"Does he regret using LangChain anywhere?" — He would say the resume assistant was right to avoid it and the document Q&A project was right to use it, which is a reasonable record.

Grounded in: Document Q&A Chatbot with LangChain, portfolio assistant framework-free design, free-tier deployment, Pulsar architecture

Q6.10 — How does the site run LangGraph on a free hosting tier?

engineer · asked by Engineer · deployment, dependencies, free-tier, architecture

By checking the actual numbers rather than assuming, and then building a fallback anyway.

The assumption people make is that LangChain and LangGraph are too heavy for a serverless free tier. He measured the real dependency closure — langgraph plus langchain-core and everything they pull in — at about 38 MB unzipped, against a 250 MB serverless limit. So the assumption was wrong, and the live site runs genuine LangChain and LangGraph rather than a simulation of them.

The design decision layered on top is the one worth explaining. This site's stated principle is that it never hard-fails: it answers questions with zero API keys configured, because retrieval is local and the fallback returns resume text directly. Introducing a hard dependency on two packages would have broken that property — `python app.py` with only Flask installed would stop working.

So the chain and the graph are written against a thin interface, with two implementations behind it: real LangChain and LangGraph when they are importable, and a small `stdlib`-only executor with the same semantics when they are not. The pipeline definition is the same code either way; only the engine underneath changes. Both paths are exercised by the test suite, so the fallback is not a claim.

He would describe this as the same pattern as the provider chain on the resume assistant — nine model vendors behind one adapter, first working one wins, degrade to local extraction if all fail. Design the failure path first, then make the happy path better than it.

The cost of the pattern, to be fair: it is more code than just depending on the library, and the shim has to be kept honest with tests. It is justified here because graceful degradation is the site's explicit thesis, and would be over-engineering in a system that could simply require its dependencies.

Follow-up

"Is the fallback a real implementation or a stub?" — Real, and tested against the same pipeline definition. The point is that the graph produces the same answer either way.

Grounded in: Measured 38 MB dependency closure, zero-key fallback principle, nine-provider chain, graceful degradation design

Part 7 — Evaluation, Guardrails and Honesty About Failure

If there is one thread that runs through everything he has built, it is this one. Three systems, three evaluation sets, and a published log of twelve bugs that shipped broken before he caught them. These questions get at whether the discipline is real or decorative — and the test is always whether he can say what an eval caught, not whether he has one.

Q7.1 — Why does he build an evaluation set for everything?

engineer · asked by Engineer, Hiring Manager · evaluation, methodology, measurement, discipline

Because with a generative system you genuinely cannot tell whether a change helped by looking at the output, and he has been wrong about this in a way he can point to.

The mechanics of the problem: output variance between runs is wide enough that sampling three responses confirms whatever you already believed. Changes to retrieval and prompts have non-local effects, so improving one class of question routinely degrades another, and nothing tells you unless both are in the set. And the failures that matter are quiet — a confident wrong answer looks exactly like a confident right one.

The case that made it concrete: someone asked his assistant "what has he actually shipped with LLMs?" and it returned a Kotlin chat app and a Java CRUD app. He wrote an eval set of 29 questions with expected source chunks, reproduced the failure at 26/29, fixed the scoring until it reached 29/29, and then had a number that could detect the problem coming back.

It has since caught things he had no way to anticipate. Writing up a bug in the site's own build log added text to the indexed corpus that reintroduced the matching behaviour the bug was about, dropping the score from 36/36 to 34/36 — documenting a bug caused it to recur. More recently, editing his experience section broke a retrieval case that had passed for months, because a question word was missing from the stopword list and the content change was what exposed it. Neither was findable by reading code.

The rule he works to now: **add the failing case to the eval before fixing it**. That way the fix is provably a fix, and it cannot silently come back.

Follow-up

"Does an eval set guarantee quality?" — No. It measures the cases you thought of. The failures that hurt most are the ones nobody put in the set, which is why it grows every time something real goes wrong.

Grounded in: eval_retrieval.py 36/36, build log bugs 02, 05 and 12, Pulsar evaluation set, NLP prompt-evaluation loop

Q7.2 — What is the must-decline half of an eval set, and why does he insist on it?

engineer · asked by Engineer, Hiring Manager · refusal, evaluation, out-of-scope, grounding

It is the set of questions the system is required to *not* answer, and he treats it as the more important half.

On this site it is 7 of the 36 cases: questions his resume genuinely does not cover, asserted to retrieve nothing so the system declines. On the Pulsar chatbot at Venera it is the requests the system must refuse rather than generate a template for.

The argument for it: an eval made only of success cases rewards a system that always produces something. That is precisely the behaviour that makes a grounded system untrustworthy, because the failure is invisible — a plausible answer to a question you had no basis for answering looks identical to a good answer.

He added it after being caught. An early version of the assistant substituted default chunks whenever retrieval scored nothing, so there was always material to answer from. A question about his hobbies came back with his career summary attached, reading as though it had addressed the question. The fix was to let retrieval return empty and decline deterministically before any model call — and then to put seven must-decline cases in the eval so the behaviour could not regress quietly.

The same logic applies at Venera with higher stakes: a confident QC template generated for an ambiguous request is worse than a refusal, because the operator gets no signal that the system guessed, and they run quality control with checks they did not ask for.

What he would say generalises: **the refusal set should be written first**. It is always the half that gets deprioritised, and it is the half that determines whether the system is trustworthy.

Follow-up

"Can I test it right now?" — Yes. Ask this assistant something his resume does not cover. It should decline and give you his email rather than improvising.

Grounded in: 7 must-decline cases in `eval_retrieval.py`, build log bug 04, Pulsar refusal cases, `OUT_OF_SCOPE` handling

Q7.3 — Tell me about a bug he shipped.

behavioural · asked by Hiring Manager, Engineer · bugs, honesty, debugging, build-log

He publishes twelve of them on this site, deliberately, under a build log that says how a system fails matters more than how it looks when it works. A few that show different things:

The one where the health check lied (bug 08). With a valid API key configured, the site reported the engine live while every answer was actually coming from the local extractive fallback. Two faults stacked: the HTTP layer sent Python's default user-agent, which the CDN in front of the provider rejects outright, and the health check only asked whether a key was *present*, never whether the provider would answer. The graceful fallback was hiding a total failure. He fixed both — a real user-agent, and splitting health into a cheap "configured" check and an opt-in probe that makes one live call. He only found it because answers report which engine produced them.

The one where documenting a bug reintroduced it (bug 05). Writing up an earlier bug in the build log added that text to the indexed corpus, which put the matching behaviour back and dropped the eval from 36/36 to 34/36. The write-up is now deliberately generic about the terms involved.

The one a user found (bug 09). Long answers opened scrolled to their last line, so you landed on the closing sentence and had to scroll back up. Standard chat behaviour is to pin the newest message to the bottom, which is right for short turns and wrong for anything taller than the panel. Reported by a user, not caught in testing.

The one that had never worked (bug 10). The sidebar close button did nothing on desktop. Below the breakpoint the sidebar is an overlay driven by a state class; above it, it is part of the layout, so the same class is inert. One handler had been written for both.

Follow-up

"Why publish these?" — Because a portfolio of things that worked says nothing about how someone handles things that do not. It is also a hedge against overclaiming.

Grounded in: Build log bugs 05, 08, 09 and 10, engine status reporting, eval regression

Q7.4 — How does he think about guardrails?

engineer · asked by Engineer, Hiring Manager · guardrails, safety, grounding, architecture

As properties enforced in code, not behaviours requested in a prompt. The distinction is the whole of his position.

A prompt saying "only produce valid output" is a preference the model will usually honour. A validator is a guarantee. Anything that must be true has to be checked outside the model.

The guardrails on this site, which he documents publicly:

- **Answers are constrained to retrieved context**, and the prompt explicitly forbids inventing employers, dates, titles or numbers — the categories a model fabricates most fluently.
- **Unknown questions return a refusal plus his email**, decided by retrieval returning empty rather than by the model choosing to decline.
- **The prompt instructs honesty about his level** rather than overselling, which is a guardrail against the system being more flattering than the truth.
- **Model output is HTML-escaped before markdown rendering**, so a response cannot inject markup. This is the one people forget: model output is untrusted input to whatever consumes it.
- **Per-IP rate limiting**, so a stray script cannot drain a free tier.

On the Pulsar chatbot the equivalent hard gate is schema validation — a generated template that does not validate does not reach the product, regardless of what the model produced.

The layering he would describe: retrieval decides what is available to say, the prompt shapes how it is said, and validation decides what is allowed out. Each layer catches a different failure, and the last one is the only one that is a guarantee.

What he is careful not to claim: that guardrails make a system safe against an adversary. Neither system he has built has meaningful privileges — no tools, no side effects, no write access — and that is the real reason they are safe rather than any filter he wrote.

Follow-up

"What is the most commonly missed guardrail?" — Escaping model output before rendering it. People treat the model as trusted because they wrote the prompt.

Grounded in: Guardrails list in About section, HTML escaping, rate limiting, Pulsar schema validation, no-tool architecture

Q7.5 — What metrics does he use to evaluate a RAG system?

engineer · asked by Engineer · metrics, evaluation, recall, groundedness

He separates the two halves, because conflating them is why teams cannot tell what is broken.

For retrieval, measured independently of generation: recall@k as the primary — did the passage that supports the answer make the cut at all — plus rank position, because the lost-in-the-middle effect means a hit at position nine is often a practical miss. Mean reciprocal rank and nDCG where the ordering matters more finely. And the refusal rate on out-of-scope queries, which is the must-decline half.

Measuring retrieval on its own is the step people skip, and it is the one that makes debugging tractable. If retrieval recall is 60%, no prompt work will fix the system, and you will waste weeks finding that out.

For generation, given correct retrieval:

- **Groundedness** — is every claim supported by a retrieved passage. This is the one that matters most for a system whose selling point is not making things up. On the document Q&A project he tracked unsupported answers as an explicit metric rather than estimating them.
- **Refusal correctness** — does it decline when it should, and not decline when it should not.
- **Format and schema validity** where output is structured, measured as a pass rate.
- **Answer correctness** against expected answers where a correct answer exists.

End to end, the number that actually matters is the pass rate on the labelled suite, with latency and cost per request alongside it — because an accurate system nobody can afford is not shipped.

What he avoids: perplexity, which is a training diagnostic rather than a product metric and is not even comparable across tokenisers. And any metric he cannot tie to a user-visible failure.

Follow-up

"How does he measure groundedness without a human?" — Requiring citations at the claim level and checking them programmatically against the retrieved text catches a lot. The agent on this site has a verification node that does exactly that.

Grounded in: eval_retrieval.py recall methodology, Document Q&A unsupported-answer tracking, interview agent verify node, Pulsar eval set

Q7.6 — How would he catch a regression before it reaches users?

engineer · asked by Engineer, Hiring Manager · regression, ci, testing, process

A scored suite that fails the build, plus the discipline of treating content changes as code changes.

The mechanics on this site: eval_retrieval.py runs in about a second, exits non-zero on failure, and the README instructs running it after any change to the profile data or the retrieval code. It is small enough and fast enough that there is no excuse for skipping it.

The part that is less obvious and that he has been caught by twice: **in a retrieval system, the corpus is the index**. Editing content is editing the search behaviour. Bug 05 was documenting a bug in the build log, which added that text to the corpus and reintroduced the matching behaviour the bug described. Bug 12 was editing his experience section, which broke a retrieval case that had passed for months — the underlying defect was a missing stopword, but the content edit is what exposed it. Neither change touched retrieval code. So "run the eval on content changes" is not a nicety.

What he would add, and what is genuinely still on his roadmap for this site: wiring the eval into CI so a bad retrieval change cannot merge. Right now it is a documented step rather than an enforced one, and he would say a check that depends on someone remembering is a check that will eventually not run.

From the quality-engineering side he brings the rest of the standard apparatus: acceptance criteria agreed before work starts, test strategy reviewed with dev and product, and release readiness as an explicit gate rather than an assumption. That is what he did on Venera's platform for six months, and the defect escape rate came down sprint over sprint as a result.

Follow-up

"Why is the eval not in CI already?" — It is on the published roadmap and not done. He lists it as a gap rather than claiming it.

Grounded in: eval_retrieval.py exit code, README instructions, build log bugs 05 and 12, site roadmap on CI, release readiness experience

Q7.7 — How does he evaluate something with no single correct answer?

engineer · asked by Engineer, Hiring Manager · evaluation, subjective, rubrics, llm-judge

By converting "is this good" into a set of properties that can each be checked, rather than trying to score quality directly.

On an open-ended answer, the properties he would actually assert:

- **Groundedness.** Every factual claim traceable to a retrieved passage. Checkable, and the most important one for his systems.
- **Refusal correctness.** Did it decline when it had no basis. Binary.
- **Format compliance.** Length bounds, structure, required elements. Programmatic.

- **Absence of specific failures.** Does it invent an employer, a date or a number. A targeted check against a known list catches the category that matters most here.

That covers most of what "good" means for a grounded assistant without needing a judgement call. What is left over — is it well-written, is it the *most useful* framing — is genuinely subjective, and for that the options are human review on a rubric, pairwise preference comparison, or an LLM judge.

His view on LLM judges: useful for scale, and requiring their own validation. A judge is a model with its own biases — position bias favouring the first option, length bias favouring longer answers, and self-preference for outputs resembling its own style. So the judge has to be calibrated against human labels on a sample before you trust its scores, and reported as "agrees with human labels 85% of the time" rather than as ground truth. Without that step you have replaced an unmeasured system with an unmeasured measurement.

The pragmatic position he holds: a small hand-labelled set you actually wrote beats a large automated one you have not validated. The prompt-evaluation loop on his NLP project uses a hand-labelled question set for exactly that reason.

Follow-up

"How many labelled examples are enough?" — Fewer than people think to start. Thirty cases covering the failure modes you know about is a real measurement; zero is not.

Grounded in: NLP Pipelines hand-labelled question set, unsupported-answer metric, must-decline eval design

Q7.8 — What does his QA background actually contribute to AI work?

hiring-manager · asked by Hiring Manager, Recruiter · qa, testing, transferable-skills, quality

Three specific habits, and they are the ones that decide whether an LLM feature is shippable.

Deciding what correct looks like before building. A test strategy is a statement of expected behaviour written in advance. Applied to a generative system that is an eval set, and it is the same artifact doing the same job. The reason he writes one for everything is not diligence, it is that he spent six months in a role where the deliverable was that artifact.

Designing the failure path first. In QA the question is always what this does when it is wrong. That reflex produced the architecture of both systems he has built: this site returns retrieved resume text directly when every model provider is unavailable, so it degrades rather than erroring; the Pulsar chatbot validates generated templates against the schema so malformed output cannot reach the product. Both are failure-path decisions made before the happy path was optimised.

Knowing that the quiet failure is the dangerous one. He intercepted a severity-1 503 before release and coordinated a same-day hotfix, avoiding roughly four hours of customer-facing downtime. The thing that generalises is that the failure was found because someone was looking, not because anything alerted. LLM systems fail in exactly that mode — no stack trace, nothing red, just a confident wrong answer — and the only defence is having decided in advance what to check.

The concrete evidence that it transferred rather than being a story he tells: bug 08 on this site, where a valid key was configured, the health check said green, and every answer was silently coming from the fallback. That is a QA finding in an AI system. He built the probe that distinguishes "configured" from "actually working" because "it looks fine" is not a status he accepts.

Follow-up

"Does the QA background limit him as a builder?" — The risk would be over-caution. He owns the implementation of the Pulsar feature, not just its testing, so the evidence is that he builds as well as verifies.

Grounded in: Internship QA record, severity-1 interception, zero-key fallback design, Pulsar schema validation, build log bug 08

Q7.9 — How does he decide what to measure versus what to assume?

engineer · asked by Engineer, Hiring Manager · measurement, judgement, pragmatism, metrics

He measures the things a decision depends on and the things that will drift, and he assumes the rest — because measuring everything is a way of measuring nothing well.

The test he applies: would a different value change what I do? If yes, measure it. Retrieval recall changes whether he works on chunking or on prompting, so it is measured. The exact latency of a function that runs once at startup changes nothing, so it is not.

The second test: will this number stop being true without anyone noticing? Anything that drifts needs measurement rather than a comment. He has a concrete example of getting this wrong — bug 06 on this site was hand-written statistics in a section whose entire claim is that figures are measured. The chunk count and index size were typed by hand and stopped being true every time content was added, three times. The fix was to compute them from the running index at startup, so the page reports its own state rather than a remembered one.

That is the principle he would state: **if you are claiming a number, derive it**. The eval score and chunk count on the About section of this site are computed at boot from the actual index and the actual test suite, not written down.

What he deliberately does not measure: things where the cost of measurement exceeds the value. He has not built a latency budget for this site because it is a portfolio, not a service with an SLA, and saying so is more honest than inventing a metric to look rigorous.

The failure mode on the other side, which he would acknowledge: measuring becomes a substitute for deciding. A number is not a decision, and a team that instruments everything and changes nothing has the same outcome as one that measures nothing.

Follow-up

"What is the cheapest useful measurement?" — Logging the inputs and outputs of a sample of real traffic. Almost everything else can be reconstructed from that; nothing can be reconstructed without it.

Grounded in: Build log bug 06, computed About statistics, eval_retrieval.py, measured 0.04 ms retrieval figure

Q7.10 — What would he do if an eval set and a stakeholder disagreed?

behavioural · asked by Hiring Manager · communication, stakeholders, conflict, judgement

Take the disagreement as information about the eval set, not as a problem to win.

If a stakeholder says the system feels worse and the suite says it improved, the most likely explanation is that the eval is measuring the wrong thing or is missing the cases they care about. That is a genuinely common outcome — an eval set built by the engineer captures the failures the engineer anticipated, and the person using it daily encounters a different distribution.

So the process would be: get specific examples from them. Not "it feels worse" but the actual inputs where it disappointed. Then run those and see. Three outcomes:

- **The system does fail on them and they are not in the eval.** The eval was incomplete; add the cases and now the score reflects reality. This is the most common result and it makes the suite better.
- **The system handles them correctly.** Then the disagreement is about something else — expectations, a change in phrasing, a different mental model of what the feature does. Worth surfacing, because it is usually a product or communication issue rather than a quality one.
- **They are right and the metric is measuring a proxy.** For example scoring retrieval recall while they care about answer usefulness. That means changing what is measured.

The thing he would not do is defend the number. His whole position is that an eval set is a tool for finding out, not evidence in an argument — and a suite that only ever confirms the engineer was right has stopped being useful.

The same reasoning applies to the QA side of his experience, where the job involved agreeing acceptance criteria with dev and product before work started. Disagreement discovered at review time is usually a specification that was never actually shared.

Follow-up

"What if they want to ship despite a failing eval?" — State the risk plainly and in terms of what it costs, once. It is their call; the job is making sure it is an informed one.

Grounded in: Eval-set methodology, acceptance criteria work with dev and product teams, build log discipline

Q7.11 — How would he evaluate an agent, rather than a single model call?

engineer · asked by Engineer · agents, evaluation, langgraph, traces

By evaluating the path as well as the destination, because an agent can reach a correct answer by a bad route and a wrong answer by a mostly good one — and only the trace tells you which.

The layers he would score, using the graph behind this assistant as the example:

End to end. The same thing you would measure for any RAG system: is the final answer grounded, correct where a correct answer exists, and a refusal where it should be. This is the number a user experiences.

Per step. Each node has its own contract, and each can be tested with fixed inputs: - Did routing send an out-of-scope question straight to a refusal without spending a model call? - Did retrieval return the passages the labelled set says it should? - Did the grading step correctly judge weak retrieval as weak, and strong retrieval as strong? A grader that approves everything makes the retry loop pointless; one that rejects everything makes it expensive. - Did the rewrite step actually improve retrieval, or just change the words? - Did the verification step catch an answer whose claims were not in its citations?

Trajectory. How many iterations did it take, how often does it hit the retry cap, and what does a typical run cost in calls and latency. An agent that is correct but takes three loops on most questions has a latency problem that no end-to-end accuracy number shows.

Termination. Every loop must end. His graph allows one query rewrite and one regeneration, then declines or quotes the source, and those caps are themselves tested: his suite feeds it an answer with an invented number and asserts it regenerates exactly once and then falls back.

This is why the graph returns a trace of which nodes ran. Without it, evaluating an agent means evaluating only its final output, which hides most of the ways it can go wrong.

Follow-up

"Is per-step evaluation worth the effort?" — For the steps that decide control flow, yes. A mis-calibrated grader silently changes the cost and quality of every run.

Grounded in: Interview assistant LangGraph agent (route, retrieve, grade, rewrite, generate, verify), bounded retry, node trace, eval methodology

Part 8 — Quality Engineering and Testing

His first six months at Venera were quality engineering, and it is the most heavily quantified part of his record. These questions come up two ways: recruiters checking the numbers are real, and engineers checking whether the QA background is an asset or a sign he is not really a builder. Both are fair, and the answers below take both seriously.

Q8.1 — Walk me through the 150 automated test scripts.

hiring-manager · asked by Hiring Manager, Engineer · testing, selenium, automation, python, metrics

Those were built during his six-month internship at Venera Technologies, on their media quality control platform, in Python with Selenium WebDriver and the Page Object Model.

The measured outcomes: manual regression time down 65%, 78% coverage of P1 and P2 cases, and three critical regressions caught before release as a direct result.

The Page Object Model detail matters more than it sounds. It means the locators and interactions for each screen live in one class, and tests call methods on that class rather than addressing the DOM directly. The reason is maintenance: without it, a UI change breaks every test that touched that screen and you spend your time repairing the suite instead of extending it. A suite of 150 scripts that nobody maintains stops being run within a couple of months, so the structure is what makes the number mean anything.

The 65% figure is the one worth understanding, because it is the business case. Manual regression on a platform like that is a person working through a checklist before every release. Automating the bulk of it converts a recurring multi-day cost into a suite that runs unattended, which is what makes a faster release cadence possible rather than just cheaper.

The 78% coverage number is deliberately *not* 100%, and he would defend that. Automating the long tail of P3 and P4 cases costs more to build and maintain than the defects it catches are worth. Choosing where to stop is the judgement in test automation; writing more tests is not.

What he would be precise about: this was QA automation, not production application code. The skill that transfers is the discipline and the Python, not the domain.

Follow-up

"What did the three critical regressions have in common?" — Worth asking him directly; the specifics are the interesting part and they are not on the resume.

Grounded in: Internship metrics: 150 scripts, 65% regression reduction, 78% P1/P2 coverage, 3 critical regressions, Selenium + POM

Q8.2 — Tell me about the severity-1 incident.

behavioural · asked by Hiring Manager, Engineer · incident, production, severity-1, impact

During his internship he intercepted a severity-1 503 failure and coordinated a same-day hotfix, which avoided roughly four hours of user-facing downtime.

What makes it the item he is proudest of is that the counterfactual is concrete. Most QA work has a diffuse benefit — defects found, coverage raised — and it is hard to say what would have happened otherwise. Here the alternative is specific: without the interception, that failure reaches production and customers cannot use the service for about four hours.

Two things in it that generalise:

It was found because someone was looking. A 503 under specific conditions is not the kind of thing that announces itself in a green build. It was caught because there was a person checking release readiness rather than assuming it. That is exactly the habit he now applies to LLM systems, where the

characteristic failure also does not announce itself — nothing goes red, there is no stack trace, and the output looks like every other output.

Coordinating the fix was as much of the job as finding it. A severity-1 found and reported is half the work; getting a hotfix specified, built, verified and released the same day means working across dev and product under time pressure. He describes the coordination as the part that was actually hard.

The honest framing he would give: "zero critical defects across seven builds" is a team outcome he contributed to, not a solo achievement. He was the person doing the coverage and the release checks, not the only person responsible for quality.

Follow-up

"What did he change afterwards?" — Worth asking him directly. The generalisable answer he gives is that a failure found once should become a check that runs every time, which is the same reasoning behind adding cases to an eval set before fixing them.

Grounded in: Internship: severity-1 503 intercepted, same-day hotfix, ~4 hours downtime avoided, zero critical defects across 7 builds

Q8.3 — What did validating 120 API endpoints actually involve?

engineer · asked by Engineer, Hiring Manager · api-testing, postman, rest, aws, microservices

Postman-based validation across Venera's AWS microservices — 75 endpoints on Quasar, the cloud QC product, and 45 on Pulsar, the on-premise one. The measured outcome was a 98.3% pass rate with two production-blocking defects found and resolved before deployment.

What that work involves beyond sending requests: checking status codes and response schemas, validating behaviour at boundaries rather than only on the happy path, checking authentication and authorisation actually enforce what they claim, and verifying error responses are correct rather than merely present. Boundary value analysis is on his skills list and that is where it applies — the defects live at the edges of ranges, not in the middle.

The two production-blocking defects are the point of the exercise. A 98.3% pass rate is not impressive on its own; finding the two things that would have broken a deployment is. That is the same asymmetry as the severity-1 interception — most of the value of this kind of work is concentrated in a small number of findings.

Working across two products with different deployment models is also relevant background for what he does now. Quasar is cloud-native on AWS; Pulsar ships on-premise to customer sites. Those have genuinely different constraints — what you can assume about the network, how you release, what "reproduce the environment" means. Knowing Pulsar's constraints from the QA side is part of why he was a reasonable person to hand its first generative-AI feature to, and it is why the architecture of that feature starts from "this has to run offline".

Follow-up

"Was any of this automated?" — The Selenium suite was his automation work; the API validation was Postman-based. Worth asking him how much of the API layer he would automate today.

Grounded in: Internship: 120 endpoints, 75 Quasar / 45 Pulsar, 98.3% pass rate, 2 production-blocking defects, Postman, BVA

Q8.4 — Is he a QA engineer or a software engineer?

sceptical · asked by Hiring Manager, Recruiter · role, identity, career, objection

His title is Associate Software Engineer, and his primary work is building a generative-AI feature. So: software engineer, with a quality engineering background he has not discarded.

The full trajectory: six months as a graduate trainee and QA engineer on Venera's media QC platform, converted to full-time in August, and since then the primary engineer on the Pulsar template-generation

chatbot — owning the implementation, not only the testing.

The reason the question comes up is that the QA work is the most quantified part of his resume, so it takes up visual space. He keeps it prominent deliberately rather than burying it, which is a defensible choice: it is a year of real production experience, and the habits it produced are the ones that make his AI work different from a portfolio of notebooks.

The fair version of the concern underneath the question is whether someone from a QA background will default to verifying rather than building — waiting for a specification instead of forming a view, or over-testing instead of shipping. That is a real pattern.

The evidence against it in his case is that he owns the implementation of the Pulsar feature. He designed the retrieval approach, the grounding, and the validation layer. Owning the eval set alongside it is not a leftover QA obligation; he argues it is the right arrangement for a generative system, because the person who knows where it is likely to be wrong is the person who built it, and the risk of self-assessment is managed by the suite being written down and scored rather than performed by intuition.

The fair summary: he is a builder whose instincts were formed in quality engineering, which shows up as designing the failure path before the happy path.

Follow-up

"Would he take a QA role?" — He is looking for AI Engineer / ML Engineer roles. Worth asking him directly about how he would feel if a role turned out to be mostly testing.

Grounded in: Title (Associate Software Engineer), Pulsar chatbot implementation ownership, internship QA role, career narrative

Q8.5 — How does testing an LLM feature differ from testing normal software?

engineer · asked by Engineer, Hiring Manager · testing, llm, determinism, evaluation

The deterministic parts are tested normally. The model call is not testable that way at all, and conflating the two is where most teams go wrong.

Everything except the model call is ordinary code. Chunking, retrieval ranking, prompt assembly, response parsing, citation extraction, schema validation. These are pure functions over data and should have real unit tests. If your architecture does not let you test them without a network call, that is the finding — the fix is an adapter boundary at the provider, which this site has for exactly that reason.

The model call gets a stub in unit tests, and the stub should return the awkward responses: empty string, truncated JSON, a refusal, a 429, a timeout. Most production incidents in LLM systems are mishandled non-happy responses, and those are trivially testable with a fake and untestable against a live model.

Quality is evaluated, not asserted. `assert answer == "..."` is meaningless against a sampled model. What works is a labelled set scored on properties — groundedness, refusal correctness, schema validity, length — with a threshold that fails the build. That is a different artifact from a unit test, it costs money and time, and it belongs on a schedule or on changes to prompts and retrieval rather than on every commit.

Determinism is not available, even at temperature 0. Greedy decoding removes sampling randomness, but floating-point reduction order on GPUs varies, batch composition on a shared endpoint changes kernel selection, and providers re-point endpoint aliases at new checkpoints. Anyone building a suite on exact-match against a hosted model will have a flaky suite within a month.

The half people forget, which he insists on: **test that it refuses**. A must-decline set of questions the corpus does not cover, asserted to produce a refusal. Seven of his 36 cases on this site are exactly that.

Follow-up

"How often should the eval run?" — Retrieval eval on every content or retrieval change, because it is fast and free. Generation eval on prompt and model changes, because it is not.

Grounded in: Provider adapter boundary in core/providers.py, 36-case eval with 7 must-decline, Pulsar eval set, fallback path testing

Q8.6 — What does he mean by release readiness?

hiring-manager · asked by Hiring Manager, Engineer · release, sdlc, process, acceptance-criteria

A decision made against agreed criteria, rather than a feeling that things look fine.

His formal experience of it is at Venera, working with dev and product teams to define acceptance criteria, review test strategies, and make AWS cloud-native release readiness an explicit gate. The measured outcome was the defect escape rate coming down sprint over sprint.

What it involves in practice:

Acceptance criteria agreed before the work starts. This is the highest-leverage part and the one most often skipped. Disagreement discovered at review time is nearly always a specification that was never actually shared — someone built what they understood and someone else reviewed against what they understood. Agreeing it up front is cheaper than arguing about it at the end.

A defined set of checks that must pass, so "ready" is a state with a definition rather than a judgement call made under schedule pressure.

Knowing what is not covered. A release decision made without knowing the gaps is not informed. Saying "this has not been tested under concurrent load" is more useful than implying everything was checked.

Someone accountable for the call. Release readiness diffused across a team is release readiness nobody owns.

The reason this transfers to AI work directly: it is the same shape as an eval set. A statement of expected behaviour, agreed in advance, checked before shipping. When he says the evaluation set for the Pulsar chatbot covers both what it must generate and what it must refuse, that is acceptance criteria for a generative system.

Where he would temper it: release gates can become theatre if they are long checklists nobody reads. The value is in a small number of checks that would actually stop a release, not in comprehensiveness.

Follow-up

"What is his criterion for shipping an AI feature?" — The eval passes, the failure path is defined and tested, and the known gaps are written down. Not "the demo went well".

Grounded in: Current role: acceptance criteria and release readiness work, defect escape rate reduction, Pulsar eval set

Q8.7 — How did he manage 79 defects?

recruiter · asked by Recruiter, Hiring Manager · defect-management, zoho, process, triage

In Zoho, across his internship — 79 defects managed, 61 resolved, 10 of them critical, with test assets maintained in Bitbucket under structured pull request review.

What "managed" covers beyond logging them: writing a reproduction that someone else can follow, assigning severity honestly, tracking them to resolution, and verifying the fix rather than accepting that it was marked done. The last one is the part that makes the number meaningful — a defect marked resolved and never verified is a defect that comes back.

The skill in defect work that is not obvious is **severity assignment**. Marking everything critical destroys the signal and means nothing gets prioritised; marking things low to avoid friction means real problems sit in a backlog. Ten of 79 being critical is a plausible distribution, and calibrating that is a judgement

developed by being wrong a few times.

The other part is the write-up. A defect report that a developer cannot reproduce is work transferred rather than work done. Clear reproduction steps, expected versus actual, environment, and severity with a reason — that is the difference between a report that gets fixed and one that gets closed as "cannot reproduce".

The habit shows up directly in his AI work. The build log on this site documents twelve bugs in exactly that structure — symptom, cause, fix — which is a defect report format applied to his own work. And the discipline of adding a failing case to the eval set before fixing it is the same instinct as verifying a fix rather than trusting it.

Follow-up

"Why were 18 not resolved?" — Not covered on the resume. In any real backlog some are deferred, some are not reproducible, and some are working as designed. Worth asking him.

Grounded in: Internship: 79 defects in Zoho, 61 resolved, 10 critical, Bitbucket PR reviews, build log format

Q8.8 — What is the most useful thing QA taught him?

behavioural · asked by Hiring Manager, Engineer · lessons, quality, philosophy, transferable

That a system is not done when it produces output — it is done when you can show it produces the right output. That sentence is the one he uses, and everything he has built since is an application of it.

The experience behind it is watching a severity-1 failure get caught an hour before release because someone bothered to check. Nothing had flagged it. The build was green. It was found because checking was somebody's job.

Why that transfers unusually well to LLM systems: their characteristic failure is a confident wrong answer. There is no exception, no stack trace, no alert. The output is fluent and well-formed and wrong, and it looks exactly like the output that is fluent and well-formed and right. The only thing that catches it is having decided in advance what correct looks like and measuring against it — which is what a test strategy is, and what an eval set is.

The second thing, which is less about process: **the failure path is a design decision, not an error handler**. In QA you spend your time on what happens when things go wrong, and you notice that most systems have a well-considered happy path bolted to an afterthought. So he designs the failure first now. This site returns retrieved resume text directly when every model provider is unavailable, so a visitor gets an answer rather than a stack trace. The Pulsar chatbot refuses rather than guessing when it cannot produce a valid template. Both of those were decided before the happy path was tuned.

The third, smaller one: **a fix you have not verified is a hypothesis**. Which is why failing cases go into the eval before they are fixed.

Follow-up

"Is there anything QA taught him that he has had to unlearn?" — Worth asking. The plausible answer is the instinct to wait for a specification rather than form a view, which building a feature end to end forces you out of.

Grounded in: About section narrative, severity-1 interception, zero-key fallback design, Pulsar refusal behaviour, eval discipline

Q8.9 — Does he write tests for his own code?

engineer · asked by Engineer · testing, practice, discipline, self

Yes, and this site is the auditable example, which is a fair thing to check rather than take on trust.

What exists here: `eval_retrieval.py`, a 36-case retrieval suite that exits non-zero on failure and is documented as the thing to run after any change to the profile data or the retrieval code. It covers 29

questions that must retrieve the right resume section and 7 that must retrieve nothing at all. It is fast enough — about a second — that there is no friction excuse for skipping it.

For the interview corpus and the agent pipeline there is a retrieval eval over the larger corpus plus tests for the deterministic parts: the corpus parser, the retrieval fusion, the prompt registry and the graph executor, including the `stdlib` fallback path so the claim that it degrades gracefully is tested rather than asserted.

What he would be honest about: the coverage is not uniform. The evaluation suites are genuinely good because they are the thing he cares most about; the unit-test coverage of the web layer is thinner. And the eval is not yet wired into CI, which is on the published roadmap — a check that depends on someone remembering is a check that will eventually not run, and he says so rather than claiming otherwise.

The pattern worth noting is what he tests. He puts the effort into the suites that catch the failures he cannot see by reading code — retrieval behaviour, refusal behaviour, the degraded path — rather than into coverage percentage. Bug 05, where writing documentation reintroduced a bug, and bug 12, where a content edit broke a retrieval case, are both things no unit test would have caught and the eval did.

Follow-up

"What is his test coverage percentage?" — Not measured, and he would rather say that than quote a number. The eval pass rate is the figure he stands behind.

Grounded in: `eval_retrieval.py` (36 cases, non-zero exit), interview corpus tests, site roadmap on CI, build log bugs 05 and 12

Q8.10 — How would he improve quality on a team that has no tests?

hiring-manager · asked by Hiring Manager · process, influence, pragmatism, quality

By starting where the pain is, not by proposing a testing strategy.

A team without tests almost always knows it, and has reasons — usually that the last attempt produced a slow, brittle suite that everybody ignored. Arriving with a coverage target confirms their prior that testing is overhead. So:

Find out what actually breaks. Look at the last few incidents or the recurring defects. There is usually a small number of failure modes responsible for most of the pain, and they are the cheapest thing to protect.

Write the first tests for those, and make them fast. A suite that takes forty minutes will not be run. The value of `eval_retrieval.py` on this site is as much that it runs in a second as that it is thorough.

Make one thing that would have caught a real incident. Nothing argues for testing like a suite that catches something the week after it is added. That is worth more than any amount of advocacy.

Agree acceptance criteria before work starts, which is process rather than tooling and is often the higher-leverage change. Most quality problems he has seen are specification problems — someone built what they understood against a shared understanding that was never actually shared.

Do not gate on coverage. Coverage percentage measures lines executed, not behaviour verified, and targeting it produces tests written to satisfy the metric.

What he would avoid: being the person who blocks releases. Quality work that is experienced as friction gets routed around. The version that works is making the fast path also the safe path, so running the checks is easier than not running them.

The honest limit: he has done this as an individual contributor on a team that already had QA practice, not as someone who turned a team around. He would be describing an approach rather than a track record.

Follow-up

"What if the team pushes back?" — Pick the single most painful recurring failure and protect that one thing. Argument loses; a suite that catches a real regression wins.

Grounded in: Internship QA practice, acceptance criteria work, `eval_retrieval.py` speed and design, experience level

Part 9 — Production, Cloud and the SDLC

Questions about whether he can operate in a real engineering organisation: AWS, CI/CD, version control, on-premise versus cloud, and how he works inside a release process. This is the part of his record with the most ordinary professional substance, and the answers stay concrete about what he has done rather than what he has read about.

Q9.1 — What is his AWS experience?

recruiter · asked by Recruiter, Hiring Manager · aws, cloud, microservices, experience

Hands-on, from working on Venera's cloud-native platform, and it is worth being precise about the shape of it.

Quasar is Venera's cloud media quality control product, built as AWS cloud-native microservices. He has worked on it since February 2026 — first validating 75 REST endpoints across those microservices during his internship, then continuing across the full SDLC including AWS cloud-native release readiness as a defined part of his current role.

So the experience is: working within an AWS microservices architecture, validating and testing services deployed on it, and owning release readiness for that environment. He understands how the product is deployed, how services interact, and what makes a release safe there.

What he would not claim: he is not a cloud architect. He has not designed an AWS architecture from scratch, has not owned infrastructure-as-code for a system, has not managed IAM policy at organisation scale, and has not run cost optimisation across an account. If a role expects someone to own the cloud infrastructure, that is beyond what he has done.

Azure also appears on his skills list, at a lower level of depth than AWS.

The related experience that is arguably more relevant to AI roles: the contrast between Quasar and Pulsar. Quasar is cloud-native; Pulsar ships on-premise to customer sites. Having worked on both means he has a real feel for what changes when you cannot assume the network — which is exactly the constraint that shaped the generative-AI feature he built, where the model has to run inside the customer's deployment with no outbound calls.

Follow-up

"Which AWS services specifically?" — Not enumerated on the resume. Worth asking him directly rather than having an assistant guess at service names.

Grounded in: Quasar AWS cloud-native microservices, 75 endpoints validated, AWS release readiness in current role, skills list

Q9.2 — What is the difference between working on a cloud product and an on-premise one?

engineer · asked by Engineer, Hiring Manager · on-premise, cloud, deployment, constraints

He has worked on both at Venera — Quasar in the cloud on AWS, Pulsar on-premise at customer sites — and the differences that matter are about control and assumptions.

You control the cloud environment; you do not control an on-premise one. In the cloud you know the machine, the versions, the network, and you can change them. On-premise, the software runs on hardware you have never seen, on a version the customer chose to install, possibly with no route to the internet. Everything has to work under assumptions rather than knowledge.

Releases are different events. A cloud release is something you do; an on-premise release is something a customer chooses to apply, possibly months later. So you support multiple versions simultaneously, cannot hotfix everyone at once, and cannot roll back centrally. That changes the bar for what ships.

Debugging is indirect. No live logs, no metrics dashboard, no reproducing it on staging with the customer's data. You get a description and whatever diagnostics the product itself collects.

Data does not leave. This is usually the reason customers choose on-premise, and it is the constraint that determined the architecture of the generative-AI feature he built. A chatbot that posted an operator's description to a hosted model API would violate the premise of the product they bought. So the model runs inside the deployment on open weights, the embedding model ships with it, and the vector store runs locally.

He would say the on-premise constraint made him a better engineer on that feature, because it removed the easy escape hatches. You cannot fix a quality shortfall by upgrading to a bigger model when the model is bounded by the customer's hardware, so the quality has to come from retrieval, grounding and validation — which is where it should come from anyway.

Follow-up

"Which would he rather work on?" — Worth asking him. The interesting answer is that the constrained one produced the better architecture.

Grounded in: Quasar (cloud, AWS) and Pulsar (on-premise), Pulsar chatbot offline requirement, Qwen open-weight choice, endpoint validation across both

Q9.3 — What is his experience with CI/CD?

recruiter · asked by Recruiter, Engineer · cicd, jenkins, automation, pipelines

Working within CI/CD pipelines rather than owning them, which is the accurate framing for his level.

On his skills list: Jenkins, CI/CD, Git and GitHub, Bitbucket, Gradle, Maven. In practice that means his automated test suites ran as part of the pipeline, he worked with builds and release processes as part of release readiness, and he maintained test assets in Bitbucket under structured pull request review.

The part he has genuinely owned is what goes *into* a pipeline as a quality gate — the automated regression suite that runs before a release, and the decision about what must pass. That is the intersection of CI and his actual expertise.

What he has not done: built a pipeline from scratch, owned build infrastructure, managed deployment automation for a service, or set up the release tooling for a team. If a role needs a platform or DevOps engineer, this is not deep enough.

There is one honest gap he flags about his own work: `eval_retrieval.py` on this site exits non-zero on failure specifically so it *can* be a CI gate, and it is not wired into CI yet. It is on the published roadmap as "expand the eval set and wire it into CI so a bad retrieval change cannot merge". He would rather list that as an unfinished item than imply the automation exists — which is also the answer to what he would do first on a team: make the check that matters run automatically rather than by convention.

Follow-up

"What would he put in CI for an LLM system?" — The retrieval eval, because it is fast and free, plus unit tests on the deterministic layers. The generation eval on a schedule or on prompt changes, because it costs money and time.

Grounded in: Skills list (Jenkins, CI/CD, Bitbucket, Gradle, Maven), test suites in pipeline, `eval_retrieval.py` exit code, site roadmap

Q9.4 — How does he work with version control and code review?

recruiter · asked by Recruiter, Engineer · git, code-review, collaboration, process

Bitbucket at Venera with structured pull request reviews, and Git and GitHub for his own work. This site's source is public.

What he would say about review specifically, because it is where the useful answer is: the most valuable thing a review catches is not a bug, it is a decision that looks wrong out of context. The code

on this site carries comments explaining *why* rather than *what*, specifically at the points where the correct-looking thing is wrong. One example: a template variable is accessed with bracket notation rather than attribute notation, with a comment saying the attribute form resolves a built-in method instead of the data — because that bug shipped once and the attribute form looks correct enough that anyone tidying the file would reintroduce it. That comment exists for a future reviewer.

He treats the same principle in the other direction: a review comment asking "why" is more useful than one proposing an alternative, because it surfaces whether the author had a reason.

On commit and branch practice he is conventional — small changes, descriptive messages, review before merge. On his own repository he keeps the build log as a record of bugs and their causes, which is effectively a review artifact for himself.

What he has less of: he has worked in structured review as a participant, not as someone setting review standards for a team or owning a branching strategy. And he has worked on a small team rather than on a repository with many concurrent contributors.

Follow-up

"What does he want from a reviewer?" — Given his level, he would say challenge on design decisions rather than style. He has explicitly named code review from someone more senior as what he needs to grow.

Grounded in: Bitbucket with structured PR reviews, public GitHub repository, in-code comments explaining non-obvious decisions, build log bug 01

Q9.5 — How does he handle a production incident?

behavioural · asked by Hiring Manager, Engineer · incident, production, process, debugging

His direct experience is the severity-1 503 he intercepted before release, where a same-day hotfix was coordinated and roughly four hours of user-facing downtime avoided. So the honest framing is that he has caught and coordinated a serious issue, rather than that he has been on call for a major outage.

The approach he would describe:

Contain before diagnosing. Reduce impact first — roll back, disable the feature, fail over — and understand it afterwards. The instinct to find the root cause while users are affected gets the order wrong.

Establish what changed. Most incidents follow a change. Deployment, configuration, a dependency, a data shift.

Reproduce with the real inputs, which requires having logged them. This is the part that decides whether debugging is possible at all. His position on LLM systems specifically is that inputs, outputs, retrieved chunk ids and model versions must be logged for a sample of traffic from day one, because without that trace an incident becomes archaeology.

Communicate while working. Coordinating the hotfix across dev and product was, in his description, the part that was actually hard about the severity-1 — finding it was quicker than getting a fix specified, built, verified and released the same day.

Make the failure a check. A failure found once should become something that runs every time. It is the same rule he applies to eval sets: add the failing case before fixing it, so the fix is provable and the case cannot silently return.

What he has not done: been the on-call owner of a system at 3am, run a formal incident command, or written a postmortem for a major outage. The build log on this site is the closest artifact — twelve bugs with symptom, cause and fix — and it is self-directed rather than organisational.

Follow-up

"What is the first thing he would set up on a new AI system?" — Logging of inputs, outputs, retrieved sources and model version for a sample of traffic. Everything else can be reconstructed from that.

Grounded in: Severity-1 503 interception and same-day hotfix, build log, eval discipline, experience level

Q9.6 — How does this site get deployed?

engineer · asked by Engineer · deployment, vercel, docker, hosting, free-tier

It is deliberately deployable four ways, because the point of the exercise was that it should not depend on any one platform.

The live site runs on Vercel as a serverless function. The repository also carries a `render.yaml` for Render, a `Dockerfile` targeting port 7860 for Hugging Face Spaces, and a `Procfile` with Unicorn for anything that reads one. The deployment plan documents the trade-offs of each — Render's free instance sleeps after about fifteen minutes idle so the first visit takes around fifty seconds to wake, Spaces does not sleep, Vercel has the fastest cold start.

The design constraint behind all of it is cost: the whole thing runs on free tiers. That is why retrieval is in-process rather than a hosted vector database, why the model providers are free-tier first with automatic failover, and why the dependency footprint is kept small enough to fit a serverless bundle.

One deployment-specific bug is on the build log and is worth mentioning because it was caught in review rather than in production. On the free serverless tier, the platform kills a slow request before the application's own timeout fires — which would have killed the provider failover, since the whole point of the chain is to try the next provider when one is slow. The app's request timeout defaulted higher than the platform's function limit, so it would never have reached the fallback. The fix was documenting a lower timeout for that platform so the app gives up and degrades gracefully inside the platform's window.

He also notes in the deployment plan which platform to avoid and why: PythonAnywhere's free tier blocks outbound HTTPS to non-whitelisted hosts, which breaks every model provider call.

Follow-up

"Does the serverless rate limiter work?" — Only per invocation, since the process does not persist. He documents that as harmless here rather than pretending it is a real limiter.

Grounded in: vercel.json, render.yaml, Dockerfile, Procfile, DEPLOYMENT_PLAN.md, build log bug 07, in-process rate limiter note

Q9.7 — How does he think about monitoring an AI feature in production?

engineer · asked by Engineer, Hiring Manager · monitoring, observability, drift, production

He would separate what he has done from what he knows should be done, because this is an area where his production exposure is short.

What he has built: the assistant on this site reports which provider actually produced each answer, and the health endpoint separates a cheap "is a key configured" check from an opt-in probe that makes one real call and reports the true error. That split exists because of a specific failure — a valid key was configured, health reported green, and every answer was silently coming from the local fallback because a CDN was rejecting the request. A health check that confirms configuration is not a health check.

What he would set up on any new system, in order:

- **Traffic sampling.** Inputs, outputs, retrieved chunk ids and scores, prompt version, model version. Without this, debugging is guesswork. It is the first thing, not the last.
- **Real health probes,** not configuration checks.

- **Cost and token counts per request**, tagged by feature. Teams that cannot attribute spend cannot reduce it, and a runaway loop will find that out for you.
- **Refusal and fallback rates.** A rising fallback rate means the primary path is failing quietly, which is exactly the failure that hides. A falling refusal rate might mean the system has started answering things it should decline.
- **Latency split by stage** — retrieval versus generation — because a single end-to-end number hides which one is the problem.
- **Query drift.** People asking things the corpus never covered is the signal that the corpus needs extending, and it is visible in the refusal rate before anyone complains.

What he has not done: run a monitoring stack for model quality in production, set up drift detection with labels arriving late, or owned alerting for an AI system. That is a genuine gap.

Follow-up

"What single metric would he alert on?" — The fallback rate. It is the one that goes up silently when everything else looks fine.

Grounded in: Engine status reporting, /api/health probe design, build log bugs 08 and 11, rate limiting, provider chain

Q9.8 — Has he worked in an Agile process?

recruiter · asked by Recruiter, Hiring Manager · agile, sprints, process, collaboration

Yes — sprint-based work at Venera across the full SDLC, working with dev and product teams.

The concrete artifacts of it in his record: defining acceptance criteria with dev and product before work starts, reviewing test strategies, and defect escape rate tracked and reduced sprint over sprint. That last one is a sprint-level metric and the fact that it moved is evidence of a feedback loop actually operating rather than ceremony being performed.

The part he would emphasise as substance rather than process vocabulary: **acceptance criteria agreed up front.** Most of the quality problems he has seen come from a specification that was never really shared — someone builds what they understood, someone else reviews against what they understood, and the gap surfaces at the end when it is most expensive. Agreeing it before the work starts is the cheapest quality intervention available and it is mostly a conversation.

He would also say the useful thing about working with product on acceptance criteria for an AI feature is that it forces the refusal question into the open. What should the system do when it cannot confidently answer is a product decision as much as an engineering one, and it does not get decided unless somebody puts it on the table early. The must-refuse half of the Pulsar chatbot's evaluation set exists because that conversation happened.

What he would not claim: he has not run a sprint, owned a backlog, or been a scrum master, and he has worked on a small team rather than in a large multi-team programme.

Follow-up

"How does estimation work for AI features?" — Worth asking him. The honest general answer is that the retrieval and evaluation work is estimable and the "make the output good enough" work is not, which is why having an eval set early matters for planning as well as quality.

Grounded in: Full SDLC work with dev and product teams, acceptance criteria definition, defect escape rate reduction, Pulsar eval refusal cases

Q9.9 — What would he need from us to be productive quickly?

hiring-manager · asked by Hiring Manager · onboarding, ramp-up, support, growth

Four things, and the first two are the ones that would actually change his ramp time.

Access to what already exists. For any AI feature: the current prompts, whatever evaluation exists, and a sample of real traffic — inputs, outputs and what people actually ask. A day with real query logs teaches more about a system than a week of reading code, because it shows what the system is being asked to do rather than what it was designed to do.

A clear statement of what failure costs. Whether a wrong answer is an annoyance or an incident determines the whole architecture. The Pulsar chatbot's design was largely determined by "must run offline" and "an invalid template reaches an operator", and knowing those on day one was worth more than a month of exploration would have been.

Code review from someone more senior. He names this himself as what he needs. He has about a year of professional experience, one production AI feature, and a narrow set of reference points. The gap between him and a senior engineer is mostly having watched more systems fail in more ways, and review is how that transfers fastest.

Exposure to larger scale. He has not worked at high traffic volumes. If the role involves that, he would want to be honest that it is new rather than pretend otherwise.

What he brings that shortens his own ramp: he has worked inside a real SDLC, is comfortable with release processes and structured review, and he has a habit of building the measurement before the feature, which means his first contribution on an undocumented system is usually an eval set that tells everyone where it actually stands.

Follow-up

"What would make him unproductive?" — A system with no way to tell whether a change helped. He would likely spend his first week building that, and would want agreement that this is worth the time.

Grounded in: Experience level, stated need for senior code review, Pulsar constraints, eval-first methodology, SDLC experience

Q9.10 — Can he work independently, or does he need direction?

hiring-manager · asked by Hiring Manager · autonomy, ownership, working-style, collaboration

The evidence points to independent with judgement about when to check, which is the combination you want at his level.

The case for independence: he was handed Venera's first generative-AI feature — something with no internal precedent — and built it from scratch, owning both implementation and testing. That is not ticket work. Everything outside his job is self-directed too: the document Q&A chatbot, the NLP pipelines with a prompt-evaluation loop, and this site, which is roughly three thousand lines carrying a retrieval index, nine provider integrations, an evaluation suite and a published build log. Nobody asked for any of it.

The case for it being judged rather than stubborn: the Pulsar feature was built alongside an external AI consultant, engaged because the company had no internal precedent for this work. He describes that as the right arrangement rather than a limitation, and he names code review from someone more senior as what he wants. Someone who could not ask for help would not say that.

The place he is most likely to need direction is on scope and priority rather than execution. His instinct is to build the measurement infrastructure first — an eval set before the feature — and that is right often enough to be a strength and wrong sometimes when the priority is to ship something rough and learn from it. A manager saying "not this time" would be useful to him.

The honest limit: one year of experience means fewer reference points, so his independent judgement on unfamiliar problems is less reliable than it will be. He is good at recognising what he does not know, which is the version of this that works.

Follow-up

"Would he push back on a decision he disagreed with?" — The site is full of documented disagreements with conventional practice — BM25 over embeddings, no framework where one was not needed — each with reasoning attached. The pattern is arguing with evidence rather than deferring or digging in.

Grounded in: Pulsar feature ownership with consultant support, self-directed projects, this site's scope, stated need for code review, documented design decisions

Q9.11 — What happens when an AI vendor retires the model your product uses?

engineer · asked by Engineer, Hiring Manager · vendors, model-retirement, resilience, production

It happened to this site, to both of its providers at once, which makes it a real answer rather than a hypothetical.

What happened. The default Groq model and the default Gemini model the site was configured for had both been retired by their vendors. Every call returned a 404 naming the model as no longer available. Because the provider chain treats any failure as "try the next one" and ends in an extractive fallback, the site kept answering throughout — from its local index, visibly labelled as such — rather than breaking.

How it was found. Every answer reports which engine produced it, so it was obvious that nothing was coming from a model. The health probe, which makes one real call, returned the vendor's error message directly.

How the replacements were chosen. Not by picking the newest name. He listed the models each key could use and timed the actual grounded question, several runs each, against the site's real budget: a six-second timeout, inside a serverless platform that kills requests at ten. The newest Gemini flash model took seven to nine seconds and returned "high demand" errors on repeat calls — on this platform, strictly worse than a lighter model answering in about two. On Groq, a Qwen model with its built-in reasoning turned off answered in under a second. Those became the defaults, with a second Gemini model as an automatic retry for capacity errors.

What it taught. Vendors change models on their own schedule, so the model name belongs in configuration you can change without a deploy, the system needs more than one provider, and the failure path has to be designed before you need it. Those three things turned what could have been an outage into a configuration change.

The same lesson applies with more force to a product like Pulsar, which is exactly why it runs on self-hosted open-weight models: nobody can retire them out from under a customer.

Follow-up

"How would you get warned earlier?" — Alert on the fallback rate. It rose the moment the models disappeared, long before anyone would have complained.

Grounded in: Retired default models (Groq and Gemini), provider chain and extractive fallback, engine reporting, benchmarked replacements, LLM_TIMEOUT=6, Pulsar open-weight choice

Part 10 — Projects

Deep dives on the things he has built outside the Venera work. Visitors ask about these to test whether a project on a resume is a finished system or a tutorial that got a screenshot. The answers include what each one does not do, because that is usually the more informative half.

Q10.1 — Tell me about the document Q&A chatbot.

engineer · asked by Recruiter, Engineer, Hiring Manager · rag, langchain, pinecone, project, streamlit

A retrieval-augmented question-answering system over a 200-page knowledge base, with grounded, citation-friendly answers. The measured result was roughly a 70% reduction in retrieval time across that corpus.

The stack: a chunking and embedding pipeline building an index in Pinecone as the vector store, LangChain orchestrating retrieval and generation, the OpenAI API for the model, and Streamlit as the interface.

The part he treats as the actual work is the grounding pass rather than the plumbing. Answers are constrained to the retrieved passages, and unsupported responses — output asserting something the retrieved material does not contain — are tracked as a metric rather than estimated. That distinction matters: "we told it to stay grounded" is a prompt, and "we measured how often it did not" is engineering. It is the same instinct that produced the eval sets on everything else he has built.

Where it sits in his record: this is the project that demonstrates the conventional RAG stack — embeddings, a managed vector database, LangChain, a hosted model. It is the counterpart to the Pulsar chatbot, which had to do the same job with none of those available, and to this site's original resume assistant, which deliberately used none of them because its corpus was thirty chunks. Having built all three is what lets him argue about when each is appropriate rather than defending one as correct — and the site itself switched to embeddings once its corpus grew and measurement said to.

What it does not have: it is a self-directed project rather than a product, so it has no real users, no operational history, and no exposure to the failure modes that only appear over time. He would weight it well below the production work.

Follow-up

"Where did the 70% come from?" — It is a retrieval-time improvement over the baseline approach to that corpus, not a claim about answer quality. Worth asking him for the specifics of the comparison.

Grounded in: Document Q&A Chatbot project: 200-page knowledge base, LangChain, OpenAI API, Pinecone, Streamlit, ~70% faster retrieval, unsupported answers tracked

Q10.2 — What are the NLP pipelines he built with Hugging Face?

engineer · asked by Engineer, Recruiter · nlp, hugging-face, transformers, prompt-evaluation

Transformer-based text pipelines for classification and summarisation, built on Hugging Face Transformers — tokenisation, inference, and fine-tuning experiments.

The component he would actually talk about is the prompt-evaluation loop attached to it. Each prompt variant is scored against a hand-labelled question set, so comparing two prompts produces a number rather than an impression, and unsupported answers are tracked as a metric. He built it because iterating on a prompt by running two or three examples and deciding it looks better is close to worthless — generative output varies enough between runs that a small sample confirms whatever you already expected.

That loop is the conceptual ancestor of the evaluation sets on everything else: the 36-case retrieval eval on this site, and the must-generate plus must-refuse evaluation set for the Pulsar chatbot at work. He describes "learn a technique, then build the thing that tells you whether it is working" as the most

consistent habit across his projects, and this is where it started.

The project also lists LangGraph in its stack as the direction he was extending toward — multi-agent, stateful workflows — which is the honest positioning: the pipelines are built, the LangGraph part was exploratory.

What it is not: it is not a trained model, not a published model, and not deployed anywhere with real traffic. The fine-tuning is described as experiments, which is accurate — he has not run a production fine-tune and does not claim to.

Follow-up

"How big was the hand-labelled set?" — Modest, and he would say so. Thirty cases you wrote yourself is a real measurement; a benchmark it is not.

Grounded in: NLP Pipelines with Hugging Face project: classification, summarisation, tokenisation, fine-tuning experiments, prompt-evaluation loop, hand-labelled set, LangGraph

Q10.3 — Tell me about the Selenium Grid automation project.

recruiter · asked by Recruiter, Engineer · selenium-grid, testing, parallel, automation

A parallel cross-browser regression suite running across three browsers and two operating systems, which cut manual cross-browser effort by about 80% and halved pre-release validation time for the team.

Selenium Grid is the piece that makes this more than a test suite: it distributes test execution across multiple machines and browser configurations so the suite runs in parallel rather than sequentially. Cross-browser testing is the case where that matters most, because the same suite has to run N times against N configurations, and run serially that is N times the wall-clock cost.

The engineering judgement in it is about what to parallelise and how to keep tests independent. Tests that share state — a fixture, a user account, a database row — cannot run concurrently without flaking, and a flaky parallel suite is worse than a slow serial one because people stop trusting it. So the work is as much about test isolation as about the grid configuration.

Where this fits his story: it is quality engineering rather than AI work, and he keeps it on the resume because the pattern generalises. Halving pre-release validation time is the same category of contribution as the 65% manual regression reduction — making the safe path fast enough that people take it. Quality work that is experienced as friction gets routed around, and the way to prevent that is to make checking cheap.

He would rank this below the AI projects in relevance for an AI engineering role, and above them in demonstrating that he has shipped infrastructure that a team depended on daily.

Follow-up

"Is Selenium relevant to AI work?" — Not directly. The transferable parts are test isolation, parallel execution and the judgement about what is worth automating.

Grounded in: Selenium Grid Automation project: 3 browsers, 2 operating systems, -80% manual cross-browser effort, halved pre-release validation

Q10.4 — What are the older projects — the chat app and the Java CRUD system?

recruiter · asked by Recruiter · android, kotlin, java, early-projects

Both pre-date his move toward AI work and he keeps them on the resume as evidence of range rather than as current skills.

The real-time chat app is an Android application in Kotlin with Firebase as the realtime backend, with secure authentication and sub-second message sync, verified stable under concurrent multi-user load. It connects to his NullClass internship, which was Android development — building and optimising

mobile applications, UI performance tuning, Firebase integration and mobile SDLC delivery.

The student data entry system is a Java servlet CRUD application over MySQL, where optimising the JDBC queries reduced average database response time by 30%.

Their honest relevance to an AI engineering role is limited, and he would say so. What they show is that he has built working software in more than one language and paradigm — mobile, backend, database — before specialising.

There is one thing worth knowing about these two specifically, which is that they are the reason his retrieval eval set exists. Someone asked his assistant "what has he actually shipped with LLMs?" and it returned exactly these two projects — a Kotlin chat app and a Java CRUD app. The cause was unweighted query expansion letting broad synonyms match every project chunk equally while the one discriminating term was outvoted. He wrote an eval set of 29 questions, reproduced the failure at 26/29, and fixed the scoring until it reached 29/29. So these two projects are, indirectly, why every retrieval change on this site is now scored.

Follow-up

"Would he still write Kotlin or Java?" — He has both on his skills list and would not claim current depth in either. Python is his primary language by a wide margin.

Grounded in: Real-Time Chat App and Student Data Entry System projects, NullClass internship, build log bug 02, eval progression 26/29 to 29/29

Q10.5 — Which project is he proudest of, and why?

behavioural · asked by Hiring Manager, Recruiter · projects, motivation, self-assessment

The Pulsar template-generation chatbot at Venera, because it shipped into a product with constraints he did not choose and consequences he did not control.

His reasoning, which is worth stating because it says something about how he evaluates work: a side project is built on the stack its author picked, targeting requirements its author set, and judged by its author. The Pulsar feature had to run offline inside an on-premise deployment, on models small enough for customer hardware, producing output conforming to a schema he did not design, for operators whose failure case is running quality control with checks they did not ask for. Working inside that is most of the engineering.

The runner-up is this site, for a different reason: it is where he got to make every decision and document the reasoning, including the ones that went against convention. Choosing BM25 over embeddings because the corpus is thirty chunks. Nine model providers behind one adapter so a rate limit on one is invisible. A system that answers with zero API keys configured. And a published log of twelve bugs that shipped broken before he caught them — including one where writing up a bug reintroduced it.

He would say the build log is the part he is most pleased with, which is an unusual thing to be proud of. The argument is that a portfolio of things that worked says nothing about how someone handles things that do not, and that publishing the failures is a hedge against overclaiming. It also means the site's central claim — that he measures rather than assumes — is testable rather than asserted, because the eval is in the repository and you can run it.

Follow-up

"Which taught him the most?" — The eval set, across all of them. Every significant thing he has learned about retrieval came from a number moving in a direction he did not predict.

Grounded in: Pulsar chatbot constraints, portfolio assistant design decisions, build log (12 bugs), eval_retrieval.py

Q10.6 — How much of this site did he actually build?

sceptical · asked by Engineer, Hiring Manager · portfolio, authorship, credibility

The source is public, which is the real answer — it is on GitHub and the reasoning is documented in the code rather than only in the write-up.

What is in it: a Flask application, a BM25 retrieval index built in-process over chunks derived from a single profile module, nine model provider adapters behind one interface with automatic failover, a prompt and grounding layer, a 36-case retrieval evaluation suite, a front end in vanilla JavaScript with no framework, and deployment configuration for four hosting platforms. Roughly three thousand lines.

The signals that it is genuinely his, if you want to check rather than take it on trust:

- **The comments explain decisions, not syntax.** They cluster exactly where the correct-looking thing is wrong — the template variable that must be accessed by key because attribute access resolves a built-in method, the synonym weight that must stay below 1.0 so an expansion cannot outvote a typed term, the breakpoint branch in the sidebar handler.
- **The build log documents twelve bugs with cause and fix**, including embarrassing ones. Bug 10 is a button that had never worked on desktop. Bug 11 is a status badge that displayed a provider name while a different engine was answering.
- **The eval set is runnable.** `python eval_retrieval.py` reproduces the 36/36 claim, and it prints the two cases it does not pass cleanly along with why.
- **The statistics on the page are computed at boot** from the real index and the real test suite, because hand-written ones drifted three times.

What he had help with elsewhere: the Pulsar feature at work was built alongside an external AI consultant, and the resume says so. This site is his.

Follow-up

"Can I break the assistant?" — There is a button on the About page that invites you to try. Ask it something the resume does not cover and it should decline.

Grounded in: Public GitHub repository, ~3000 lines, build log bugs 01, 10 and 11, `eval_retrieval.py`, computed About statistics

Q10.7 — What would he build next?

engineer · asked by Engineer, Hiring Manager · roadmap, future, improvement

The published roadmap for this site, plus what the Pulsar work exposed as gaps.

On this site, listed publicly as unfinished rather than claimed as done:

- **Hybrid retrieval** — BM25 plus embeddings — once the corpus grew past a few hundred chunks. That threshold has now been crossed by the interview corpus, and the retrieval over it is hybrid, so this one is done and the rest are not.
- **Wire the eval into CI** so a bad retrieval change cannot merge. It already exits non-zero for exactly this purpose; it is not automated yet, and a check that depends on someone remembering will eventually not run.
- **Stream tokens to the client** rather than waiting for the full response. A user-experience gap rather than a correctness one.

Beyond it, the things he would want to build to close real gaps in his experience:

- **A reranker in a production path.** He has not had one; it is the single highest-value addition to most RAG systems and his corpora have been small enough to avoid needing it, which is a reason rather than an excuse.

- **A real fine-tune with a proper before-and-after evaluation.** He has built the dataset and eval scaffolding and run experiments; he has not taken one to production.
- **An agent with actual tool access**, with the permission and cost controls that requires. His current graph has no side effects, which is what makes it safe, and that is also why it does not teach him the hard parts.

The pattern in that list is that he names what he has not done rather than what he would like to demonstrate — which is consistent with a site whose main feature is a published log of its own bugs.

Follow-up

"Why has he not done the CI wiring already?" — No reason he defends. It is listed as a gap.

Grounded in: Site roadmap (hybrid retrieval, eval in CI, streaming), interview corpus hybrid retrieval, no reranker in production, fine-tuning limits

Q10.8 — Do his projects have real users?

sceptical · asked by Hiring Manager, Engineer · users, production, scale, honesty

One does. The rest do not, and that distinction is the right way to weigh them.

The Pulsar template-generation chatbot is a feature in a commercial product that Venera's customers run. Operators use it. That is the only one of his systems with users who did not choose to look at his portfolio.

This site has visitors — recruiters and engineers, which is the intended audience — and at least one reported bug came from a real user rather than from testing: long answers opened scrolled to their last line, so you landed on the closing sentence and had to scroll back up. That is logged as bug 09 and credited as user-reported. So there is some real usage signal, but it is a portfolio, not a service.

The document Q&A chatbot, the NLP pipelines and the Selenium work are self-directed or internal. The Selenium Grid suite was used by the team daily, which is a form of real usage — infrastructure other people depended on — but not external users.

Why he would rather state this plainly than blur it: a system with users behaves differently from one without. Users ask things you did not anticipate, use it in ways you did not design for, and surface the failure modes that only appear over time. He has had a small amount of that, and he has not had the experience of maintaining a system through a year of accumulated edge cases and drift, which he names as where the genuinely hard parts of this work live.

Follow-up

"How much traffic does the site get?" — Not instrumented in a way that is published, and he would rather say that than quote a number.

Grounded in: Pulsar chatbot in commercial product, build log bug 09 (user-reported), Selenium Grid team usage, experience level

Q10.9 — What is the most technically interesting thing he has built?

engineer · asked by Engineer · technical-depth, design, retrieval, interesting

The retrieval layer on this site, which sounds like the least impressive option and is the one with the most actual engineering in it.

The naive version is BM25 over resume chunks. The interesting problem is that it does not work, for a reason that is specific and generalises: recruiters ask "can he do X" while a resume says "built X". The vocabulary of hiring and the vocabulary of a CV barely overlap, so lexical matching fails on exactly the questions that matter most.

What he built to close that gap, each piece added because the eval caught a failure:

- **Light suffix stemming**, so "LLMs" matches "LLM" and "hallucinating" matches "hallucination", with a minimum stem length so short technical tokens like "aws" and "api" survive intact.
- **Phrase normalisation**, collapsing known multi-word terms to a canonical token *before* tokenising. This exists because "large language model" was matching his *programming languages* list — token overlap with no meaning overlap.
- **Weighted query expansion**. A synonym layer maps hiring language onto resume vocabulary, but expansions carry 0.3 weight against 1.0 for terms the visitor actually typed. Unweighted, a broad synonym like "shipped" expanding to "project" matched every project chunk equally and drowned out the single discriminating term.
- **Doubled title and tag tokens**, so a tag match outweighs an incidental body match — but only 2×, because some tags are shared across a whole category and carry no discriminating signal.
- **Empty as an honest return value**. Retrieval returns nothing when nothing matches, rather than substituting defaults, because that empty result is the only signal meaning "not covered".

Every one of those is a small decision with a documented reason and a test case. That is the part he finds interesting: the system is a few hundred lines and almost all of it is judgement.

Follow-up

"Why not just use embeddings?" — Because at thirty chunks it would have added a network hop, a cold start and a bill for recall he could not measure a gain from. The vocabulary gap was the real problem, and embeddings would not have fixed the phrase-sense failure either.

Grounded in: core/rag.py (stemming, phrase normalisation, 0.3 expansion weight, doubled tags, empty returns), build log bugs 02, 03 and 04

Q10.10 — How do his projects connect to each other?

hiring-manager · asked by Hiring Manager, Recruiter · narrative, progression, coherence

Three retrieval systems with three different architectures, and a measurement habit that carries through all of them.

The progression. The document Q&A chatbot is the conventional stack — embeddings, Pinecone, LangChain, a hosted model. This site's resume assistant was the opposite: no embeddings, no vector database, no framework, no required API key, because its corpus was thirty chunks and the conventional stack would have added cost and failure modes for unmeasurable gain. When the site's corpus grew to 163 questions and answers, keyword search found the right answer for under half of reworded questions and embeddings for about nine in ten, so the live assistant now leads with embeddings — same principle, different corpus, different answer. The Pulsar chatbot at work is a third point — embeddings and a vector store, but everything running locally inside an on-premise deployment because no outbound call is permitted.

Having built all three is what makes his opinions about retrieval architecture arguments rather than preferences. He can say when a vector database is worth it because he has chosen for and against one on defensible grounds.

The habit. Every one of them has an evaluation set. The prompt-evaluation loop on the NLP pipelines scores prompt variants against a hand-labelled question set. This site has 36 retrieval cases, seven of which assert that nothing should be retrieved. The Pulsar chatbot has a set covering both what it must generate and what it must refuse. That is consistent enough across independent projects to be a trait.

The QA thread underneath. The failure path is designed before the happy path in all of them — this site answers with zero API keys, the Pulsar chatbot refuses rather than guessing, and the document Q&A project tracks unsupported answers as a metric. That is the six months of quality engineering

showing up in the architecture rather than in a bullet point.

Follow-up

"Is the coherence deliberate or retrospective?" — The eval sets predate the site's write-up, and the build log is dated by the bugs it describes. Worth judging from the repository rather than the narrative.

Grounded in: Document Q&A (Pinecone/LangChain), portfolio assistant (BM25, no deps), Pulsar chatbot (local embeddings), three eval sets, failure-path designs

Part 11 — How This Website Works

Engineers visiting the site tend to ask about the thing in front of them. That is a fair test: it is the one system of his that anyone can inspect end to end, including its source, its evaluation suite and its published list of bugs. These answers describe it as it is, including the parts that are unfinished.

Q11.1 — How does the assistant on this site work, end to end?

engineer · asked by Engineer, Hiring Manager · architecture, rag, pipeline, self-referential

A LangGraph agent over about a hundred pages of questions and answers about his record. Every step has a fallback, so the chat answers even when parts of it are down.

- 1. Route.** An empty or oversized question is turned away before anything runs.
- 2. Retrieve.** The question is embedded and compared with every answer in the corpus, and a BM25 keyword pass votes alongside at a quarter weight, fused by rank. Measured on questions reworded the way visitors actually type, this finds the right answer in the top three about nine times in ten, against roughly half for keyword search alone. The embedding call has a two-second deadline; if it misses, BM25 and a local latent-semantic index answer in milliseconds instead.
- 3. Grade.** If there is not enough evidence to answer, a follow-up is re-read with the previous question once; otherwise the assistant declines without calling a model at all.
- 4. Generate.** A LangChain chain assembles the retrieved answers, two retrieved examples of good answers, and a versioned prompt chosen by a scored evaluation, then sends them to the first healthy provider — Groq, then Gemini, and seven more behind the same adapter. The prompt tells the model to decline anything outside his professional record.
- 5. Verify.** Every number in the answer must appear in the retrieved sources, and every citation must point at a retrieved entry. A failure triggers one stricter regeneration, then the source is quoted instead.

Each answer carries its citations and a trace of the path it took — nodes, engines, prompt version, timings — under "How this was answered". If the agent itself were ever unavailable, the chat falls back to the site's original assistant: BM25 over resume chunks, which still passes its own 36-case evaluation.

The honest caveat: the site's rule has been that one module, `data/profile.py`, is the source of truth for the pages, the resume index, the system prompt and the resume PDF. The interview corpus is a second, hand-written source, so the two could in principle drift apart. What contains that is that every corpus answer lists the resume facts it rests on, and the build refuses an answer that lists none — a discipline, not a structural guarantee.

Follow-up

"What happens if I ask something his record doesn't cover?" — It says so and gives his email. Declining is measured: the model declined all 12 out-of-scope test questions in the prompt evaluation, including personal ones that retrieval alone could not tell apart from real ones.

Grounded in: Interview agent (route, retrieve, grade, generate, verify), retrieval eval (embeddings vs keyword-only recall@3), prompt eval decline results, provider chain, resume-index fallback with 36-case eval

Q11.2 — Why nine model providers? Isn't that over-engineering for a portfolio?

sceptical · asked by Engineer, Hiring Manager · providers, failover, resilience, design-decision

It would be, if it cost nine implementations. It costs roughly two.

Seven of the nine — Groq, Cerebras, OpenRouter, Mistral, Together, Ollama and OpenAI — speak the same OpenAI-style chat completions format, so they share a single adapter and differ only in base URL, default model and environment variable. Gemini and Anthropic have different wire formats and get their own. Adding a provider is a class with four attributes.

The reason for it is his own: a portfolio that dies when one free tier rate-limits is worse than no portfolio. Recruiters look at these at unpredictable times, and free tiers have unpredictable limits. With two or more keys set, a 429 on one provider is invisible to the visitor — the chain moves on.

It has earned its keep in a way he did not anticipate. Model vendors retire models on their own schedule. In a single session both configured providers' default models turned out to have been retired, and one provider's current model returned "high demand" errors on repeat calls. Because the chain and the extractive fallback existed, the site kept answering throughout — degraded, but never broken — and the fix was a changed default in one class rather than an outage.

The decision he would defend most is the extractive fallback at the bottom of the chain. Retrieval is local, so it cannot fail with the network. With zero API keys configured, the site still answers by returning the matching resume sections. That is the QA habit showing up in architecture: design the failure path first, then make the happy path better than it.

Follow-up

"Which provider is live right now?" — The badge next to the chat shows it, and it is corrected from each real answer rather than from configuration, after it once displayed a provider name while a different engine was answering.

Grounded in: core/providers.py adapter design, nine providers with seven OpenAI-compatible, zero-key fallback, build log bug 11, retired-model incident

Q11.3 — What is the single most important design decision on this site?

engineer · asked by Engineer, Hiring Manager · design, source-of-truth, architecture, decisions

That one file is the source of truth for everything.

data/profile.py holds every fact: roles, dates, metrics, projects, skills, the About write-up and the build log. The HTML pages render from it. The retrieval index is built from it at import time. The system prompt pulls his name, role and email from it. And the resume PDF is now generated from it by a script, rather than maintained as a separate document.

The reason it is the most important decision is the class of bug it eliminates. When the site and the assistant read from different places, they drift — someone updates the page and forgets the chatbot's knowledge, or the other way round, and both halves look correct in isolation. Nobody notices until a recruiter gets two different answers. That bug is hard to find precisely because nothing is broken; the two sources are just quietly inconsistent.

He has a concrete example of the drift it prevents. The resume PDF used to be a hand-maintained export, and the CGPA in it and the CGPA on the page were different numbers. Generating the PDF from the profile module closed that permanently. A content change now updates the page, the assistant and the PDF together, or none of them.

It also makes updating the portfolio a one-file edit, which matters more than it sounds: the easier it is to keep something current, the more likely it is to stay current.

The trade-off, which he would acknowledge: it couples everything to one module's shape. Changing a field name touches the template, the chunker and the resume builder. For a system this size that is a good trade. For a large system it would argue for a schema with validation.

And there is now one deliberate exception, which he would rather state than hide: the interview corpus behind the assistant is written by hand, in Markdown, because a hundred pages of considered answers cannot be generated from a list of facts without reading like it. That makes it a second source that could drift from the profile. It is contained by rule rather than by structure — every answer names the resume facts it rests on, and the build refuses one that names none — which is weaker than the guarantee the rest of the site has, and he says so.

Follow-up

"Does the eval run when the profile changes?" — It should, and the README says to. It is not enforced in CI yet, which is on the roadmap.

Grounded in: Design decision "one file is the source of truth", profile.py, build_resume.py, CGPA drift, knowledge.py chunk derivation

Q11.4 — How are the statistics on the About page produced?

engineer · asked by Engineer · measurement, statistics, honesty, build-log

They are measured from the running system when the app starts, rather than typed into the page.

The chunk count comes from the actual retrieval index. The evaluation score comes from actually running the evaluation suite — the app imports the eval module at boot, runs it quietly, and writes the pass count and the breakdown of must-match versus must-decline questions into the About section. If the eval fails to import or run, the stat keeps its previous value rather than breaking the page, because a statistic should never be the reason a portfolio returns an error.

The reason is bug 06 on the build log. Those figures were originally written by hand, in a section whose entire claim is that the numbers are measured. They stopped being true three times — every time content was added, the chunk count and index size quietly went stale, and nothing flagged it. A page asserting "these are measured" while displaying remembered numbers was a small dishonesty that kept recurring.

The fix is a principle he now applies generally: **if you claim a number, derive it**. A figure computed from the system cannot drift from the system.

The retrieval latency figure — 0.04 milliseconds — is the one exception, because timing at boot on a serverless function would measure cold-start noise rather than retrieval. That number was measured over 2,900 runs and is documented as such. He would rather state which figures are live and which are recorded than let them all look equally measured.

Follow-up

"Can I reproduce the eval score?" — Yes. Clone the repository and run `python eval_retrieval.py`. It prints the score, the per-question ranking with `-v`, and the known limitations.

Grounded in: app.py _measured_about(), build log bug 06, eval_retrieval.py run_quiet, 0.04 ms measured over 2, 900 runs

Q11.5 — How is the site hosted, and what does it cost?

recruiter · asked by Recruiter, Engineer · hosting, cost, vercel, free-tier

It runs on free tiers, and costs nothing to operate. That was a design constraint from the start rather than a happy accident.

The live site is a serverless Flask function on Vercel. The repository also carries configuration for Render, for a Docker image targeting Hugging Face Spaces, and a Procfile for Gunicorn, with a written deployment plan comparing them: Render's free instance sleeps after about fifteen minutes idle so the first visit takes around fifty seconds to wake; Spaces does not sleep; Vercel has the fastest cold start but a hard per-request time limit.

What keeping it free forced, architecturally:

- **Retrieval is in-process**, so there is no vector database bill and nothing to rate-limit.
- **Model providers are free-tier first**, with automatic failover so one tier's limit is not an outage.
- **The request timeout is tuned to the platform**. Vercel's free plan cuts a function off at ten seconds, so the app gives up on a slow provider at six and degrades gracefully inside that window — a lesson recorded as bug 07, caught in review before it shipped.

- **Model choice is measured against that budget.** When a newer model turned out to take around nine seconds per answer, it was replaced by a lighter one answering in about two, because on this platform a slow answer is indistinguishable from no answer.
- **Dependencies stay small**, so the bundle fits a serverless function.

The deploy itself is scripted: environment variables are pushed from a local `.env` to the platform without echoing any key, and the deploy runs in one command.

Follow-up

"Would this architecture survive real traffic?" — Retrieval would, trivially. The free-tier model quotas would not; at real volume you would pay for inference or self-host.

Grounded in: `vercel.json`, `render.yaml`, `Dockerfile`, `Procfile`, `DEPLOYMENT_PLAN.md`, build log bug 07, `LLM_TIMEOUT=6`, deploy scripts

Q11.6 — How does the site defend itself against abuse?

engineer · asked by Engineer, Security · security, rate-limiting, guardrails, xss

Proportionately — it is a portfolio, not a bank — and he is explicit about which defences are real and which are limited.

Rate limiting. Twenty chat requests per minute per client IP, enforced in-process. It exists to stop a stray script draining a free-tier quota, not as a security boundary. On a serverless platform the process does not persist between invocations, so the limiter effectively resets; he documents that as harmless here rather than pretending it is a real control.

Output escaping. The model's answer is HTML-escaped before the small markdown renderer runs, so no response can inject markup or script into the page. This is the one people miss — model output is untrusted input to whatever renders it, however much you trust your own prompt.

Input bounds. Questions are capped at a thousand characters; conversation history is trimmed to the last eight turns, each capped in length, with roles validated. Nothing in the request body is trusted because it arrived there.

No privileges to abuse. The assistant has no tools, no write access and no ability to send anything. A successful prompt injection gets you a wrongly-worded answer about his resume. That — not any filter — is the real reason it is safe.

Secrets never leave the server. Keys live in environment variables, `.env` is excluded from both git and the deploy upload, and the deploy script pushes keys without printing them.

Health checks that do not burn quota. The deep probe that makes a real model call is opt-in, so an uptime monitor pinging the endpoint every few minutes does not spend the free tier.

Follow-up

"Could someone extract the system prompt?" — Probably, and it would not matter. Nothing in it is secret, which is the right way to write one.

Grounded in: `RATE_LIMIT 20/min`, HTML escaping in `app.js`, `MAX_QUESTION_CHARS 1000`, `MAX_HISTORY_TURNS 8`, opt-in health probe, `.gitignore/.vercelignore`

Q11.7 — What is the "build log" and why is it on a portfolio?

hiring-manager · asked by Hiring Manager, Engineer · build-log, bugs, honesty, culture

It is a list of twelve bugs that shipped broken on this site before he caught them, each with the symptom, the cause and the fix. It sits collapsed at the bottom of the About page, deliberately secondary.

Its stated reason: how a system fails says more than how it looks when it works. A portfolio of successes tells you nothing about how someone behaves when something goes wrong, which is most

of the job.

Some of what is in it:

- A template that resolved a Python built-in method instead of his data, returning a 500 on every page.
- An assistant that answered "what has he shipped with LLMs?" with a Kotlin chat app and a Java CRUD app, which is why the evaluation suite exists.
- An earlier version that answered questions outside the resume by attaching his career summary.
- **Documenting a bug reintroduced it** — writing up the previous item added its trigger words to the indexed corpus and dropped the eval from 36/36 to 34/36.
- A health check that reported green while every answer was silently coming from the fallback.
- A sidebar button that had never worked on desktop.
- A status badge naming a provider that was not the one answering.
- A missing stopword that only surfaced when an unrelated content edit changed which chunk won.

Two things he would point out about it. First, several were found by the evaluation suite rather than by him, which is the argument for having one. Second, one was reported by a real user — long answers opened at their last line — and it is credited as such rather than presented as self-discovered.

The wording is intentionally generic about the specific terms involved, for a reason that is itself in the log: the build log is indexed like everything else, so naming the exact words a retrieval bug involved puts them back in the corpus.

Follow-up

"Isn't publishing bugs risky for a job search?" — He would say the opposite. Anyone who has shipped software has a list like this; the question is whether they can talk about it.

Grounded in: BUGS in profile.py (12 entries), build log bugs 01–12, About page collapsed build log

Q11.8 — What is not finished on this site?

engineer · asked by Engineer, Hiring Manager · roadmap, gaps, honesty, limits

The published roadmap lists these, and they are genuinely undone rather than modestly undersold.

The evaluation suite is not in CI. It exits non-zero on failure specifically so it can gate a merge, and it does not yet. A check that depends on someone remembering to run it will eventually not run.

Answers are not streamed. The client waits for the whole response before showing anything. With current response times of a couple of seconds this is tolerable; it is still a user-experience gap.

There is no reranker. Retrieval over thirty resume chunks does not need one. The larger interview corpus would benefit from one, and it is the addition he would make next.

Traffic is not instrumented. He can see what the providers report and what the eval says, but there is no sampled log of real questions and answers — which is the first thing he would build on a real system and the one thing a portfolio has managed without.

Unit-test coverage of the web layer is thin. The retrieval behaviour is well tested; the Flask routes and the front end much less so.

And the honest meta-point: several things that were on this list are now done — hybrid retrieval over a larger corpus, a stateful agent graph, dark mode, a generated resume — which is the pattern he would want you to notice. The roadmap is written to be completed, not to look ambitious.

Follow-up

"Why publish a list of gaps?" — Because every system has one, and a portfolio that shows none is either tiny or not being honest. It also stops the site overclaiming.

Grounded in: ABOUT["next"] roadmap, eval not in CI, no streaming, no reranker, test coverage gaps

Q11.9 — How does the front end work without a framework?

engineer · asked by Engineer · frontend, javascript, css, accessibility

Plain server-rendered HTML from Jinja templates, one stylesheet and one vanilla JavaScript file of a few hundred lines. No React, no build step.

The reasoning is proportionality. The page is a handful of sections and one chat panel. A framework would add a build pipeline, a dependency tree and a bundle for interactions that are a few event listeners. Server rendering also means the content is present in the HTML, so it is readable by crawlers and by anyone with scripts disabled.

The pieces worth noting:

- **Design tokens.** Every colour in the stylesheet is a semantic variable with a light and a dark value, so the theme is one attribute on the root element. The theme is resolved by a tiny inline script before first paint — that is what prevents a white flash for dark-mode visitors — follows the operating system until the visitor chooses, and only remembers an explicit choice.
- **Fixed scales.** A seven-step type ramp and a four-pixel spacing grid, replacing an accumulation of one-off sizes that read as slightly off without being locatable.
- **Accessibility.** Visible keyboard focus everywhere, a skip link, reduced-motion support, and ARIA on the navigation, the question picker and the chat log so screen readers announce new answers.
- **Safe rendering.** Model output is escaped before a minimal markdown pass, so answers cannot inject markup.
- **Deep links.** A `?ask=` parameter opens the page with a specific question already answered, so a link can point at an answer rather than an empty chat box.

He would not claim front-end specialism — he builds interfaces that work and hold up to scrutiny, which is a different thing from designing them.

Follow-up

"Why do long answers scroll to their top?" — Because a user reported landing on the last line of long answers. Chat convention pins the newest message to the bottom, which is wrong for anything taller than the panel.

Grounded in: templates/index.html, static/css/style.css design tokens, static/js/app.js, `?ask=` parameter, build log bug 09

Q11.10 — How is the interview corpus behind this assistant built?

engineer · asked by Engineer, Hiring Manager · corpus, rag, hybrid-retrieval, langgraph, interview

It is roughly a hundred pages of the questions visitors actually ask about him — fit, the Venera work, retrieval, prompting, fine-tuning, evaluation, projects, behaviour — each with the answer the assistant should give, and each tagged with the resume facts it rests on.

It is authored as Markdown because a person has to be able to read and edit it; it is a document first and a dataset second. A parser turns it into structured records — question, answer, follow-up, tags, difficulty, who asks it, and what it is grounded in — and reports its own length in words and pages against a stated constant, so "a hundred pages" is a measurement rather than a claim.

The "grounded in" field is the discipline that matters. The corpus speaks about his real record, so every answer has to be traceable to something the profile actually says. An answer that cites nothing is a candidate for fabrication, and an audit step flags it.

Over that corpus sit the techniques he works with, each for a specific job:

- **Hybrid retrieval** — lexical and semantic passes fused by rank — because at this size, and with this much variety in phrasing, lexical retrieval alone starts missing paraphrases.
- **A versioned prompt registry with an evaluation harness**, so prompt changes are scored rather than eyeballed.
- **A fine-tuning dataset** built from the question–answer pairs, with contamination checks against the evaluation set, plus retrieved few-shot examples as the zero-cost stand-in for serving a tuned model.
- **A LangChain pipeline and a LangGraph agent** that routes, retrieves, grades, retries once, generates and verifies — running on real LangChain and LangGraph where installed, with a small stdlib fallback so the site never hard-fails.

Follow-up

"Why write the answers rather than letting the model generate them?" — Because the answers are claims about a real person. Written and grounded answers can be checked; generated ones would drift toward whatever sounds impressive.

Grounded in: `interview/corpus/build.py`, `grounded_in` field, 500 words/page constant, interview retrieval, prompt registry, fine-tuning dataset, LangGraph agent

Part 12 — Behavioural Questions

The "tell me about a time" questions. Interviewers use these to check whether the technical story holds up when told as events rather than as claims. Every answer here is built from something that is on his record, told in situation–action–result order, with the result stated as honestly as the record supports.

Q12.1 — Tell me about a time he caught a serious problem before it reached users.

behavioural · asked by Hiring Manager, Recruiter · incident, severity-1, star, impact

Situation. During his internship at Venera Technologies he was doing release checks on the media quality control platform ahead of a build going out.

Task. Release readiness — confirming the build was safe to ship, not just that the tests were green.

Action. He found a severity-1 failure: a service returning 503 under conditions the build had not been exercised against. Rather than logging it and moving on, he raised it immediately and coordinated the same-day hotfix across dev and product — getting the fix specified, built, verified and into the release rather than letting the build slip or ship with it.

Result. The failure never reached customers. The estimate is roughly four hours of user-facing downtime avoided. It is also one of the reasons his internship record shows zero critical defects reaching production across seven consecutive builds.

What he takes from it, which is the part worth listening for: nothing had flagged it. The build was green. It was found because checking was somebody's job and that person bothered. He applies the same lesson to LLM systems, where the characteristic failure is also silent — a fluent, confident, wrong answer with no error anywhere — and the only defence is having decided in advance what to check.

He would also be precise that the zero-defect record is a team outcome he contributed to, not something he achieved alone.

Follow-up

"What was the hardest part?" — The coordination, not the finding. Getting a fix through several people under a same-day deadline was the part he describes as difficult.

Grounded in: Internship: severity-1 503 intercepted, same-day hotfix, ~4 hours downtime avoided, zero critical defects across 7 builds

Q12.2 — Tell me about a mistake he made and what he did about it.

behavioural · asked by Hiring Manager · mistakes, ownership, debugging, star

Situation. An early version of his portfolio assistant was designed so that retrieval always returned *something*. When nothing in the resume matched a question, it substituted a few default sections so the model always had material to answer from.

Task. It felt like robustness. It was a design error, and he was the one who made it.

Action. The failure surfaced when someone asked about his hobbies — which the resume does not cover — and the assistant replied with his career summary, reading as though it had answered the question. He traced it back to his own substitution logic: by filling empty results with defaults he had destroyed the only signal that meant "the resume doesn't cover this", so neither the prompt nor the fallback had any way to decline. He removed the substitution so retrieval returns empty honestly, made the application decline deterministically before any model call when that happens, and added seven must-decline cases to the evaluation suite so the behaviour could never quietly regress.

Result. The assistant now declines out-of-scope questions reliably, with his email address, and the evaluation suite enforces it — 36 of 36 cases passing, seven of which assert that nothing should be retrieved.

The reflection he would add: the mistake came from optimising for the system always having an answer, which is the wrong goal for anything grounded. "Always answers" and "never makes things up" are in tension, and he had quietly chosen the wrong side. He publishes it on the site's build log as bug 04.

Follow-up

"Would you have caught it without a user asking?" — Probably not quickly. That is exactly why the must-decline half of an eval set is the part he now writes first.

Grounded in: Build log bug 04, empty-retrieval design, 7 must-decline eval cases, 36/36

Q12.3 — Tell me about a time he had to learn something new quickly.

behavioural · asked by Hiring Manager, Recruiter · learning, adaptability, genai, star

Situation. When he converted to full-time at Venera in August, he was moved onto the company's first generative-AI feature — a chatbot to generate QC templates for Pulsar, their legacy on-premise product. There was no internal precedent; nobody at the company had built one.

Task. Become the primary engineer on it, owning both the implementation and the testing, having spent the previous six months in quality engineering rather than building AI systems in production.

Action. He worked alongside an external AI consultant brought in precisely because the expertise was not in-house, and he learned the parts that were new to him in the context of the real constraint rather than in the abstract: running open-weight Qwen models entirely inside an on-premise deployment with no outbound calls, local embedding-based retrieval over the template corpus, and structured-output validation against the existing schema. The parts he already knew — test strategy, release readiness, the product and its domain — he brought straight across, which is why he wrote the evaluation set himself.

Result. The feature was built from scratch and is his primary ongoing work. He has owned it since August.

He would frame the speed honestly: he did not learn generative AI from zero in August. He had been building retrieval systems and LLM pipelines on his own for months — a LangChain and Pinecone document Q&A system, NLP pipelines on Hugging Face, and this site. What was new was doing it inside someone else's constraints, and having expert support available made that a reasonable bet for the company rather than a reckless one.

Follow-up

"What would he do differently with more time?" — Worth asking him directly; that is the most specific thing he can offer about the project.

Grounded in: Pulsar chatbot since August, external AI consultant, prior self-directed projects, QA background on the same platform

Q12.4 — Tell me about a time data told him something he didn't expect.

behavioural · asked by Hiring Manager, Engineer · evaluation, surprise, measurement, star

Situation. After fixing a retrieval bug on his assistant, he wrote it up in the site's public build log, which is part of the corpus the assistant searches.

Task. It was documentation. It was not supposed to change any behaviour.

Action. The next run of the evaluation suite dropped from 36 of 36 to 34 of 36. Nothing in the retrieval code had changed. He traced it and found that the write-up itself was the cause: describing the bug had named the exact terms involved, and because the build log is indexed like every other part of the site, those words were now in the corpus and reintroduced the very matching behaviour the bug was about. He rewrote the entry in deliberately generic language and the suite returned to 36 of 36.

Result. Fixed, and a permanent change in how he thinks about it: in a retrieval system the corpus *is* the index, so any content edit is a behaviour change. The build log's wording has stayed generic about specific terms ever since, and the log itself says why.

It happened again in a different form later — editing his experience section broke a case that had passed for months, because it exposed a missing stopword. Both times, reading the code would never have found it; the only reason he knew was that a scored suite ran after a change that "could not" affect retrieval.

Follow-up

"What is the lesson for a team?" — Run the retrieval eval on content changes, not only on code changes. Most teams do not.

Grounded in: Build log bugs 05 and 12, eval drop 36/36 to 34/36, indexed build log

Q12.5 — Tell me about a time he disagreed with the conventional approach.

behavioural · asked by Hiring Manager, Engineer · judgement, trade-offs, decisions, star

Situation. Building the assistant for this site, the conventional recipe was obvious: embed the resume, store the vectors in a hosted vector database, retrieve by similarity.

Task. Choose a retrieval approach for a corpus of about thirty short chunks, on free hosting, that had to keep working with no API keys at all.

Action. He went against the default and used BM25 lexical retrieval in-process, then wrote down why: embeddings would add a network round trip, a cold start and a bill, for a recall gain he could not measure at that size. He identified that the real problem was not similarity at all but vocabulary — recruiters ask "can he do X" and resumes say "built X" — and solved that directly with stemming, phrase normalisation and a weighted synonym layer. Then he built an evaluation suite to prove the choice rather than just argue it.

Result. Retrieval runs in about 0.04 milliseconds, cannot fail with the network, costs nothing, and passes 36 of 36 evaluation cases. The site answers even when every model provider is down.

The part that makes it judgement rather than contrarianism: he made the *opposite* choice on the Pulsar chatbot at work, where the vocabulary gap between how operators describe intent and how the schema encodes it is genuinely semantic, and that system uses embeddings. And on the larger interview corpus behind this assistant he moved to hybrid retrieval, exactly as the site's own roadmap had said he would once the corpus grew. The position was never "BM25 is better"; it was "choose from the corpus, and measure".

Follow-up

"How did he know he was right?" — The eval set. Without it, it would have been an opinion.

Grounded in: Design decision "BM25 instead of embeddings", 0.04 ms, 36/36, Pulsar embedding retrieval, interview hybrid retrieval

Q12.6 — Tell me about working with people outside engineering.

behavioural · asked by Hiring Manager, Recruiter · collaboration, stakeholders, product, communication

Situation. Throughout his time at Venera he has worked with dev and product teams on the media QC platform.

Task. Define what "done" means for features before they are built, and agree when a release is actually ready.

Action. His concrete contribution has been defining acceptance criteria with product before work starts, reviewing test strategies with developers, and making release readiness an explicit, agreed gate rather than a feeling. The principle he works from is that most quality problems are specification problems —

someone built what they understood, someone else reviewed against what *they* understood, and the gap only appears at the end when it is most expensive to close. Agreeing it up front is mostly a conversation, and it is the cheapest quality intervention there is.

Result. The defect escape rate on the platform came down sprint over sprint, which is the measurable outcome of that work.

It carried directly into the generative-AI feature. Deciding what the Pulsar chatbot should *refuse* to do was as much a product decision as an engineering one — it is not the model's call — and getting it into the evaluation set meant putting that question in front of product explicitly. Without that conversation, the refusal behaviour would have been whatever the model happened to do.

He would describe himself as better at making requirements explicit than at persuading people, which is a fair self-assessment at his stage.

Follow-up

"How does he handle a stakeholder who wants to ship anyway?" — State the risk plainly, once, in terms of what it costs. Then it is their decision; his job is to make sure it is an informed one.

Grounded in: Current role: acceptance criteria with dev and product, release readiness, defect escape rate reduction, Pulsar refusal cases

Q12.7 — Tell me about a time a user found a problem he missed.

behavioural · asked by Hiring Manager · user-feedback, humility, ux, star

Situation. His assistant gives answers that can run to several paragraphs, displayed in a chat panel.

Task. It worked in his testing. A user reported that it did not work for them.

Action. The report was that long answers opened scrolled to their *last* line — you landed on the closing sentence and had to scroll back up to find the start. He had followed standard chat behaviour, which pins the newest message to the bottom of the panel. That is right for short turns and wrong for anything taller than the panel, and his own tests had mostly used short questions. He changed it so a new answer scrolls its first line to the top, clamped so a short reply does not leave empty space.

Result. Fixed, and recorded on the site's build log as bug 09 — explicitly credited as reported by a user rather than caught in testing.

Why he keeps it in the log, and credits it that way: the most useful thing about it is that it was not found by him. His testing reflected his own usage pattern, and real users use things differently. Attributing it honestly matters to him more than looking like he catches everything, and it is a cheap reminder that his evaluation sets measure the cases he thought of, not the ones he did not.

Follow-up

"How would he find this class of issue earlier?" — Watch someone else use it. No test suite would have caught a behaviour that was working exactly as designed.

Grounded in: Build log bug 09 (user-reported), `scrollToStartOf` in `app.js`

Q12.8 — Tell me about a time he worked under a hard constraint.

behavioural · asked by Hiring Manager, Engineer · constraints, on-premise, design, star

Situation. Pulsar, the product his generative-AI feature lives in, is deployed on-premise at customer sites. Those customers run it that way specifically so their media and workflows stay on their own infrastructure.

Task. Build a chatbot that turns an operator's plain-language description into a valid QC template — without any call to an external AI service.

Action. Every decision followed from that constraint. The model had to run inside the deployment, so the feature uses open-weight Qwen models. Retrieval had to be local too, so the embedding model ships with the product and the vector store runs alongside it. And because the model is bounded by what customer hardware can run, the usual escape hatch — upgrading to a bigger model when quality falls short — was not available. So quality had to come from retrieval and grounding instead, and the validation layer that checks every generated template against the schema became load-bearing rather than a safety net.

Result. A working feature built from scratch that respects the product's core promise, which he has owned since August.

What he would say he learned: the constraint produced a better design than an unconstrained API would have. It forced the parts that generalise — getting the right context in front of the model and refusing to let invalid output through — to carry the weight, instead of relying on the model being excellent.

Follow-up

"Would he choose open weights without the constraint?" — For this feature, probably, for the reproducibility. But he is clear the constraint came from the product, not from him.

Grounded in: Pulsar on-premise deployment, Qwen open-weight models, local embeddings and vector store, schema validation

Q12.9 — Tell me about a time a system looked healthy but wasn't.

behavioural · asked by Engineer, Hiring Manager · observability, debugging, health-checks, star

Situation. His assistant had a valid API key configured, and its health check reported the model engine as live.

Task. Everything looked green. The answers, though, were coming back as extracted resume text rather than written responses.

Action. He noticed because every answer reports which engine actually produced it, and it kept saying the local index rather than the model. He traced two faults stacked on top of each other. The HTTP layer was sending Python's default user-agent, which the CDN in front of the provider rejected outright, so no request ever reached the model. And the health check only asked whether a key was *present*, never whether the provider would actually answer — so the graceful fallback was silently hiding a total failure. He set a real user-agent on every request, and split health into a cheap "is it configured" check and an opt-in probe that makes one real call and reports the true error.

Result. Both fixed, recorded as bug 08. The same wrong assumption resurfaced later in the status badge visitors see, which showed a provider name while a different engine answered; that is bug 11, and it is now corrected from each real answer.

The principle he took away: make the system report what it actually did, not what it was configured to do. It paid off again later, when vendors retired the models this site used — the answers naming their own engine made it obvious within minutes.

Follow-up

"Why does graceful degradation make this worse?" — Because it converts a loud failure into a quiet one. The fallback was working exactly as designed, which is precisely what hid the problem.

Grounded in: Build log bugs 08 and 11, /api/health probe design, engine name on every answer, retired-model incident

Q12.10 — Tell me about something he's proud of that isn't technical.

behavioural · asked by Recruiter, Hiring Manager · values, honesty, character

The honest version is that the resume does not cover much outside his work, and this assistant will not invent hobbies or personal achievements for him. If you want to know him as a person, that is a conversation to have with him directly — his email is jeetendrapatel1711@gmail.com.

What the record does show is a disposition, and it is visible in how he presents his own work rather than in any single achievement:

- **He publishes his failures.** The site carries a log of twelve bugs that shipped broken before he caught them, including embarrassing ones, and one credited to a user rather than to himself.
- **He instructs his own assistant not to oversell him.** The system prompt behind this chat explicitly tells it to be honest that he is early-career and not to present him as a senior researcher. Most people building a self-promotional tool would do the opposite.
- **He credits help.** The resume states that the Pulsar feature was built with an external AI consultant, rather than implying he did it alone.
- **He lists what is unfinished.** The site's roadmap publishes its own gaps.

He would probably say the thing he values is being trustworthy about what he claims — that if he says something works, it has been measured, and if he does not know, he says so. That is as much a character trait as an engineering practice, and the site is built around it.

Follow-up

"Is that just good marketing?" — It is a strategy that only works if it is true. A recruiter can check every claim here against the resume and the source code.

Grounded in: Build log (12 bugs, bug 09 user-credited), SYSTEM_PROMPT honesty instruction, consultant credit on resume, published roadmap, contact email

Q12.11 — Tell me about a time a check passed but the system was still broken.

behavioural · asked by Engineer, Hiring Manager · health-checks, latency, debugging, star

Situation. After replacing a retired model, he redeployed this site and ran its health probe — the check that makes one real call to the model provider to prove the key works.

Task. Confirm the live site was answering with the model again.

Action. The probe passed: reachable, "ok". But when he then asked the live site a real question, the answer came back from the local fallback instead of the model. The probe had asked the model to reply with a single word, which it did instantly. A real question carries a full retrieved context and needs a paragraph back, and with the new model that took around nine seconds — past the site's six-second timeout, so the app gave up and fell back exactly as designed. The probe was measuring whether the model was reachable, not whether it could do the actual job in time. He timed several models on the real question, switched to one answering in about two seconds, redeployed, and confirmed with real questions rather than the probe.

Result. The live site answers from the model in about two seconds, and the verification steps in the deployment guide now include asking a real question, not just running the probe.

What makes it worth telling is that it is the same lesson as an earlier bug on his build log, one level deeper. The first time, a health check reported a key as *present* when it was not *working*; he fixed that by making the probe do a real call. This time the probe did a real call and still did not represent real use. A check is only as good as its resemblance to the thing you actually care about.

Follow-up

"Would a better probe fix it?" — Partly. The dependable fix is to verify with the real workload: a real question, with a real context, against the real time budget.

Grounded in: Health probe design (/api/health?probe=1), 9 s vs 2 s model timing, LLM_TIMEOUT=6, build log bug 08, deployment verification steps

Part 13 — Hard and Sceptical Questions

The questions a careful interviewer asks when they suspect a portfolio is better than the person behind it. They deserve straight answers, and the answers below do not try to win the argument — they concede what should be conceded and hold the ground the record actually supports.

Q13.1 — Isn't "AI Engineer" an inflated title for someone a year out of college?

sceptical · asked by Hiring Manager, Recruiter · title, level, honesty, objection

It would be if he claimed seniority with it. He does not.

His actual job title is Associate Software Engineer at Venera Technologies. "AI Engineer" is the role he is targeting and the kind of work he is doing: since converting to full-time in August, his primary work has been building a generative-AI feature — a retrieval-augmented chatbot on Qwen models that generates QC templates for Pulsar. Describing that as AI engineering work is accurate. Describing him as a senior AI engineer would not be, and nothing on this site does.

The site goes further than most in the other direction. The assistant you are talking to is instructed, in its system prompt, to be honest that he is early-career and not to oversell him as a senior researcher. The pitch describes his edge as production discipline, not depth of experience. The build log publishes his mistakes. That is not how someone inflating a title behaves.

What "AI Engineer" means in current industry usage is also narrower than it sounds: building products on top of models — retrieval, prompting, integration, evaluation, guardrails — rather than training them. That is precisely the work he has done in production and outside it. By that definition, the title describes what he does; the question is only what level, and the honest answer is junior to mid.

If a role reserves the title for people who train models, he does not fit it, and he should not be presented as if he did.

Follow-up

"So what should his title be?" — Whatever your levelling says for about a year of experience with production ownership of an AI feature. Titles vary too much between companies for the label to settle it.

Grounded in: Title Associate Software Engineer, Pulsar chatbot since August, SYSTEM_PROMPT honesty instruction, pitch, build log

Q13.2 — How much of his Venera AI work was really the consultant's?

sceptical · asked by Hiring Manager · ownership, consultant, credibility, objection

A reasonable suspicion, and the resume invites it by stating the consultant openly rather than hiding them — which is itself a small point in his favour.

The division as he describes it: he is the primary engineer on the feature, built it day to day, and owns both the implementation and the testing. The consultant was engaged because Venera had never built a generative-AI feature and had no internal expertise to check direction against. That is expert support on a first-of-its-kind project, not someone else doing the work.

What is unambiguously his, by his account:

- The implementation — the retrieval over the template corpus, the grounding, the structured-output validation that gates every generated template.
- The evaluation set, covering templates it must generate correctly and requests it must refuse.
- The testing strategy, where his six prior months of QA on the same platform made him the obvious person.

What he would not claim: that the overall direction was his alone with no expert input, or that he has built many of these. This is his first.

An assistant cannot settle this for you, and should not pretend to. The way to settle it is in an interview, with questions that separate ownership from attendance: *what did the validation layer do when a generated template failed the schema? what did the evaluation set catch that surprised him? what did he decide that the consultant disagreed with?* Someone who built it answers those with specifics and with the parts that went wrong. Someone who watched answers in generalities.

Follow-up

"Why mention the consultant at all?" — Because leaving it out would be discovered in the first reference check, and because crediting help is consistent with everything else he publishes.

Grounded in: Pulsar chatbot with external AI consultant, implementation and testing ownership, eval set, prior QA on same platform

Q13.3 — Anyone can build a RAG demo in a weekend. What makes his different?

sceptical · asked by Engineer, Hiring Manager · rag, differentiation, evaluation, objection

He would agree with the premise. A RAG demo is a weekend. A RAG system that is right, and that knows when it is not, is the work — and the difference is almost entirely in things that do not show up in a demo.

What his have that a weekend demo does not:

- **An evaluation suite that fails the build.** 36 cases on this site, 29 that must retrieve the right section and 7 that must retrieve nothing, currently all passing. A demo is judged by whether the answers look good; his are judged by a number that moves when something breaks.
- **A refusal path that is designed, not hoped for.** Retrieval returns empty when nothing matches, and the app declines before any model call. Most demos answer everything, which is exactly what makes them untrustworthy.
- **A failure path.** Nine providers behind one adapter, automatic failover, and extractive answers when every provider is down. When both configured vendors' default models were retired without warning, the site degraded instead of breaking.
- **A record of what went wrong.** Twelve bugs, published with causes. Retrieval that returned the Kotlin app for an LLM question. Documentation that reintroduced the bug it described. A health check that said green while nothing worked.
- **Constraints.** The Pulsar version runs fully offline inside an on-premise product with every generated template validated against a schema it did not design.

And the less glamorous part: he chose *not* to use embeddings where they did not help, and wrote down why. A demo uses the fashionable stack; a system uses the one the corpus justifies.

Follow-up

"Couldn't someone copy the eval idea?" — Easily, and they should. The point is not that it is clever; it is that it is rare, and he does it by default.

Grounded in: eval_retrieval.py 36/36, refusal design, provider chain and fallback, build log, Pulsar constraints, BM25 decision

Q13.4 — His experience is mostly testing. Why would we hire a tester as an engineer?

sceptical · asked by Hiring Manager · qa, career, objection, builder

You would not, if that were the record. It is half of it.

Six months of his career were quality engineering: automated test suites, API validation, release readiness, defect management. Since August he has been building — the primary engineer on a generative-AI feature, owning its implementation. Outside work, everything he has built is software: a

document Q&A system on LangChain and Pinecone, NLP pipelines on Hugging Face, and this site, which is a few thousand lines of retrieval, provider integration, evaluation and front end with nobody asking him to build it.

The deeper answer is that the QA half is not a detour on this particular path; it is the part that makes the building half different. LLM systems fail quietly — a confident wrong answer, no stack trace, nothing red. The engineers who do well with them are the ones whose first instinct is "how will I know if this is wrong?". That instinct is exactly what six months of production QA trains, and it shows in every system he has built: all of them have evaluation sets, all of them design the failure path first.

The risk you are right to probe is whether he defaults to verifying instead of building — waiting for a specification, over-testing, reluctant to ship something rough. The evidence against it is that he owns the implementation of a feature that had no internal precedent, and that his personal projects ship with known gaps listed rather than waiting to be perfect.

Follow-up

"What would make him a bad fit?" — A team that wants speed above everything and treats correctness as a later problem. He will push to measure first, and that will feel like friction there.

Grounded in: Internship QA record, Pulsar implementation ownership since August, self-directed projects, eval sets, failure-path design

Q13.5 — Why should we trust anything this assistant says about him?

sceptical · asked by Hiring Manager, Engineer · trust, grounding, verification, meta

You should not trust it blindly, and the site is designed on the assumption that you will not.

What it does to earn a measured amount of trust:

- **It answers only from his resume and a corpus grounded in it.** The system prompt forbids inventing employers, dates, titles, tools or numbers, and every answer in the interview corpus is tagged with the resume facts it rests on.
- **It cites its sources.** Each answer names the sections it drew from, so you can check the claim against the source.
- **It names the engine that produced the answer,** including when that engine is the local index rather than a model — a fact the site learned to report after once displaying the wrong one.
- **It declines instead of guessing.** Ask it something the resume does not cover and it says so and gives his email. That behaviour is enforced by dedicated test cases.
- **It is instructed not to oversell him,** which is unusual for a self-promotional tool and is the clause most worth noticing.

What it cannot do: verify that the resume itself is true. It is grounded in what he claims, not in an independent record. Every number on the site traces back to his resume, and the resume is his account.

So the right way to use it is as a fast, honest summary of what he claims, with the claims made specific enough to check. The checking — references, a technical interview, asking him to walk through a failure — is yours to do, and nothing here is a substitute for it.

Follow-up

"Can I see the source?" — Yes. The repository is public, including the retrieval code, the evaluation suite and the list of bugs.

Grounded in: SYSTEM_PROMPT rules, source citations, engine reporting (build log bug 11), refusal cases, public repository

Q13.6 — What is the most likely way he fails in his first six months?

sceptical · asked by Hiring Manager · risk, failure-modes, self-awareness, onboarding

The honest candidates, in order of likelihood:

Over-investing in measurement before shipping. His strongest habit — build the eval set first — becomes a weakness when the right move is to ship something rough and learn from real use. On a team moving fast he could spend a week on an evaluation harness for a feature that should have gone out on day two. A manager saying "not this time" would fix it quickly, and he would likely accept it.

Scale he has not seen. He has not worked on a high-traffic system. Problems that only appear at volume — concurrency, capacity, cost at scale, the operational side of serving models — are things he knows about rather than things he has handled. His first incident at real scale will be his first.

Narrow reference points. About a year of experience means fewer patterns to recognise. Senior engineers are largely valuable because they have seen many systems fail in many ways. He will sometimes not recognise a problem that someone more experienced would spot immediately — which is why he names code review from someone senior as the thing he wants most.

Depth outside Python. If the role leans on another language, he is starting closer to scratch than the skills list suggests.

What is *unlikely* to be the failure: claiming something works when it has not been checked, hiding a mistake, or overselling a result. The record is unusually consistent on those.

Follow-up

"How would you mitigate the first one?" — Agree up front, per feature, how much measurement it warrants. He responds well to an explicit bar.

Grounded in: Eval-first methodology, experience level, stated need for senior review, skills list, build log disposition

Q13.7 — Couldn't his metrics be exaggerated? They're all self-reported.

sceptical · asked by Hiring Manager, Recruiter · metrics, verification, honesty, objection

Yes — they are his claims, and an assistant built on his resume cannot independently verify them. It would be dishonest to pretend otherwise.

What can be said about them:

Some are verifiable right now. The 36/36 retrieval evaluation and the 0.04 millisecond retrieval time are about this site, and the repository is public. Clone it, run `python eval_retrieval.py`, and you have checked it yourself. The chunk count and eval score on the About page are computed by the running application at startup, not typed in, precisely so they cannot drift.

Some are specific enough to probe. 150 test scripts, 65% less manual regression time, 78% P1/P2 coverage, 120 endpoints split 75 and 45 across two named products, a 98.3% pass rate, 79 defects of which 61 resolved and 10 critical, zero critical defects across seven builds, a severity-1 503 caught with about four hours of downtime avoided. Vague metrics are easy to inflate; specific ones invite a follow-up question he has to be able to answer. Ask him how the 65% was measured.

Some are carefully scoped. He describes the zero-defect record as a team outcome he contributed to rather than a personal achievement, and the 70% figure on the document Q&A project as a retrieval-time improvement, not an accuracy claim.

One is deliberately absent. The generative-AI feature at Venera — the most important thing on the resume — has no headline number attached, because the measurement that exists is an internal evaluation set rather than a public metric. Inventing one would have been easy.

The references are the real check. A former manager can confirm or deny any of these in a sentence.

Follow-up

"Which number would you verify first?" — The severity-1 incident. It is the most specific, and the easiest for a former colleague to confirm or contradict.

Grounded in: Internship metrics, eval_retrieval.py, computed About stats, team-outcome scoping, absence of a Pulsar metric

Q13.8 — Why hasn't he built anything with larger scale or real traffic?

sceptical · asked by Engineer, Hiring Manager · scale, experience, limits

Because about a year into a career you mostly work on what your employer works on, and his employer builds a commercial media quality control product rather than a consumer platform.

That is a genuine gap and not one he would argue away. The largest things he has worked on are Venera's platform — a cloud product on AWS microservices and an on-premise product deployed at customer sites — and the busiest system he owns is a portfolio that recruiters visit. He has not had to reason about thousands of requests per second, sharded indexes, autoscaling inference, or the cost behaviour of a system under real load.

What partially offsets it, without closing it:

- He has worked inside a production SDLC with release gates, defect management and a live incident, which is the operational discipline that scale depends on.
- The on-premise constraint taught him to design for environments he cannot see or control, which is a different but real kind of difficulty — you cannot hotfix everyone at once or read their logs.
- His systems are designed to degrade rather than fail, which is the habit that matters most once scale arrives.

What would close it is working on something with real traffic, alongside people who have done it before. That is part of what he is looking for in his next role, and a reasonable thing to ask a company about.

Follow-up

"Is scale learnable on the job?" — The concepts, yes, quickly. The judgement comes from incidents, and those take time regardless of how capable someone is.

Grounded in: Venera platform scope (Quasar cloud, Pulsar on-premise), portfolio scale, severity-1 incident, degradation-first design

Q13.9 — What would a former colleague say about him?

sceptical · asked by Recruiter, Hiring Manager · references, colleagues, character

That is a question for a former colleague, not for an assistant trained on his resume. It would be inventing testimony, and it will not do that.

What the record does support, which a reference could confirm or contradict:

- He was converted from a six-month internship to a full-time offer at Venera, on the basis of measured results — a decision his managers made, which is some evidence about how they saw his work.
- After converting, he was trusted with the company's first generative-AI feature, with ownership of both its implementation and its testing. Companies do not usually hand a first-of-its-kind feature to someone they are unsure of.
- His QA work involved coordinating with dev and product teams — defining acceptance criteria, reviewing test strategies, and pulling a same-day hotfix together across people — which is collaborative work, not solo work.

What it cannot tell you: how he is to work with day to day, how he takes criticism, how he behaves under pressure from a person rather than a deadline. Those are exactly what references are for.

If you want to speak to people who have worked with him, ask him directly at jeetendrapatel1711@gmail.com; the resume does not list references, and this assistant will not name anyone.

Follow-up

"Would his manager at Venera give a reference?" — Not something the resume covers. Ask him.

Grounded in: Internship-to-full-time conversion, Pulsar feature ownership, cross-team coordination, contact email

Q13.10 — Why is he leaving a job where he just got the most interesting project?

sceptical · asked by Recruiter, Hiring Manager · motivation, job-search, retention

The resume says he is open to AI Engineer and ML Engineer roles. It does not say why, or on what timeline, and this assistant will not invent his reasons.

What can be said from the record:

- He is currently employed at Venera Technologies, and the generative-AI feature there is his primary work. "Open to roles" is not the same as "leaving".
- The direction of his career is consistent: from QA into building, and from building software into building AI systems. A role that is primarily AI engineering, with larger scale and more experienced people to learn from, would continue that direction. He names exposure to larger systems and review from senior engineers as the things he most wants.

The right way to find out is to ask him, and it is a fair question to ask directly: what would make him stay at Venera, and what would a new role need to offer. His answer will tell you more about fit than anything here.

A reasonable caution in the other direction: someone a year into their career who is open to moving is common and not a warning sign in itself. The question that matters for retention is whether your role offers what he says he wants.

Follow-up

"Would he stay long enough to be worth hiring?" — Not something the resume can answer. Ask what he is looking for over the next two years.

Grounded in: Availability note (open to AI/ML Engineer roles), current role at Venera, career direction, stated wants

Q13.11 — What doesn't he know that he should, for this role?

sceptical · asked by Engineer, Hiring Manager · gaps, learning, self-awareness

Measured against a typical AI engineering role, the gaps he would name himself:

- **Operating model inference at scale** — serving stacks, continuous batching, GPU capacity and utilisation. He knows the concepts and has shipped open-weight models inside an on-premise product, but has not run an inference platform.
- **Production fine-tuning** — he has built the dataset and evaluation side and run experiments, not taken a tuned model into production and maintained it.
- **Rerankers in production** — the single highest-value addition to most RAG systems, and one he has not had to ship because his corpora were small enough to avoid needing one.
- **Agents with real permissions** — his agent graph has no tools and no side effects, which is why it is safe and also why it has not taught him the hard parts: permissions, cost control and failure compounding across steps.
- **Monitoring model quality over time** — drift detection, delayed labels, alerting on quality rather than uptime.

- **Depth beyond Python**, and front-end specialism.

What he has that often goes missing at this level: evaluation discipline, refusal design, failure-path thinking, and a habit of writing down what is not done. Those are harder to teach than a serving framework.

The way he would put it: every item on the gap list is something he has read about and can reason about, and none is something he has been responsible for when it broke. That is the distinction worth probing in an interview.

Follow-up

"Which gap would he close first?" — A reranker in a real retrieval path, because it is the cheapest to learn and has the largest effect.

Grounded in: Skills list, fine-tuning limits, no reranker, no-tool agent, site roadmap, experience level

Part 14 — Skills, Tools and Stack

Questions that check a skills list against reality. The pattern in these answers is to separate three levels honestly — used in production, used in finished projects, and learned or explored — because a flat list of keywords hides exactly the distinction a hiring manager needs.

Q14.1 — How strong is his Python?

engineer · asked by Engineer, Hiring Manager · python, skills, depth

Python is his primary language by a wide margin, and it is the one skill on his list used everywhere: in production at Venera, in every project, and throughout this site.

The evidence of level, rather than the claim:

- platform, and the generative-AI chatbot he is building now.
- **This site's backend** is Python with no framework beyond Flask: a BM25 index written from scratch, with stemming, phrase normalisation and weighted query expansion; a provider layer where seven vendors share one adapter through subclassing; prompt assembly and history sanitisation; and an evaluation harness that exits non-zero for use as a build gate. Plus a PDF generator that auto-fits the resume to one page, and the parser for this interview corpus.
- **Idioms that show up in the code:** dataclasses for records, generators for streaming data, decorators, careful use of None versus empty, type hints on public interfaces, standard-library HTTP to avoid a dependency where one was not needed, and comments that explain *why* at the exact points where the obvious-looking code is wrong.

Where it stops: he has not built a large Python service with many contributors, has not done serious performance work below the level of choosing the right algorithm, and has not worked much with async at scale. Call it solid working Python for application and tooling code, applied with care, at an early-career level of breadth.

The honest way to check it is to read the repository. It is public, and most of it is his own.

Follow-up

"Does he write tests in Python?" — Yes: the retrieval evaluation suite and the checks around the interview corpus, plus six months of pytest and Selenium work professionally.

Grounded in: Skills list (Python primary), internship 150 Python scripts, `core/rag.py`, `core/providers.py`, `build_resume.py`, interview corpus parser

Q14.2 — Which of his AI skills are production-grade and which are learned?

hiring-manager · asked by Hiring Manager, Recruiter · skills, levels, honesty, ai

The skills list is flat; the reality is layered. Sorted honestly:

Used in production, at Venera: - Retrieval-augmented generation, with embedding-based retrieval over a local vector store - Open-weight models (Qwen) running inside an on-premise deployment - Prompt engineering and grounding - Structured output with schema validation - LLM evaluation — writing and maintaining the eval set

Used in finished projects with measured results: - LangChain and Pinecone (document Q&A over 200 pages, ~70% faster retrieval) - OpenAI API, Streamlit - Hugging Face Transformers for classification and summarisation, with a prompt-evaluation loop - BM25 and lexical retrieval engineering, multi-provider LLM integration with failover (this site) - Hybrid retrieval and a LangGraph agent over the interview corpus behind this assistant

Learned, explored, or experimental: - LangGraph beyond the graph on this site — listed on his resume as "exploring" - Fine-tuning with LoRA — dataset and evaluation scaffolding plus experiments,

not a production fine-tune - The breadth of the AI Engineer Bootcamp syllabus, which is in progress

Not claimed: model training from scratch, distributed training, inference serving at scale, research.

The reason to lay it out this way is that the bottom tier is where resumes usually overstate, and a hiring manager who discovers one inflated keyword discounts all the others. The top tier is the part worth testing in an interview, because it is where he has had to make decisions under constraints.

Follow-up

"Is anything on the list there just for keyword matching?" — The items he would defend least are the ones in the third tier, and they are labelled as such here.

Grounded in: Skills list, Pulsar chatbot, Document Q&A and NLP projects, portfolio assistant, certifications, "exploring LangGraph"

Q14.3 — Does he know SQL and databases?

recruiter · asked by Recruiter, Engineer · sql, mysql, databases, skills

SQL and MySQL are on his skills list, supported by one project and one certification.

- **Student Data Entry System** — a Java servlet CRUD application backed by MySQL, where optimising the JDBC queries reduced average database response time by 30%. That is the concrete evidence of working database code, including some query tuning.
- **Database Fundamentals** from LinkedIn Learning — relational modelling and SQL — completed.

What that adds up to: comfortable with relational modelling, writing and tuning queries, and application-level database access. It is not a claim of database administration, large-schema design, or performance work on big datasets.

For AI engineering specifically, the relevant database skills are shifting. Vector stores are the new part — he has used Pinecone on a project and a locally run vector store in production at Venera — and relational databases remain where the metadata, filtering and access control around a retrieval system usually live. A well-built RAG system filters by metadata in the query to the index, not after retrieval, and that is a SQL-shaped way of thinking even when the store is not relational.

He would say plainly that SQL is a supporting skill for him rather than a strength, and that he would expect to deepen it on a team that uses it heavily.

Follow-up

"Has he used Postgres or pgvector?" — Not covered by the resume. Worth asking him directly rather than assuming.

Grounded in: Skills list (SQL/MySQL), Student Data Entry System (-30% DB response time), Database Fundamentals certification, Pinecone, Pulsar vector store

Q14.4 — What testing tools does he know?

recruiter · asked by Recruiter, Engineer · testing, tools, selenium, pytest, postman

This is the most thoroughly exercised part of his toolset, because it was his job for six months.

- **pytest** — Python test framework, used professionally and in his own work.
- **Selenium WebDriver** with the **Page Object Model** — 150 automated scripts on Venera's platform; the POM structure is what keeps a suite that size maintainable.
- **Selenium Grid** — parallel cross-browser execution across three browsers and two operating systems, which cut manual cross-browser effort by about 80%.
- **Postman** — REST API validation across 120 endpoints on AWS microservices, at a 98.3% pass rate.
- **JUnit** — Java unit testing.
- assets under pull-request review in Bitbucket.

And the one that matters most for an AI role, which is not a tool: **evaluation design for non-deterministic systems**. Knowing that you cannot assert exact output from a sampled model, that the deterministic layers around it should be unit tested normally, that quality is measured on a labelled set scored on properties, and that the must-refuse cases are the half people forget. He has built three such evaluation sets.

What he has less of: load and performance testing tools, contract testing, and property-based testing. If a role needs someone to own performance testing, that is new ground.

Follow-up

"Does he still write tests now that he's building?" — Yes. He owns the testing of the feature he built, including the evaluation set, and he argues that is the right arrangement.

Grounded in: Skills list (Testing & Quality), internship metrics, Selenium Grid project, Postman endpoint validation, eval sets

Q14.5 — What cloud and DevOps experience does he have?

recruiter · asked by Recruiter, Engineer · aws, azure, devops, cicd, skills

Working experience within established systems rather than ownership of infrastructure — accurate for his level.

AWS is the substantive one. He has worked since February 2026 on Quasar, Venera's cloud-native media QC product built as microservices on AWS: validating 75 of its REST endpoints, and doing AWS cloud-native release readiness as a defined part of his role.

CI/CD and tooling: Jenkins, Git and GitHub, Bitbucket, Gradle and Maven are on his list. In practice he has worked within pipelines — automated suites running as quality gates, builds and releases as part of release readiness — rather than building them.

Deployment of his own work: this site runs on Vercel's free tier with deployment configuration for Render, Docker on Hugging Face Spaces and Gunicorn as alternatives, deploys automatically when he pushes to GitHub, and has scripts that push environment configuration to the platform without echoing secrets. He tuned the request timeout to the platform's function limit so the model failover can complete inside it.

Azure is on the list at a lower level of depth than AWS.

What he has not done: designed cloud architecture from scratch, written infrastructure as code for a real system, managed IAM or networking at organisation scale, run Kubernetes, or done cloud cost optimisation. If a role needs a DevOps or platform engineer, he is not that.

Follow-up

"Can he containerise an application?" — The repository includes a working Dockerfile for the site. He has not operated containers in production at scale.

Grounded in: Quasar AWS microservices, 75 endpoints, release readiness, skills list (Cloud & DevOps), Dockerfile, deploy scripts, Vercel GitHub integration

Q14.6 — Does he know Java, Kotlin or C++?

recruiter · asked by Recruiter · languages, java, kotlin, cpp

They are on his skills list, each backed by specific work, and he would not claim current depth in any of them.

- **Java** — the Student Data Entry System, a servlet-based CRUD application over MySQL with JDBC query tuning, plus JUnit on the testing side and object-oriented design generally.
- **Kotlin** — his Android internship at NullClass, building and optimising mobile applications, and a real-time chat app with Firebase, secure authentication and sub-second message sync.

- **C and C++** — from his computer science degree.

The honest ranking is Python far ahead, then Java and Kotlin as languages he has shipped something in, then C and C++ as academic. If a role needs production Java or Kotlin, he would be ramping up rather than contributing at full speed from the first week.

The part worth knowing about the older projects: they are why his retrieval evaluation exists. Asked what he had shipped with LLMs, an early version of his assistant returned the Kotlin chat app and the Java CRUD system, because broad synonyms matched every project equally. So the projects are less relevant to an AI role than the lesson they produced.

Follow-up

"Would he rather work in Python?" — For AI engineering, yes, since that is where the ecosystem is. Worth asking him about openness to other languages if the role needs one.

Grounded in: Skills list (Languages), Student Data Entry System, NullClass internship, Real-Time Chat App, build log bug 02

Q14.7 — What is his experience with Git and collaboration tools?

recruiter · asked by Recruiter, Engineer · git, collaboration, tools

Routine professional use.

At Venera: **Bitbucket** for source control with structured pull-request reviews on test assets, and **Zoho** for defect tracking — 79 defects managed, 61 resolved, 10 critical.

For his own work: **Git and GitHub**. This site's repository is public, and its history is written the way he would want a teammate's to be — small commits with messages that explain *why* the change was made, not just what changed. The build log on the site is effectively a set of structured incident notes.

He has worked in structured review as a participant rather than as someone setting review standards, and on a small team rather than a large repository with many concurrent contributors. He has not had to manage complex branching, long-lived release branches, or large merge conflicts across a big team.

What he would say about review, from being reviewed: the most valuable review comment is a "why" rather than a proposed alternative, because it surfaces whether the author had a reason. And the code comments he writes are aimed at a future reviewer — they appear exactly where the correct-looking code is wrong, so nobody "tidies" a deliberate choice back into a bug.

Follow-up

"Is the portfolio history clean?" — Look at it; it is public. Commits are scoped to one concern each and explain the reasoning.

Grounded in: Bitbucket with PR reviews, Zoho defect management, public GitHub repository, build log

Q14.8 — How good is he at explaining technical work to non-technical people?

hiring-manager · asked by Hiring Manager, Recruiter · communication, writing, stakeholders

The best evidence is the site itself, since most of it is technical work written for an audience that is not purely technical.

The About section explains a retrieval system — why BM25 instead of embeddings, why nine providers, why it answers with no API keys — in terms of the decision and its consequence rather than the mechanism. The build log explains twelve bugs as symptom, cause and fix, in plain language, without hiding the embarrassing ones. That is a structure that works for a recruiter and an engineer at the same time.

His professional communication is the other evidence: defining acceptance criteria with product teams, which is translating between what product wants and what engineering can verify; and defect reports, which are only useful if a developer can reproduce the problem from the write-up alone.

The specific habit he has is naming consequences instead of mechanisms. "An invalid template means an operator runs quality control with checks they did not ask for" lands with a product manager where "the output failed schema validation" does not. Same fact, different audience.

What he would not claim: presentation experience to large audiences or executive stakeholders. His communication is mostly written and one-to-one, at the level you would expect a year into a career.

Follow-up

"Can he write documentation?" — The repository's README and deployment guide are his, including a runbook for deploying changes. Judge from those.

Grounded in: About section write-up, build log format, acceptance criteria work, defect management, README and DEPLOYMENT_PLAN

Q14.9 — What tools would he need to learn to be effective here?

hiring-manager · asked by Hiring Manager · onboarding, learning, gaps

It depends on your stack, and this assistant does not know your stack — so the useful answer is which categories are new to him and which transfer immediately.

Transfers immediately: Python, retrieval and RAG design, prompt engineering and grounding, evaluation design, structured output and validation, LangChain, Git-based workflows, AWS basics, working inside a release process.

Short ramp — he has adjacent experience: - A different vector store or embedding model (he has used Pinecone and a local store; the concepts carry over). - A different LLM provider or SDK (he has integrated nine behind one interface). - A different agent framework (he has built with LangGraph). - A different CI system (he has worked within Jenkins-based pipelines).

Genuinely new — expect real learning time: - Inference serving at scale (vLLM, TensorRT-LLM, GPU capacity planning). - Observability for model quality in production, beyond logging. - Infrastructure as code, Kubernetes, or container orchestration. - Any language other than Python at production depth. - Tools specific to your domain.

His learning pattern is visible across his projects: he learns a technique and then immediately builds the thing that tells him whether it is working. The Hugging Face work came with a prompt-evaluation loop; this site came with a retrieval evaluation suite; the Venera feature came with an evaluation set he wrote himself. That habit is probably a better predictor of ramp-up speed than any list of tools.

Follow-up

"How fast did he pick up the Venera AI stack?" — He moved onto it in August and it is his primary work now, with an external consultant available for direction. Ask him what was hardest.

Grounded in: Skills list, provider integrations, LangGraph agent, Pinecone and local vector store, learning pattern across projects

Q14.10 — Has he used Streamlit, Flask or other app frameworks?

recruiter · asked by Recruiter, Engineer · flask, streamlit, frameworks, web

Two, for different purposes, and he chose each for a reason.

Streamlit — on the document Q&A chatbot. It is the right tool for putting an interface on a data or ML workflow quickly: a working UI in a few dozen lines with no front-end code. The trade-off is limited control over layout and behaviour, which is fine for an internal tool or a demo and constraining for a public product.

Flask — for this site. He needed full control over the page, a public API for the chat, server-rendered HTML so the content is readable without JavaScript, and something small enough to deploy as a serverless function on a free tier. Flask with Jinja templates, one stylesheet and a vanilla JavaScript file does all of that with almost no dependency weight. The API has a chat route, a health route with an

opt-in live probe, and a resume route.

The deliberate absence on the list is a front-end framework such as React. For a handful of sections and one chat panel it would have added a build pipeline and a bundle for interactions that are a few event listeners, so he did not use one. That is a proportionality call, not a claim that he could not.

He has not used FastAPI or Django in anything significant, and he would not claim otherwise; for an async model-serving API, FastAPI would be the more natural choice than Flask, and he would expect to pick it up quickly.

Follow-up

"Why not FastAPI for the site?" — The site is a few routes with synchronous calls under a strict timeout, and Flask's footprint suited the free serverless target. For a high-concurrency inference gateway he would choose differently.

Grounded in: Document Q&A Chatbot (Streamlit), portfolio Flask app, app.py routes, vanilla JS front end

Q14.11 — What does his AI Engineer Bootcamp cover, and is it worth anything?

recruiter · asked by Recruiter, Hiring Manager · certification, bootcamp, learning, credibility

It is **The AI Engineer Bootcamp 2026** from 365 Careers on Udemy, currently in progress. The syllabus covers LLMs, NLP, LangChain, LangGraph, Hugging Face, Pinecone, the OpenAI API, RAG and Streamlit.

On its worth, he would give the same answer a sceptical hiring manager would: a course certificate on its own proves very little. It shows that someone chose to learn a subject, not that they can apply it.

What makes it more useful in his case is what it sits next to. The techniques on that syllabus show up in things he actually built and measured:

- RAG, LangChain and Pinecone → the document Q&A chatbot, with a measured ~70% retrieval-time reduction.
- Hugging Face and NLP → the pipelines with a prompt-evaluation loop scored against a labelled set.
- LangGraph → the agent graph running over the interview corpus behind this assistant.
- LLMs and RAG generally → the production feature at Venera, which runs on open-weight models rather than the hosted API the course centres on — so he has had to apply the ideas beyond what the course covered.

He would describe the course as where he learned the vocabulary and the job as where he learned the work. Worth weighting accordingly: low on its own, a reasonable signal of deliberate learning when it lines up with shipped work.

Other certifications, all completed: Android Developer Badges from Google, and Database Fundamentals and Design Thinking from LinkedIn Learning.

Follow-up

"When will he finish it?" — Not covered by the resume. It is listed as in progress.

Grounded in: Certifications list (AI Engineer Bootcamp 2026 in progress; Google and LinkedIn completed), projects mapped to syllabus

Part 15 — Education, Trajectory and Logistics

The closing questions: degree, training, direction, and the practical details a recruiter needs to move forward. Several of these have the same honest answer — the resume does not cover it, ask him — and that is deliberate. An assistant that guesses at someone's notice period or salary expectation does more damage than one that says "I don't know".

Q15.1 — What is his educational background?

recruiter · asked by Recruiter · education, degree, cgpa

He holds a **B.E. in Computer Science Engineering from Chandigarh University, Punjab**, graduating in **June 2026** with a **CGPA of 8.4 out of 10**.

The degree gives him the standard computer science foundation — data structures, algorithms, operating systems, databases, networks, and programming in C, C++ and Java — which is where the lower-ranked languages on his skills list come from.

What is worth noting about the timeline is that his professional work overlaps the end of his degree. He joined Venera Technologies as a graduate trainee in February 2026, before graduating in June, and converted to full-time in August. So the "one year of experience" is real production work that started while he was completing his studies, not a year that began only after them. His Android internship at NullClass came before that.

On how much weight to give the degree: for an engineering role his production record and shipped systems are the stronger evidence, and he would not lean on the CGPA as the main argument. It is a solid result and it satisfies screening requirements that ask for one, which is what it is for.

If you need an official transcript or degree certificate for verification, that is a request to make to him directly.

Follow-up

"Did he specialise in AI at university?" — The resume does not say so. His AI work is professional and self-directed, plus the bootcamp he is working through.

Grounded in: Education: B.E. CSE, Chandigarh University, June 2026, CGPA 8.4/10; Venera from Feb 2026; NullClass internship

Q15.2 — What training has he done outside his degree?

recruiter · asked by Recruiter · certifications, training, learning

Four certifications or courses on record:

- **The AI Engineer Bootcamp 2026** — 365 Careers, on Udemy. **In progress**. Covers LLMs, NLP, LangChain, LangGraph, Hugging Face, Pinecone, the OpenAI API, RAG and Streamlit.
- **Android Developer Badges** — Google. Completed. Kotlin and the Android SDK.
- **Database Fundamentals** — LinkedIn Learning. Completed. Relational modelling and SQL.
- **Design Thinking** — LinkedIn Learning. Completed. Product discovery and problem framing.

The pattern is more telling than any single item: each course lines up with something he built. The Android badges sit next to his NullClass internship and his Kotlin chat app. Database Fundamentals sits next to the MySQL project where he cut average response time by 30%. The AI bootcamp sits next to a RAG system on LangChain and Pinecone, NLP pipelines on Hugging Face, the LangGraph agent behind this assistant, and — most importantly — the production feature at Venera.

He would weight them the same way a sceptical reader would: a certificate shows intent to learn, and the projects show whether the learning stuck. The Design Thinking course is the one that shows up least directly, though framing the refusal behaviour of an AI feature as a product question rather than a

model question is arguably an application of it.

Follow-up

"Is he doing any other AI courses?" — Not covered by the resume beyond the bootcamp. Ask him what he is reading or studying now.

Grounded in: Certifications list, projects corresponding to each

Q15.3 — Where does he want to be in a few years?

recruiter · asked by Recruiter, Hiring Manager · career-goals, trajectory, motivation

The resume states the near-term goal — AI Engineer or ML Engineer roles — and the direction of travel is clear from his record. What his specific five-year ambitions are is for him to say; this assistant will not put words in his mouth.

What the record shows about direction:

- **From verifying to building.** Six months of QA, then ownership of a feature's implementation. Every step has moved toward building things rather than checking them — while keeping the checking.
- **From software to AI systems.** His self-directed work has been entirely LLM systems for over a year, and his job moved in the same direction in August.
- **From small to larger.** The things he names as wanting next are exposure to larger-scale systems and review from more experienced engineers, which is what someone aiming at senior engineering, rather than management or research, typically looks for.

The skills he is deliberately building toward, visible in what he has taken on: retrieval and evaluation depth, agent workflows, and the model-adaptation layer. The gaps he names — inference at scale, production fine-tuning, monitoring model quality over time — read like a list of what a strong mid-level AI engineer would be expected to have, which is a reasonable signal of where he is heading.

If long-term fit matters for the role, ask him directly. The useful question is not "where do you see yourself" but "what would you want to have built and owned in two years".

Follow-up

"Does he want to go into research?" — Nothing on the record suggests so. Everything points to applied engineering.

Grounded in: Availability note, career progression, self-directed AI work, stated wants (scale, senior review), named gaps

Q15.4 — Where is he based, and is he open to relocating or remote work?

recruiter · asked by Recruiter · location, relocation, remote, logistics

He is based in **Noida, Uttar Pradesh, India**, and currently works at Venera Technologies in Noida.

His preferences on relocation, remote, hybrid or on-site work are **not covered by his resume**, and this assistant will not guess at them — getting that wrong would waste both your time and his. Ask him directly at jeetendrapatel1711@gmail.com.

Two things that are on the record and may be relevant:

- He has done fully remote work before: his Android internship at NullClass and his self-directed AI work were remote.
- His current role is on-site in Noida.

If the role is outside the Delhi NCR region or outside India, work authorisation and relocation timing are also questions for him rather than for a resume-based assistant.

Follow-up

"Is he open to international roles?" — Not covered. Ask him.

Grounded in: Location (Noida, Uttar Pradesh), current role location, NullClass remote internship, contact email

Q15.5 — What is his notice period and when could he start?

recruiter · asked by Recruiter · notice-period, availability, logistics

That is not on his resume, so this assistant does not know and will not estimate it.

What is on the record: he is **currently employed** as an Associate Software Engineer at Venera Technologies, and he is **open to AI Engineer and ML Engineer roles**. Being employed usually means a notice period applies, but its length depends on his contract, and guessing would only mislead you.

Ask him directly at **jeetendrapatel1711@gmail.com**, or by phone at **+91 7000957216**. Both are on his resume for exactly this kind of question.

Follow-up

"Is he actively interviewing?" — The resume says he is open to roles. How actively he is looking is for him to say.

Grounded in: Current role at Venera Technologies, availability note, contact details

Q15.6 — What are his salary expectations?

recruiter · asked by Recruiter · salary, compensation, logistics

His resume does not include compensation expectations, and this assistant will not guess at a number. Any figure it produced would be invented, and a made-up salary expectation is exactly the kind of answer that causes real problems later in a hiring process.

Please ask him directly at **jeetendrapatel1711@gmail.com**.

If it helps frame the conversation, the facts that usually inform compensation at this stage are all on the record: about a year of professional experience starting in February 2026, full-time since August, primary engineer on a production generative-AI feature, a B.E. in Computer Science with a CGPA of 8.4, and based in Noida.

Follow-up

"Is he negotiable?" — Not something the resume covers. That is a conversation for him.

Grounded in: Contact email, experience timeline, education, location

Q15.7 — How do I contact him, and what should I send?

recruiter · asked by Recruiter, Hiring Manager · contact, next-steps, logistics

What makes a first message easy to act on, from his side: the role and level, the team and what they build, whether it is primarily LLM product work or something else, the location and working arrangement, and the next step you are proposing. If you want him to prepare something — a walkthrough of the Venera feature, a code review exercise, or a take-home — saying so up front lets him give it proper time.

If you are an engineer rather than a recruiter and want to test the technical claims first, the fastest route is to clone the repository and run `python eval_retrieval.py`, then read the build log on the About page. That will tell you more than a phone screen.

Follow-up

"Can I link someone to a specific answer here?" — Yes. Add `?ask=` and a question to the site's URL, and the page opens with that question already answered.

Grounded in: Contact details (email, phone, LinkedIn, site, GitHub), resume link, `?ask=` parameter, `eval_retrieval.py`

Q15.8 — Would he do a take-home assignment or a technical test?

recruiter · asked by Recruiter, Hiring Manager · interviews, take-home, process

His willingness and availability for one are questions for him; the resume does not say.

What the record suggests about how he would approach one, which is the more useful half of the answer:

- **He would build the evaluation before the solution.** Every system he has built comes with a way to tell whether it works — an eval set, a must-refuse set, a probe. On a take-home that usually means a small labelled set and a number, rather than a demo that looks right on the three inputs he tried.
- **He would write down what is not done.** His site publishes its own roadmap of gaps. Expect a short "what I would do next" section rather than a submission that pretends to be complete.
- **He would explain the trade-offs.** The About page of this site is mostly decisions and their reasons. A take-home submission from him is likely to read the same way.

For an AI engineering role, a take-home that tests those things — "here is a small corpus, build retrieval over it and tell us how you know it works" — would show his strengths better than an algorithm puzzle, and would also expose his gaps honestly, which is the point of a test.

A live alternative that tends to work well: ask him to walk through the Venera feature or this site's retrieval code and explain a decision he would now make differently.

Follow-up

"How long would a take-home take him?" — Depends on the task. Ask him to estimate before starting; how he scopes it is itself informative.

Grounded in: Eval-first methodology, published roadmap, About section decisions write-up

Q15.9 — What makes him different from other candidates at his level?

recruiter · asked by Recruiter, Hiring Manager · differentiation, summary, pitch

Three things, and they are distinct enough to be checkable rather than generic.

He has shipped generative AI into a real product. Most early-career AI candidates have projects. He is the primary engineer on Venera Technologies' first generative-AI feature: a retrieval-augmented chatbot on Qwen open-weight models that generates QC templates for an on-premise product, running entirely inside the customer's deployment, with every generated template validated against a schema before it can be used. Constraints, consequences and a customer — that is a different kind of evidence from a project.

He arrived at AI through quality engineering. Six months of production QA — 150 automated scripts, 120 validated endpoints, zero critical defects across seven builds, a severity-1 failure intercepted before release. That is why every system he builds has an evaluation set, a refusal path and a designed failure mode. For LLM systems, whose characteristic failure is a confident wrong answer with no error anywhere, that instinct is the scarce part.

He is unusually honest about his own record. He publishes twelve of his own bugs. He instructs his own assistant not to oversell him. He credits the consultant who helped on his biggest project. He lists what is unfinished. For a hiring manager, that means the claims that remain are more likely to be true.

What he is not, to be equally clear: senior, experienced at scale, or a researcher. He is roughly a year into his career. The case for him is that he is a year in and already working the way good AI engineers work.

Follow-up

"If I remember one thing about him?" — He ships LLM features with the evaluation set attached.

Q15.10 — Can I see his resume?

recruiter · asked by Recruiter · resume, pdf, logistics

Yes. The resume is a one-page PDF linked from the **Resume** button at the top of every page on this site, and from the Contact section.

One detail worth knowing: the PDF is generated from the same source file as this website and this assistant's knowledge. That was done on purpose — the earlier, hand-maintained version drifted out of step with the site — so the PDF, the pages and the answers here cannot disagree. If you notice a discrepancy anyway, it is a bug, and he would want to hear about it.

It is laid out in a single column with standard section headings and real text rather than images, so applicant tracking systems parse it cleanly.

The resume covers: a professional summary; skills grouped into AI and LLM engineering, languages, testing and quality, and cloud and DevOps; experience at Venera Technologies (current role and internship) and NullClass; selected projects; education; and certifications.

Follow-up

"Is there a longer CV?" — Not published. For more depth, this site's About page and build log, or a conversation with him, go further than a longer document would.

Grounded in: /resume route, build_resume.py generated from profile.py, ATS-friendly single-column layout, resume sections

Q15.11 — Is there anything he would want a recruiter to know that isn't on the resume?

recruiter · asked by Recruiter, Hiring Manager · closing, summary, honesty

This assistant can only speak from what is on the record, so it cannot tell you anything that is not on the resume — and it will not invent something to sound helpful.

What it can do is point to the things on the record that a resume format tends to flatten:

- **The build log.** Twelve bugs he shipped and fixed, with causes. A resume lists what worked; this shows how he handles what did not. It is collapsed at the bottom of the About page.
- **The decisions write-up.** Why BM25 instead of embeddings, why nine providers, why the site answers with no API keys, why one file is the source of truth. Resume bullets say what; this says why.
- **The evaluation suite.** A claim on a resume is a claim. The retrieval evaluation is runnable, in a public repository, and currently passes 36 of 36.
- **The constraints behind the Venera feature.** "RAG chatbot on Qwen" on a resume reads like a keyword. The fact that it runs fully offline inside a customer's on-premise deployment, with every output validated against a schema, is the part that makes it hard.

For anything genuinely beyond the record — what he is like to work with, what he is looking for, what he would want from a team — ask him directly at jeetendrapatel1711@gmail.com. That conversation is the part no assistant should replace.

Follow-up

"What is the one question I should ask him?" — Ask what his evaluation set for the Venera chatbot caught that he did not expect. The specificity of the answer will tell you most of what you need.

Grounded in: Build log, About decisions, eval_retrieval.py 36/36, Pulsar constraints, contact email